HOCHSCHULE
RAVENSBURG-WEINGARTEN
UNIVERSITY
OF APPLIED SCIENCES

Embedded Computing

Lecture notes

Prof. Dr. Markus Pfeil
with kind permission of Prof. Dr.-Ing. Franz Brümmer

Winter semester 2026/27

Contents

1 Introduction **4**
- 1.1 What is an Embedded System? . . . 5
- 1.2 Historical Background . . . 6
- 1.3 Characteristics of Embedded Systems . . . 7
- 1.4 Software Development and AI . . . 8
- 1.5 About this Lecture . . . 14

2 Modeling **15**
- 2.1 General Requirements . . . 15
- 2.2 State Oriented Models . . . 19
- 2.3 Implementing a State Machine . . . 23
- 2.4 Implementing StateCharts . . . 27
- 2.5 Activity Oriented Models . . . 31

3 Cyber-Physical Systems **35**
- 3.1 Overview . . . 35
- 3.2 Modelling with Modelica . . . 37
- 3.3 Case Study: a Digital Twin . . . 42

4 Processors **46**
- 4.1 Processors for Embedded Systems . . . 46
- 4.2 Microcontroller Families . . . 49
- 4.3 Inside a Microcontroller . . . 50
- 4.4 Application Processors and Selection . . . 52

5 Programming **54**
- 5.1 From Source Code to Firmware . . . 54
- 5.2 Programming in Assembly . . . 56
- 5.3 C for Embedded Systems . . . 57
- 5.4 Rust for Embedded Systems . . . 59
- 5.5 Python and MicroPython . . . 61
- 5.6 Debugging . . . 63

6 Peripherals and Interfaces **67**
- 6.1 Peripherals of a Microcontroller . . . 67
- 6.2 Digital Inputs and Outputs (GPIO) . . . 69
- 6.3 Interrupts . . . 74
- 6.4 Timers, PWM and Watchdog . . . 76
- 6.5 Analog Input (ADC) . . . 79
- 6.6 Serial Interfaces . . . 80

7 Networking (TCP/IP) **87**
- 7.1 Networks in Embedded Systems . . . 87
- 7.2 Link Layer: Ethernet and Wi-Fi . . . 90
- 7.3 Network Layer: IP . . . 91

7.4 Transport Layer: UDP and TCP . . . 94
7.5 Socket Programming . . . 97
7.6 Application Protocols: HTTP and MQTT . . . 98
7.7 Security and Analysis . . . 102

8 Operating Systems **104**
8.1 Why an Operating System? . . . 104
8.2 Tasks and Scheduling . . . 105
8.3 Communication and Synchronisation . . . 108
8.4 Case Study: a Controller and its Simulated Environment . . . 112
8.5 Operating Systems for Embedded Systems . . . 117

References **123**

1 Introduction

Most computers are not on a desk. They are built into washing machines, cars, heating systems, medical devices and smartphones, and they control these devices without the user ever noticing that a computer is involved. Such embedded systems have their own requirements. They must react in time, work for years without a reset, run on a battery, cost a few euros at most, and they must never endanger anybody.
This lecture covers the whole path from the requirements to a working device. It starts with models and cyber-physical systems, continues with processors and their programming in assembly, C, Rust and MicroPython, then treats the peripherals and interfaces of a microcontroller, networking and real-time operating systems. All examples run in the browser-based simulator Wokwi or on Replit, so they can be tried without any hardware.

1.1 What is an Embedded System?

Definition

Definition 1.1 An **embedded system** is a computer system that is part of a larger technical system and performs dedicated functions in it – usually with requirements on timing, energy, reliability and cost.

- the user sees the device (a car, a coffee machine), not the computer in it
- interaction with the environment through **sensors** and **actuators**, often without a screen or keyboard
- the software is fixed for the purpose ("firmware"), updates are rare and must be safe

Embedded systems are everywhere

- tens of billions of microcontrollers are produced every year – many times more than PC and server processors
- a modern car contains 70 to over 100 control units (engine, brakes, airbag, lights, seats, infotainment)
- household: washing machine, heating, smart meter, router, TV, smart speaker
- industry: programmable logic controllers, robots, drives, sensors with network interfaces
- medicine: pacemakers, insulin pumps, ventilators
- communication: smartphones (several processors), base stations, satellites

Structure of an embedded system

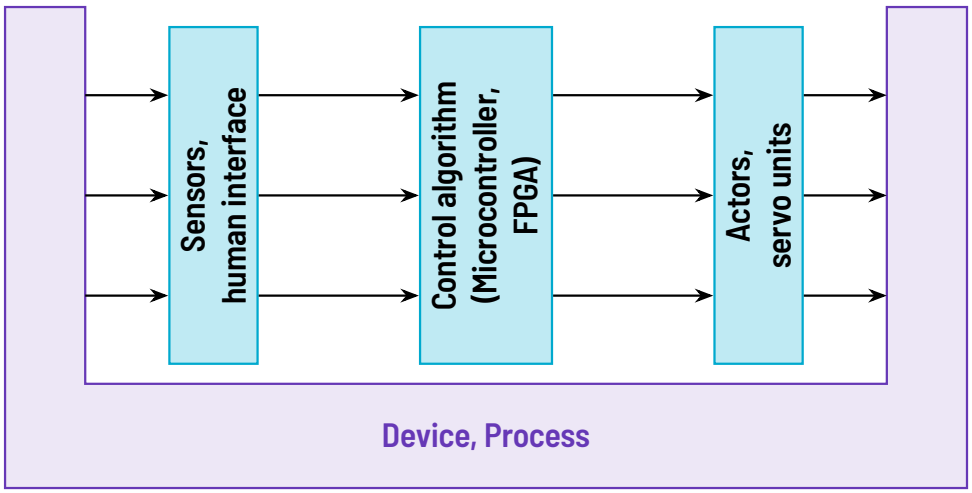


Figure 1.1.1: Block diagram of an embedded system

A first example: a coffee machine in the simulator

```
1 void loop() {
2   float t = readTemperature();                    // sensor: 10 mV per degC on A0
3   if (t < 91.0) digitalWrite(PIN_HEATER, HIGH);   // actuator: two-point control
4   if (t > 94.0) digitalWrite(PIN_HEATER, LOW);
5   bool ready = t >= 90.0 && !brewing;
6   digitalWrite(PIN_READY, ready);                 // user interface
```

```
  if (digitalRead(PIN_BUTTON) == LOW && ready) {    // "make coffee"
    brewing = true;  brewStart = millis();  digitalWrite(PIN_PUMP, HIGH);
  }
  if (brewing && millis() - brewStart >= 20000) {   // 20 s = one cup
    brewing = false;  digitalWrite(PIN_PUMP, LOW);
  }
}
```

- the **device** is simulated as well: a Wokwi **custom chip** with a physical model of the boiler, $mc\dot{T} = P_{heater} - k\,(T - T_{room}) - \dot{m}_{pump}\,c\,(T - T_{room})$
- the program knows the machine only through pins – like the real one
- Arduino Uno, runs in the browser: `examples/intro/coffee` (Rust: `coffee-rust`)

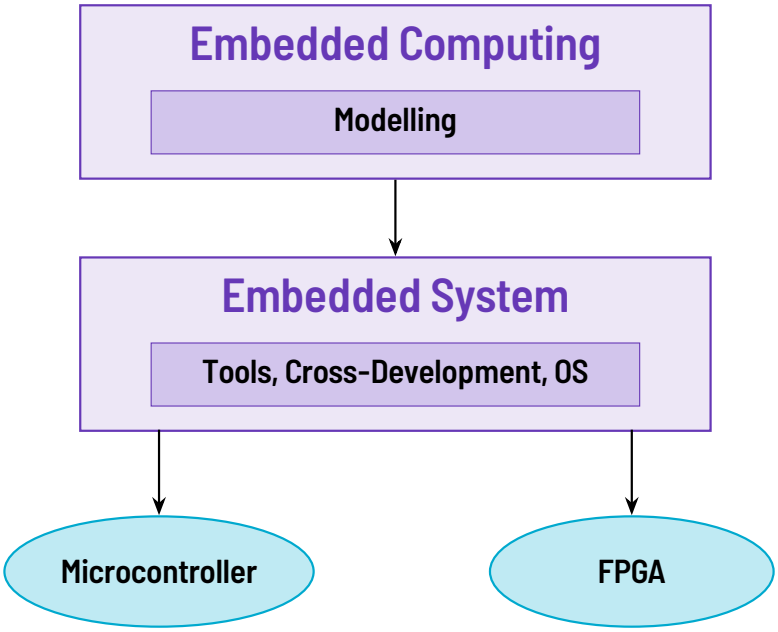


Figure 1.1.2: Embedded computing and the embedded system around it

1.2 Historical Background

From room-sized computers to the microcontroller

- the first computers (Zuse Z3 1941, Mark I 1944) filled rooms; the Apollo Guidance Computer (1966) was one of the first computers built into a vehicle
- the microprocessor (1971) and the microcontroller with memory and I/O on one chip (1974) made embedded systems cheap
- today a 32 bit microcontroller with Wi-Fi costs about one euro

Computer generations

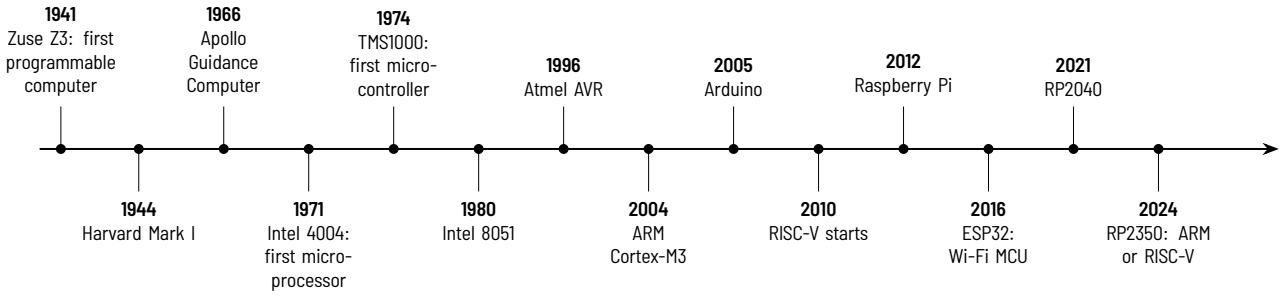


Figure 1.2.1: Milestones on the way to today's embedded systems

generation	period	technology	typical systems
1	1945 – 1955	vacuum tubes	mainframes
2	1955 – 1965	transistors	mainframes, process control
3	1965 – 1975	integrated circuits	minicomputers
4	1975 –	LSI / VLSI, microprocessors	workstations, PCs, microcontrollers
today		systems on chip, many cores	smartphones, IoT, AI accelerators

- the trend continues: more functions on one chip, less energy per operation, lower price
- **central vs. distributed**: instead of one central computer, many small connected controllers close to the sensors and actuators

1.3 Characteristics of Embedded Systems

Dependability

- **reliability**: probability that the system works correctly for a given time, provided it worked at the start
  - operation for years without restart; recovery from errors by itself (watchdog)
- **maintainability**: how quickly the system works again after an error
- **availability**: probability that the system works at a given time
- **safety**: the system does not endanger people or the environment – even if it fails
- **security**: the system resists attacks; connected devices must be protected and updatable

> Even perfectly designed systems can fail if the assumptions about the workload and possible errors turn out to be wrong.
> Dependability must be considered from the very beginning – it cannot be added afterwards.

Efficiency

- **energy**: battery operation for months or years, sleep modes, energy harvesting
- **code size and memory**: kilobytes instead of gigabytes; no hard disk
- **run time**: real-time requirements – a correct answer that arrives too late is wrong
- **size and weight**: the electronics must fit into the device

- **cost**: in a series of millions every cent counts; the development costs are spread over the number of devices

Further characteristics

- **reactive**: continuous interaction with the environment, at its pace (chapter Modelling)
- **real time**: deadlines are part of the specification (chapter Operating Systems)
- **dedicated**: one purpose, known in advance – hardware and software can be optimised for it
- **heterogeneous**: hardware and software, analog and digital, different processors and networks in one system
- **long lifetime**: industrial devices and cars are used for 10 to 20 years – software must be maintained that long
- **cyber-physical**: computation and physics are coupled in feedback loops (chapter Cyber-Physical Systems)

1.4 Software Development and AI

AI tools write code. Since 2021 they have gone from completing the current line to working on whole projects: they edit several files, run the compiler and repair their own mistakes. Most developers use them today, and so did we when we revised this lecture.
Two questions follow for an embedded systems course. What changes in the daily work? And what does an engineer still need to know when a machine writes much of the code? The studies disagree on how much time AI saves. Some measured faster delivery, one controlled experiment measured the opposite. On the second question they are clearer. The generated code is often almost right, and the last errors are found by people who understand the hardware, read the datasheet, write tests and know how to debug. We found the same while preparing this lecture. So it still teaches these things in detail, with or without AI.
The section also maps where AI is used along the development of an embedded system today – from code and tests to PCB layout – and where it is not yet: on the lab bench, in timing guarantees, in certified safety software and in the final responsibility for a product. It ends with the skills that matter when working with AI, the skills that must stay active without it, and the rules for using AI in this lecture.
The job market reacts as well. Employers now expect graduates to work with AI tools as a matter of course, and they pay more for these skills. At the same time there are fewer of the simple entry-level tasks that used to train beginners, so new engineers have to judge, test and debug earlier in their career. The second half of this section collects current figures and the skills that job advertisements ask for.

From code completion to agents

- **code completion** (e.g. GitHub Copilot, 2021): the tool proposes the next lines while you type
- **chat assistants** (since 2022): questions in natural language, explanations, code snippets to copy
- **agents** (since about 2025, e.g. Claude Code, Copilot agent mode, Cursor, OpenAI Codex): the tool reads the repository, changes several files, runs compiler and tests and repairs its own mistakes
- "vibe coding" (A. Karpathy, 2025): accept generated code without reading it; fine for a throw-away prototype, not for a product
- the developer types less and reviews, tests and decides more

How a language model writes code

- a large language model continues a text with the most probable next words, learned from huge amounts of public code and documentation
- good at frequent patterns: boilerplate, explanations of code, translation between C and Rust, test cases, summaries of documentation
- weak at
  - rare or new things: a specific chip variant, the latest version of a HAL, an erratum in the datasheet
  - things that are not visible in the code: timing, electrical limits, the state of the hardware
  - it cannot tell when it is wrong; an invented function or register looks just as convincing as a real one ("hallucination")
- the answer is only as good as the context: give it the datasheet section, the existing code, the exact error message and the tool versions

Where AI is used in embedded development

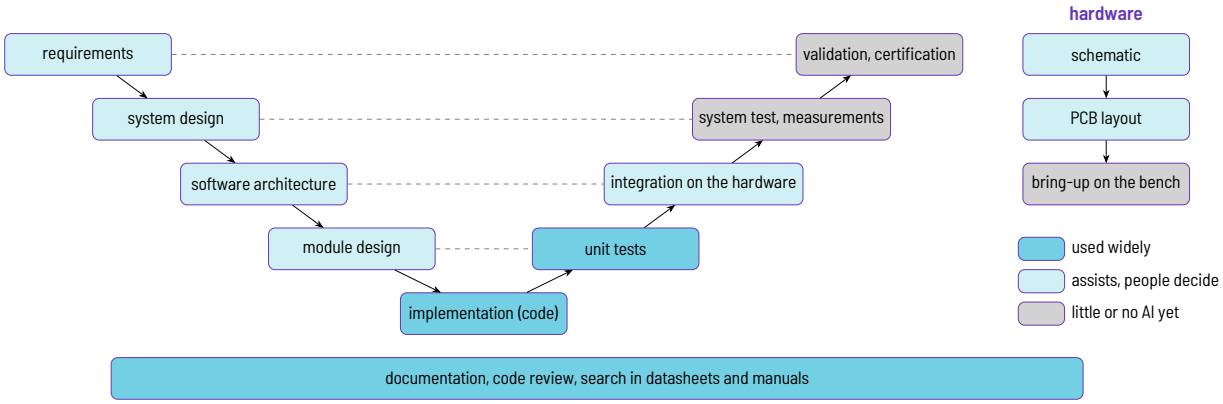


Figure 1.4.1: AI along the V-model of an embedded system (own assessment, 2026)

- AI is strongest where its result can be checked quickly and automatically: code that compiles and passes tests, text that a person reads
- it is weakest where the physical world, measurements or responsibility come in: the right side of the V and the lab bench

AI in the development process today

- **code**: completion, chat and agents – by far the most common use [17]

- **tests**: unit tests, stubs for hardware drivers, simulation scenarios (like the Wokwi tests in this lecture)
- **reviews and static analysis**: review comments on pull requests, fix proposals for analyzer warnings
- **knowledge**: questions about datasheets and reference manuals, summaries, porting between C and Rust or between SDKs
- **debugging**: explaining crash dumps, logs and traces – the AI proposes hypotheses, the engineer checks them on the target
- **hardware**: schematic assistants (e.g. Flux), AI placement and routing of PCBs (e.g. Cadence AI-legro X AI, Quilter); checked by design rules and engineers
- **chip design**: machine learning optimises the physical implementation of chips (e.g. Synopsys DSO.ai, Cadence Cerebrus)
- and **inside the product**: edge AI on microcontrollers – keyword spotting, anomaly detection on motors, small vision models

What studies show

study	finding
Stack Overflow 2025 [17] 49 000 developers	84 % use or plan to use AI tools 46 % distrust the accuracy; 45 % find debugging AI code slow
DORA 2025 [18] 5 000 professionals	90 % use AI at work; more changes delivered, but delivery is less stable; AI amplifies a team's habits
METR 2025 [19] randomised trial	experienced developers 19 % **slower** with AI, although they felt 20 % faster; 2026: probably faster, uncertain [20]
Veracode 2025 [21]	45 % of generated code samples had security flaws
Anthropic 2026 [22] randomised, 52 people	learning with AI: 50 % in a test without AI, without AI: 67 %; largest gap in **debugging**

- feeling faster is not being faster; measure it

AI in embedded systems

- embedded code depends on details that are rare in the training data: chip variants, register maps, SDK versions, wiring
- benchmark with 27 language models on the ESP32 [23]
  - simple sensor applications: 19 to 23 models delivered working code
  - complex applications (sensor, cloud database, dashboard): only 3 to 5 models
  - most frequent failure: libraries and functions that do not exist, or wrong use of real APIs
- benchmark EmbedAgent (Raspberry Pi Pico, ESP32, Arduino, 126 tasks) [24]: the best model solved about 56 % of the tasks; porting code from Arduino to ESP-IDF succeeded in fewer than 30 % of the cases
- many errors show up only on the hardware or under real timing; that the compiler accepts the code proves little

Where AI is not (yet) active

- **on the bench**: bring-up, oscilloscope and logic analyzer, EMC and thermal tests – the AI does not see the hardware, it only sees what we give it

- **timing guarantees**: worst-case execution time and schedulability come from measurements and static analysis tools, not from a language model
- **certified safety software** (ISO 26262, IEC 61508, DO-178C)
  - tools need a confidence or qualification argument; LLM output is not deterministic
  - generated code is treated like hand-written code: same reviews, tests and coverage
  - qualified code generators exist, but they are deterministic and work from models (e.g. SCADE, Simulink Embedded Coder)
- **responsibility**: release decisions, safety cases and liability stay with people and companies – the EU Product Liability Directive (2024) includes software
- **AI inside safety functions** is still rare: ISO/PAS 8800 for AI in road vehicles appeared in 12/2024 [30]; the AI Act obligations for AI in regulated products were postponed to 08/2028 [31]
- and where the data must not leave the company: confidential code, customer projects, export control

Experience from preparing this lecture

- we wrote the examples of this lecture with an AI assistant, then compiled each of them and ran it in the Wokwi simulator
- the tests found errors in code that compiled without warnings, for example
  - a FreeRTOS demo that could not show the race condition at all, because the first task finished before the second one was created (chapter Operating Systems)
  - an MQTT client that the public test broker sometimes rejected, because a packet was sent in two TCP segments (chapter Networking)
  - a PWM calculation that assumed a 125 MHz system clock, while the Arduino core runs the RP2040 at 200 MHz
  - a delay that took 4 μs instead of 1 ms in the simulator; the debugger and a measurement found it (chapter Programming)
- to find them we needed to know how the scheduler starts tasks, how TCP splits data and how the clocks are configured, plus one long debugging session; tests that check the behaviour caught what the compiler could not

How the development environment changes

- more code is **read and reviewed** than written by hand
- specifications, interface descriptions and test cases become the input for the AI, so precise requirements pay off twice
- automatic checks become more important: compiler warnings, static analysis, unit tests, simulation (Wokwi), continuous integration
- projects are organised so that tools find the context: clear structure, documented interfaces, README files
- the engineer stays responsible: safety standards (IEC 61508, ISO 26262) require traceable requirements, reviews and tests, no matter who or what wrote the code
- new questions: confidentiality (company code in external services), licenses of generated code, reproducibility

AI and the job market

source	finding
WEF Future of Jobs 2025 [25] 1 000 employers, 55 economies	AI and big data are the fastest-growing skills, followed by networks and cybersecurity; 39 % of core skills change by 2030
PwC AI Jobs Barometer 2026 [26] over 1 billion job ads	jobs with AI skills pay on average 62 % more; US entry jobs exposed to AI ask 7 times more often for senior skills
Stanford, "Canaries" 2026 [27] US payroll data	age 22–25 in AI-exposed jobs: employment 19 % below less-exposed peers; fewer hires, no gap for experienced staff; no general job loss
Bitkom 2026 [28] 602 companies	79 000 open IT positions in Germany (2023: 149 000); 63 % expect AI skills in **all** IT jobs; 55 % expect classical entry tasks to disappear

- AI shifts the work more than it removes it: jobs where AI assists people grow, jobs where it replaces routine tasks hire fewer beginners

What employers ask for

- working with AI tools is a basic requirement, not a speciality: giving context, splitting tasks, reviewing and testing the result
- judgment from the first day: beginners do less routine work and more checking, integrating and deciding
- governance: 74 % of the companies in the Bitkom survey expect steering and controlling AI to become more important [28]
- AI literacy is a legal duty: since 02/2025 the EU AI Act (Art. 4) requires employers to train staff who use AI, supervised since 08/2026 [29]
- in embedded systems in particular
  - security and update processes: the EU Cyber Resilience Act requires vulnerability reporting since 09/2026 (chapter Networking)
  - edge AI: running small neural networks on microcontrollers (TensorFlow Lite Micro, quantisation, memory and timing budgets)
  - functional safety: traceable requirements and tests, also for generated code
- and still: analytical thinking, which 7 of 10 employers in the WEF survey call the most important core skill [25]

Preparing for your career

- learn the basics so well that you notice when the AI is wrong; this is what distinguishes you from the tool
- build projects that you can explain in an interview line by line, including the bugs you found
- show that you can verify: tests, simulation, measurements, reviews
- learn one agentic tool properly, including its limits, and the rules of your employer on confidential code
- keep learning: 61 % of the companies expect more pressure on IT staff to train continuously [28]

Critical skills for working with AI

- **specifying**: a precise task with interfaces, limits and acceptance criteria – a good prompt is a good specification
- **giving context**: chip and board, SDK version, datasheet section, memory, timing and power budget
- **verifying**: tests designed independently of the code, simulation, measurement on the target

- **reviewing**: spotting the errors that look right – races, interrupts, missing `volatile`, overflow, resource limits
- **knowing the tool**: how language models fail (invented APIs, outdated versions), what an agent may change and run
- **protecting data**: no secrets, no confidential or personal data in external services; licenses of generated code
- **thinking in systems**: hardware, software and the physical process together – as in the case study of chapter Operating Systems

Skills that must stay active – also with AI

- use it or lose it: practise regularly **without** AI – when the tool is wrong, unavailable or not allowed, you need these skills
- computer architecture, memory, interrupts and concurrency: without them you cannot tell wrong code from right code
- reading datasheets and reference manuals, the only reliable source for registers, timing and electrical limits
- specifying a problem so clearly that a colleague, or an AI, can solve it: requirements, state machines, interfaces
- reading code critically; you commit only what you understand
- testing: designing tests, simulating, measuring, and knowing when something is really done
- debugging with debugger and logic analyzer, step by step; this was the largest gap in the Anthropic study
- asking what happens if the system fails or is attacked
- seeing hardware, software and physics as one system
- explaining and documenting decisions for the team

Responsible use of AI in this lecture

- **learn first**: try a problem yourself, then use AI to ask *why* – in the Anthropic study, learners who asked conceptual questions learned the most [22]
- **understand**: hand in only code and text that you can explain line by line – we will ask
- **verify**: check every API and register against the documentation, test in Wokwi or on the hardware – "it compiles" is not "it works"
- **be transparent**: state which tool you used and for what (one line in the lab report is enough)
- **protect data**: no personal data, no exam material and no confidential company code in external AI services
- **follow the rules**: what is allowed in graded work is announced with each assignment; the examination rules of the university apply
- **the lecture does the same**: its examples were written with an AI assistant, then compiled and tested in the simulator – report the errors you still find

Exercises

- Let an AI assistant write the debouncing example of chapter Peripherals and Interfaces in C. Review the code, run it in Wokwi with a bouncing button and list all problems you find.
- Ask an AI for the address of the register `GPIO_OUT_XOR` of the RP2040 and for the meaning of its bits. Check the answer in the datasheet.

- Solve one exercise of chapter Programming without AI and one with AI. Compare time, result and what you can explain afterwards.
- Read five current job advertisements for embedded software engineers. List which AI-related skills they ask for and which of them this lecture covers.

1.5 About this Lecture

Contents

chapter	topic	platform of the examples
2	Modelling: state machines, data flow	Pico (coffee machine FSM)
3	Cyber-physical systems, Modelica	OpenModelica, Python
4	Processors: AVR, Cortex-M, RISC-V	compiler output for 4 processors
5	Programming: build process, assembly, C, Rust, MicroPython, debugging	Pico, PC
6	Peripherals and interfaces: GPIO, interrupts, timers, ADC, UART, I^2C, SPI	Pico
7	Networking: Wi-Fi, IP, TCP/UDP, sockets, HTTP, MQTT	ESP32-C3, PC
8	Operating systems: FreeRTOS, Embassy, embedded Linux	ESP32-C3

- every example exists in **C** and in **Rust** (some also in Python); all are in the folder `examples/` of the lecture repository

Tools

- **Wokwi** (`https://wokwi.com`): simulator for Raspberry Pi Pico, ESP32, Arduino and STM32 in the browser – with LEDs, buttons, sensors, displays, logic analyzer and Wi-Fi
  - C (Arduino core + SDK) and MicroPython run directly in the browser
  - Rust: build locally with `cargo`, simulate with the Wokwi extension for VS Code
- **Replit** (`https://replit.com`) or your own PC: programs for the PC side (sockets, Python, Rust tests)
- **Rust**: `rustup`, `cargo`, targets `thumbv6m-none-eabi` (Pico) and `riscv32imc-unknown-none-elf` (ESP32-C3)
- **git**: all sources of the lecture and the examples are under version control
- real hardware is optional: Raspberry Pi Pico and ESP32-C3 boards cost a few euros, and the same programs run on them

Exercises

- List ten embedded systems in your home. For each: which sensors, which actuators, which communication?
- Which of the characteristics (dependability, efficiency, real time, cost) is most important for an airbag control unit, a smart light bulb and a heart pacemaker?
- Open `https://wokwi.com`, start a new Raspberry Pi Pico project and let the on-board LED blink.
- Run the coffee machine (`examples/intro/coffee`). Press the button while the water is still cold. Then change the switching points to 85 and 95 °C: how does the temperature curve change, and how often does the heater switch?

2 Modeling

2.1 General Requirements

A small embedded project can start directly with programming: one microcontroller, a few hundred lines of code, a test on the bench. Most products are larger. Several developers, hardware and software designed in parallel, safety requirements and a lifetime of many years make a more structured approach necessary. Models describe the system before it is built. They make it possible to discuss and check the behaviour, to simulate it, to generate code and documentation from it, and to test the implementation against it.
No single kind of model covers all aspects of an embedded system. This chapter introduces the most important ones: state based models, which describe how a system reacts to events, and activity oriented models, which describe data flow and the order of operations. We implement a state machine in C and in Rust, then two StateCharts with hierarchy, timers and concurrency, and run all of them on the Pico in Wokwi.

- Classification of systems from the point of view of programming

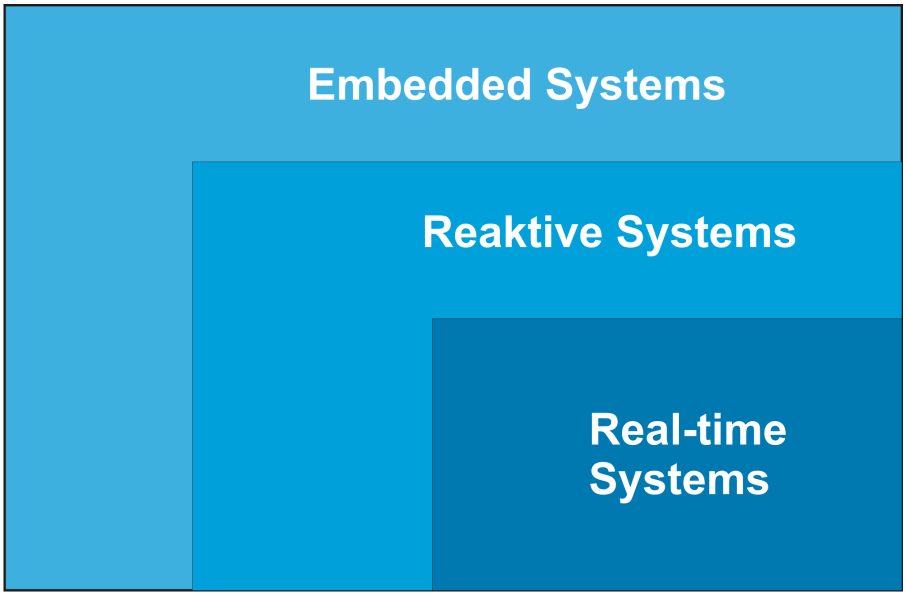


Figure 2.1.1: Classification of embedded systems and real-time systems

Definition 2.1 An **embedded system** is a computer system designed to perform a dedicated function within a larger mechanical or electronic system. Unlike a general-purpose computer, it performs one or a few pre-defined tasks, usually with specific requirements on timing, energy and reliability.

Definition 2.2 "A **reactive system** is one which is in continual interaction with its environment and executes at a pace determined by that environment" [Bergé, 1995]

- interaction with the environment through sensors and actuators
- minimum and maximum response times are part of the specification
- the system may consist of parallel and distributed parts (subsystems)

Definition 2.3 A **real-time system** must deliver a correct result within a given time (its deadline). The correctness depends not only on the result but also on the time at which it is available.

- **hard** real time: missing a deadline is a failure of the system
- **soft** real time: missing a deadline reduces the quality of the service
- a real-time operating system helps to build a real-time system, but it is neither necessary nor sufficient (see chapter Operating Systems)

Requirements for models

- concurrency → real-time systems are usually concurrent
- synchronisation and communication → modelling the communication between distributed parts

- timing → deadlines, periods, response times
- programming elements → arithmetic operations, loops and function calls should be expressible
- non-standard I/O devices → switches, analog and digital I/O
- non-functional properties → e.g. fault tolerance, availability, EMC, usability, extendibility, lifetime, power consumption
- co-design → modelling of hardware and software functions

but

- models must be readable
- models should be hierarchical → humans can only keep about 5 to 7 objects in mind at the same time

> A model should be as simple as possible, but not simpler. [attributed to A. Einstein]

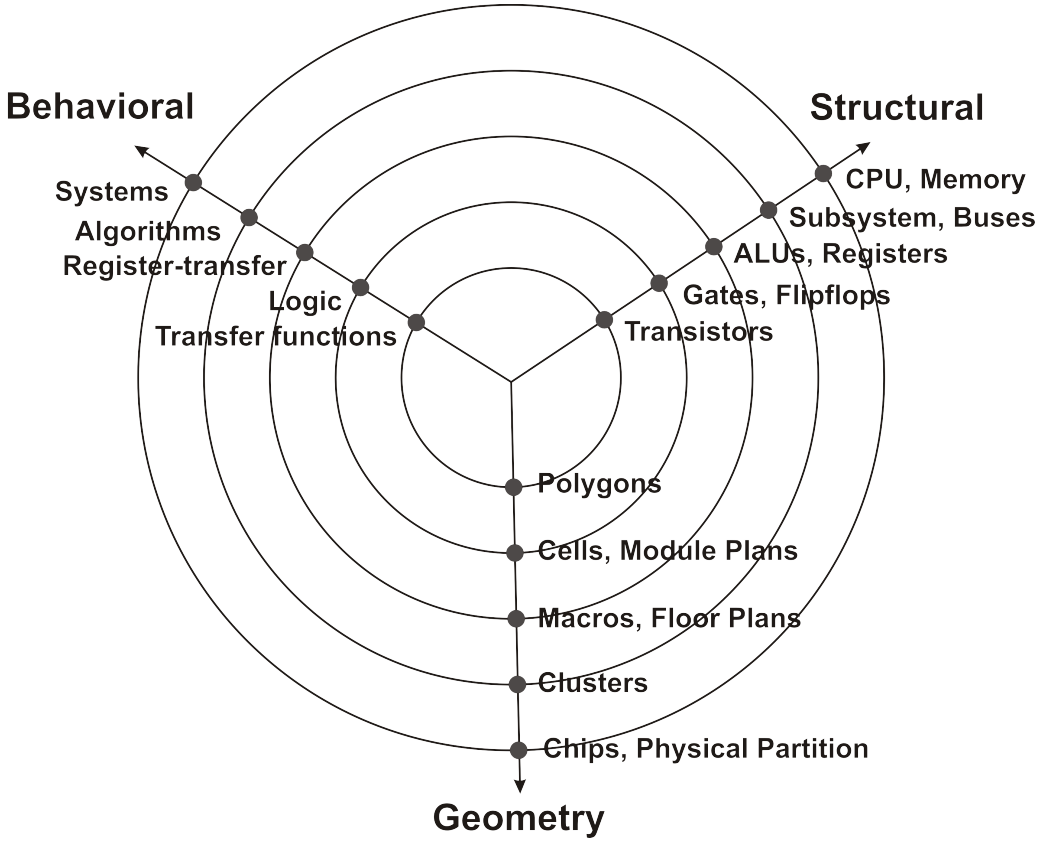


Figure 2.1.2: Y-chart of Gajski and Kuhn

- a model can move between different levels of abstraction in the Y-chart
- the Y-chart (Gajski and Kuhn, 1983) was developed for hardware design
- concurrency and timing are difficult to express in it

Design methods

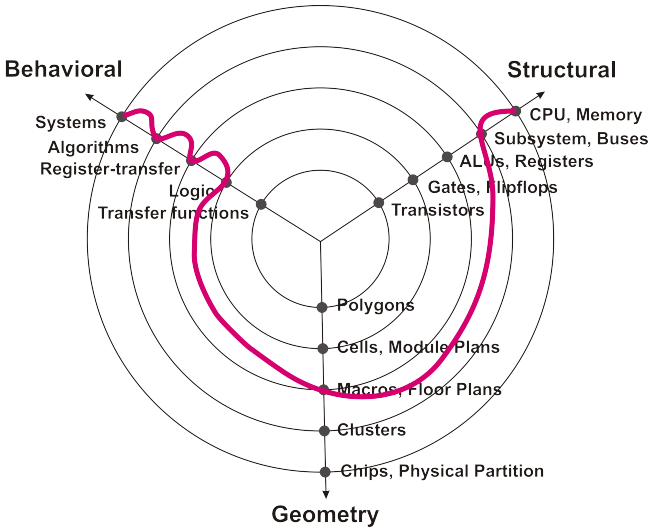


Figure 2.1.3: Different levels in a Y-chart

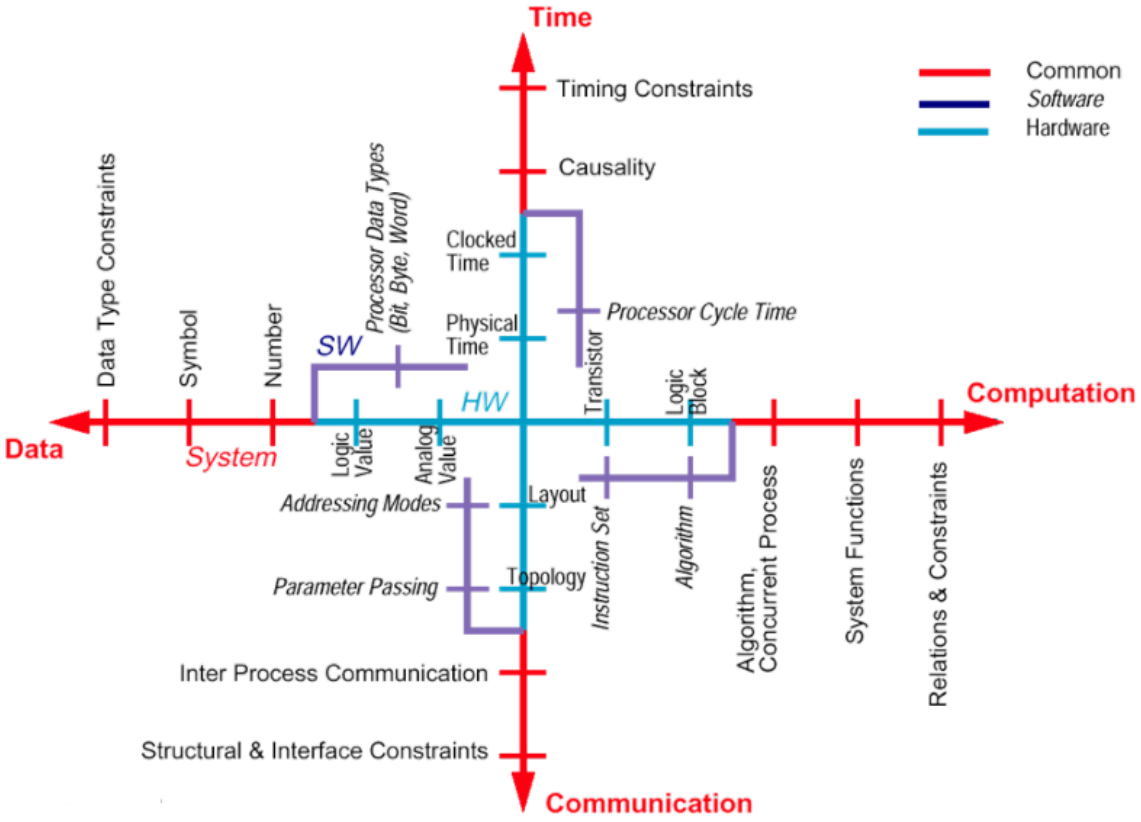


Figure 2.1.4: Hardware and software co-design

- capture and simulate
  - specify the system, draw block diagrams (details not yet specified)
  - simulate and verify
- describe and synthesise
  - generate hardware and software from the description with tools
  - logic synthesis from hardware description languages (VHDL, Verilog)
  - code generation from models (e.g. Simulink, state machine tools)
- specify – explore – refine
  - tools support the exploration of alternatives (e.g. which functions in hardware, which in software)
  - the chosen solution is refined step by step

Different views of a model

- behavioural view
  - treats the system or parts of it as black boxes
  - focuses on states, processes, procedures
- structural view
  - shows how the system is built from components and how they are connected
  - processors, boards, modules, buses
- physical view
  - physical properties of the components: size, placement, heat, connections

Classes of models

- no single model covers all requirements of an embedded system
- each class of models focuses on certain aspects
  - state oriented (state machines, StateCharts)
  - activity oriented (data flow graphs, flow charts, sequence diagrams)
  - structure oriented (block diagrams, component diagrams)
  - data oriented (entity relationship models)
  - heterogeneous (combinations, e.g. UML / SysML)

2.2 State Oriented Models

Most embedded systems react to events: a button is pressed, a value exceeds a limit, a message arrives. What they do depends on the history, which a state machine summarises in its current state. State machines are easy to draw, easy to check (has every state a reaction to every input?) and easy to implement. For large systems they become unwieldy, and hierarchical state machines (StateCharts) were invented for this case.

- classical automata (FSM, finite state machine) are state oriented models
- two variants: Moore and Mealy machines

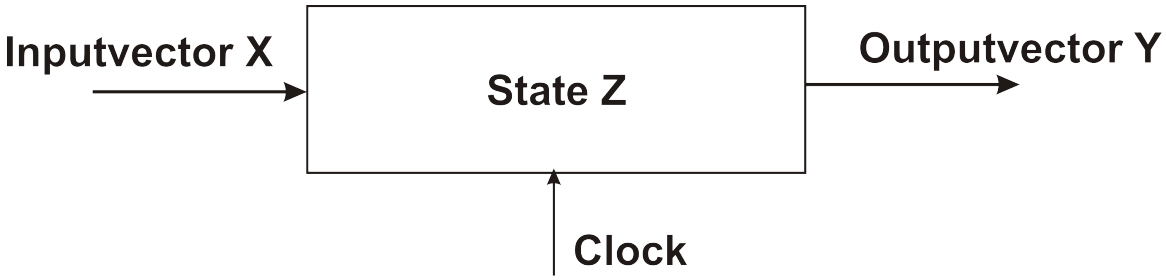


Figure 2.2.1: Block diagram of an FSM

$$\text{Moore machine:} \quad Y = \lambda(Z), \quad Z^+ = \delta(X, Z)$$
$$\text{Mealy machine:} \quad Y = \lambda(X, Z), \quad Z^+ = \delta(X, Z)$$

λ : output function
δ : transition function, computes the next state Z^+

- Moore: the output depends only on the state – stable outputs, but one clock later
- Mealy: the output depends on state and input – faster reaction, fewer states

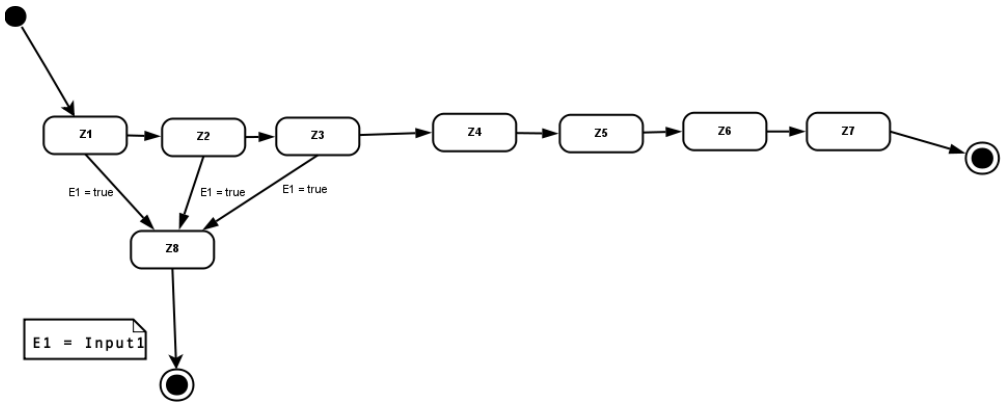


Figure 2.2.2: FSM in more detail

- flat FSMs are not useful for complex systems: too many states and transitions
- hierarchy in state machines → StateCharts [Harel, 1987]

Definition 2.4 The current states of FSMs are called **active states**.
States which are not composed of other states are called **basic states**.
States containing other states are called **super-states**.
A super-state S is called an **OR-super-state** if exactly one of its sub-states is active whenever S is active.

- in an OR-super-state exactly one sub-state is active

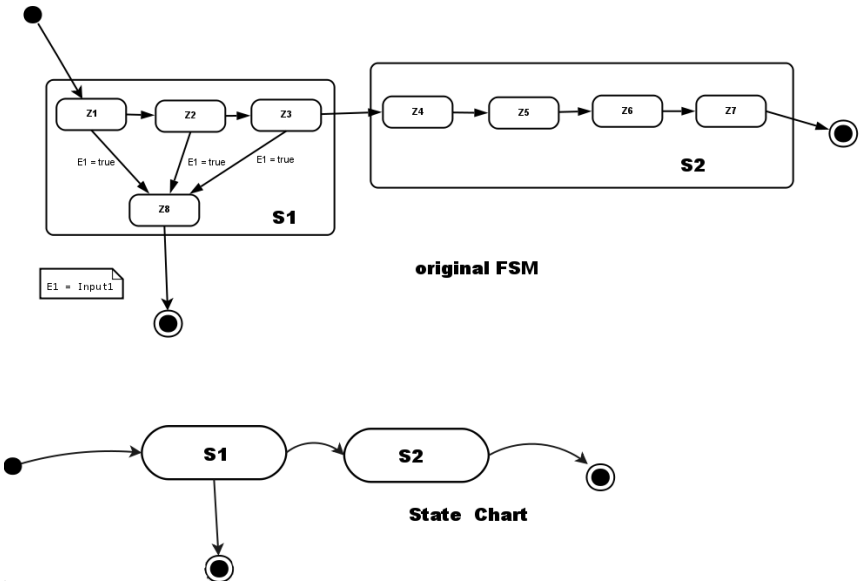


Figure 2.2.3: StateChart

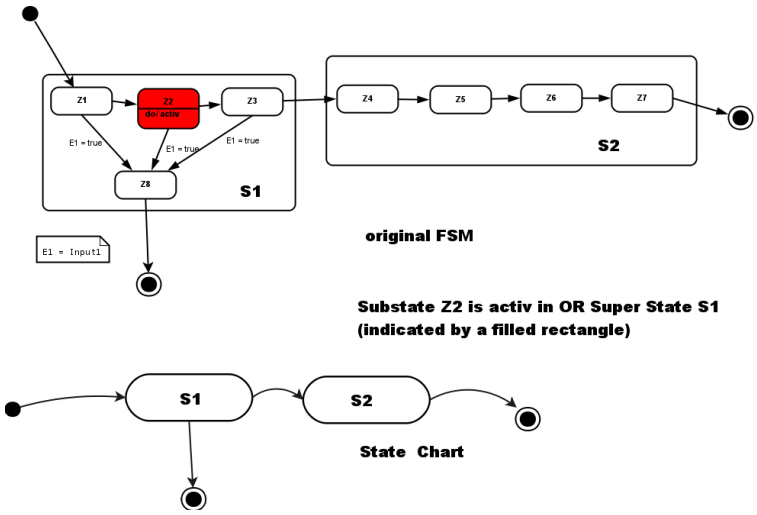


Figure 2.2.4: Active sub-state in a super-state

- concurrency cannot be modelled this way

Definition 2.5 A super-state S is called an **AND-super-state** if all of its sub-states are active whenever S is active.

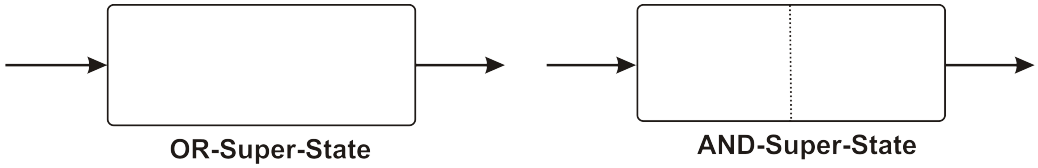


Figure 2.2.5: Concurrency in an AND-super-state

- AND-super-states $\rightarrow$ concurrent state machines
- an AND-super-state is equivalent to the "product" of its sub-machines

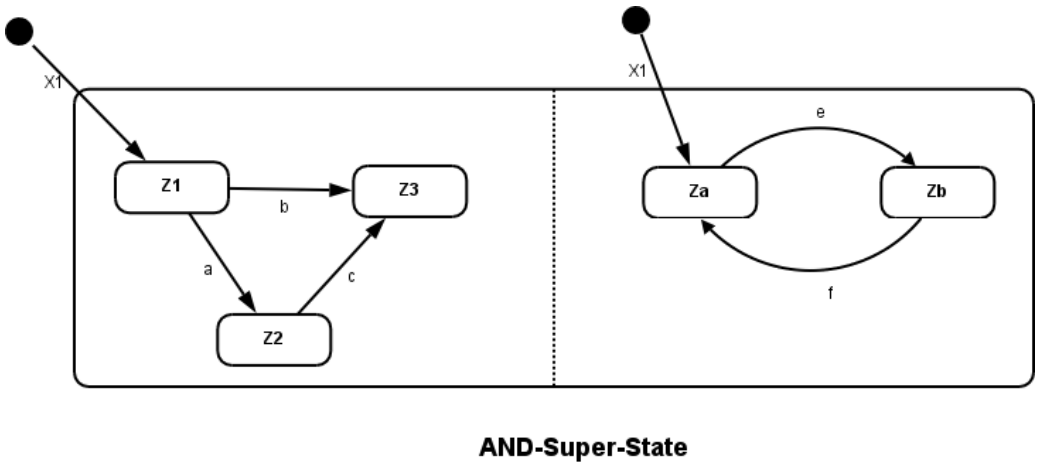


Figure 2.2.6: Example of an AND-super-state

- **advantages of StateCharts**
  - hierarchy allows arbitrary nesting of AND- and OR-super-states
  - well-defined semantics; many tools can simulate them
  - code generators produce C, C++ or VHDL
  - today part of UML and SysML (state machine diagrams)
- **disadvantages**
  - generated code is sometimes inefficient
  - not suited for distributed systems
  - no description of non-functional behaviour or of the structure

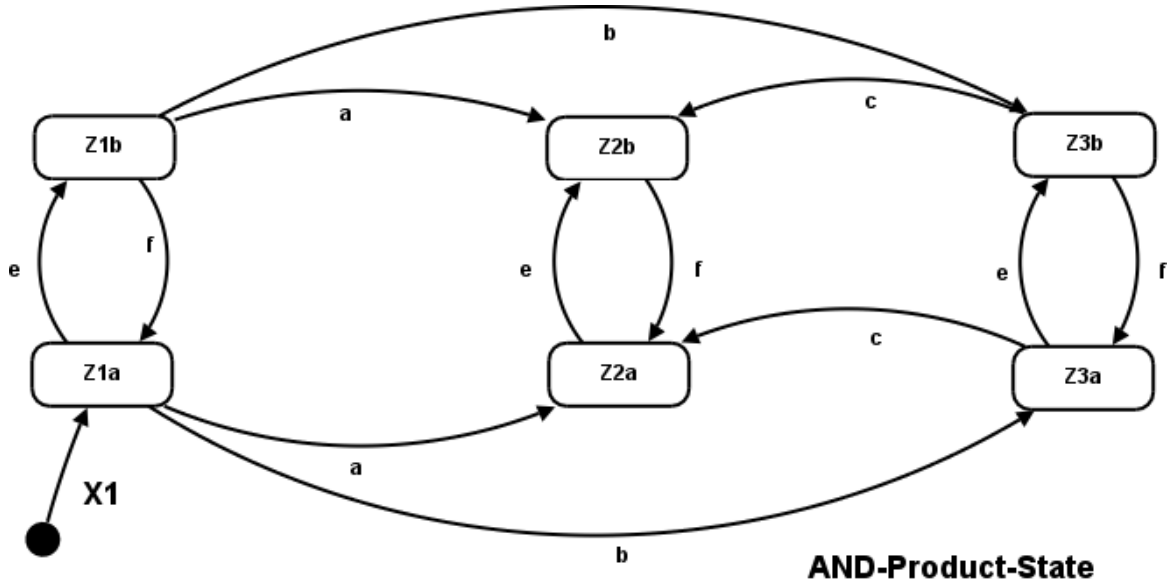


Figure 2.2.7: Decomposition of an AND-super-state into a product machine

2.3 Implementing a State Machine

A state machine translates directly into code: one variable holds the current state, and a function that is called periodically or for every event computes the next state and the outputs. As an example we use a coffee machine with two selection buttons and a timer that signals "ready". The same machine is implemented in C with a `switch` statement and in Rust with `enum` and `match`. Both versions run on the Pico in Wokwi.

Example: a coffee machine

- inputs: buttons **left** and **right**, event **ready** from a timer 3 s after brewing started (coffee finished)
- outputs: add coffee, milk, sugar – depend only on the state ⇒ Moore machine
- from idle: left = coffee with milk, right = coffee; then left = make it, right = add sugar

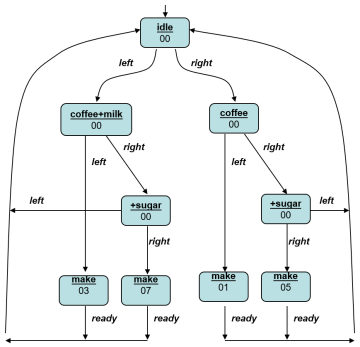


Figure 2.3.1: Coffee machine FSM

State transition table

no.	state	left	right	output
0	idle (initial)	2: coffee+milk	1: coffee	000
1	coffee	5: make coffee	3: coffee+sugar	000
2	coffee+milk	6: make coffee+milk	4: coffee+milk+sugar	000
3	coffee+sugar	0: idle	7: make coffee+sugar	000
4	coffee+milk+sugar	0: idle	8: make coffee+milk+sugar	000
		ready (timer, after 3 s)		
5	make coffee	0: idle		001
6	make coffee+milk	0: idle		011
7	make coffee+sugar	0: idle		101
8	make coffee+milk+sugar	0: idle		111

- output bits: bit 0 = coffee, bit 1 = milk, bit 2 = sugar
- inputs not listed in a row are ignored: the state stays the same
- a table like this is the best check for completeness: every state × every input

C: states, outputs and inputs

```
// the FSM states
enum {
  FSM_STATE_IDLE, FSM_STATE_COFFEE, FSM_STATE_COFFEE_MILK,
  FSM_STATE_COFFEE_SUGAR, FSM_STATE_COFFEE_MILK_SUGAR,
  FSM_STATE_MAKE_COFFEE, FSM_STATE_MAKE_COFFEE_MILK,
  FSM_STATE_MAKE_COFFEE_SUGAR, FSM_STATE_MAKE_COFFEE_MILK_SUGAR,
  NUM_FSM_STATES
};
// the FSM outputs, one per state (Moore machine)
// bit0: add coffee, bit1: add milk, bit2: add sugar
const uint8_t fsm_output[NUM_FSM_STATES] = {0, 0, 0, 0, 0, 1, 3, 5, 7};

// the FSM inputs
enum { FSM_INP_NONE = 0, FSM_INP_LEFT, FSM_INP_RIGHT, FSM_INP_READY };

int fsm_state = FSM_STATE_IDLE;
```

- always use enums for states, inputs and outputs: readable names, and `NUM_FSM_STATES` sizes the table automatically
- define the initial state explicitly

C: the transition function

```
int fsm_tic(int fsm_inp) {         // called periodically, every 50 ms
  switch (fsm_state) {
  case FSM_STATE_IDLE:
    if (fsm_inp == FSM_INP_LEFT) fsm_state = FSM_STATE_COFFEE_MILK;
    if (fsm_inp == FSM_INP_RIGHT) fsm_state = FSM_STATE_COFFEE;
    break;
  /* ... COFFEE, COFFEE_MILK, COFFEE_SUGAR, COFFEE_MILK_SUGAR: same pattern ... */
  case FSM_STATE_MAKE_COFFEE:          // wait until the coffee is ready
  case FSM_STATE_MAKE_COFFEE_MILK:
  case FSM_STATE_MAKE_COFFEE_SUGAR:
  case FSM_STATE_MAKE_COFFEE_MILK_SUGAR:
    if (fsm_inp == FSM_INP_READY) fsm_state = FSM_STATE_IDLE;
```

```
    break;
  default:
    fsm_state = FSM_STATE_IDLE;       // invalid state: back to a safe state
  }
  return fsm_output[fsm_state];       // Moore FSM: one output per state
}
```

- the period defines the response time; sampling the buttons every 50 ms also filters the contact bounce

C: a timer creates the event ready

```
const uint32_t BREW_TIME_MS = 3000;  // READY comes 3 s after a MAKE state is entered
uint32_t state_entered_ms;           // time of the last state change

void loop() {
  int inp = FSM_INP_NONE;
  /* ... LEFT or RIGHT from the buttons (edge detection) ... */
  // timer: the brewing time is over -> event READY (only MAKE states react)
  if (fsm_state >= FSM_STATE_MAKE_COFFEE && millis() - state_entered_ms >=
    BREW_TIME_MS)
    inp = FSM_INP_READY;
  int old_state = fsm_state;
  int out = fsm_tic(inp);
  if (fsm_state != old_state) state_entered_ms = millis();  // restart the timer
  /* ... write out to the LEDs, delay(50) ... */
}
```

- a time-out is an event like a button press: `fsm_tic()` is unchanged, only the source of **ready** is new
- StateCharts and UML write this as a **timed transition** `after(3 s)`
- never `delay(3000)` inside the state machine: it must keep reacting to other inputs (e.g. a cancel button) while it waits
- `millis() - start` with `uint32_t` stays correct when `millis()` wraps around after 49 days

Rust: the same machine with enum and match

```
pub enum State { Idle, Coffee, CoffeeMilk, CoffeeSugar, CoffeeMilkSugar,
                 MakeCoffee, MakeCoffeeMilk, MakeCoffeeSugar, MakeCoffeeMilkSugar }
pub enum Input { Left, Right, Ready }

impl State {
    pub fn next(self, input: Input) -> State {
        use {Input::*, State::*};
        match (self, input) {
            (Idle, Left) => CoffeeMilk,
            (Idle, Right) => Coffee,
            (Coffee, Left) => MakeCoffee,
            (Coffee, Right) => CoffeeSugar,
            (CoffeeMilk, Left) => MakeCoffeeMilk,
            (CoffeeMilk, Right) => CoffeeMilkSugar,
```

```
            (CoffeeSugar, Left) | (CoffeeMilkSugar, Left) => Idle,
            (CoffeeSugar, Right) => MakeCoffeeSugar,
            (CoffeeMilkSugar, Right) => MakeCoffeeMilkSugar,
            (MakeCoffee | MakeCoffeeMilk | MakeCoffeeSugar | MakeCoffeeMilkSugar,
    Ready) => Idle,
            (state, _) => state, // all other inputs are ignored
        }
    }
}
```

- a `State` can only hold these values – no "invalid state" as with an `int`

Rust: output function and tests

```
pub fn output(self) -> Output {
    let (coffee, milk, sugar) = match self {
        State::Idle | State::Coffee | State::CoffeeMilk
        | State::CoffeeSugar | State::CoffeeMilkSugar => (false, false, false),
        State::MakeCoffee => (true, false, false),
        State::MakeCoffeeMilk => (true, true, false),
        State::MakeCoffeeSugar => (true, false, true),
        State::MakeCoffeeMilkSugar => (true, true, true),
    };
    Output { coffee, milk, sugar }
}

#[test]
fn coffee_with_everything() {
    let s = run(&[Left, Right, Right]);
    assert_eq!(s, MakeCoffeeMilkSugar);
    assert_eq!(s.output(), Output { coffee: true, milk: true, sugar: true });
}
```

- no wildcard in `output()`: if a state is added later, the compiler reports every `match` that does not handle it
- the library has no hardware access: `cargo test` on the PC, the same code on the Pico

Recipe for a Moore state machine

- analyse the problem before coding: draw the state diagram, write the transition table
- use enums to encode states, inputs and outputs in a readable form
- define the outputs with a suitable encoding, e.g. as bit fields
- implement a periodically called tick function (or an event handler) for transitions and outputs
- handle invalid states and unexpected inputs explicitly
- test the transition function separately from the hardware
- our example, tested in Wokwi (C and Rust): left, right, right → all three LEDs on; 3 s later (ready) → all off

Tools for state machines

- drawing, in the browser

  - diagrams.net (draw.io), free: `https://app.diagrams.net/`
  - Visual Paradigm Online (free version available): UML state machine diagrams
- text-based diagrams that live in the repository next to the code: PlantUML, Mermaid (`https://mermaid.live/`)
- simulation of automata in the browser: `http://ivanzuzak.info/noam/webapps/fsm_simulator/`
- modelling and simulation
  - OpenModelica, library `Modelica.StateGraph` – free, good for co-simulation with physical models (chapter Cyber-Physical Systems)
  - MATLAB/Simulink with Stateflow – commercial, code generation, widely used in automotive
  - UML/SysML tools (e.g. Enterprise Architect, Cameo); SysML v2 was adopted by the OMG in 2025

2.4 Implementing StateCharts

The coffee machine is a flat state machine. Real controllers need the elements of StateCharts: super-states whose transitions apply to all their sub-states, timed transitions, actions on entering and leaving a state, and regions that run concurrently. None of this needs a tool or a framework; a few rules turn a StateChart into plain C. Two examples show how. A garage door controller uses hierarchy, timers and a concurrent warning lamp, and the door itself is a model as well: a Wokwi custom chip that simulates the motor, the end switches and a light barrier. A pedestrian crossing uses an AND-super-state with two communicating regions and an OR-super-state for the night mode. Both are in `examples/modelling`, with Wokwi scenarios that test them.

Example: a garage door

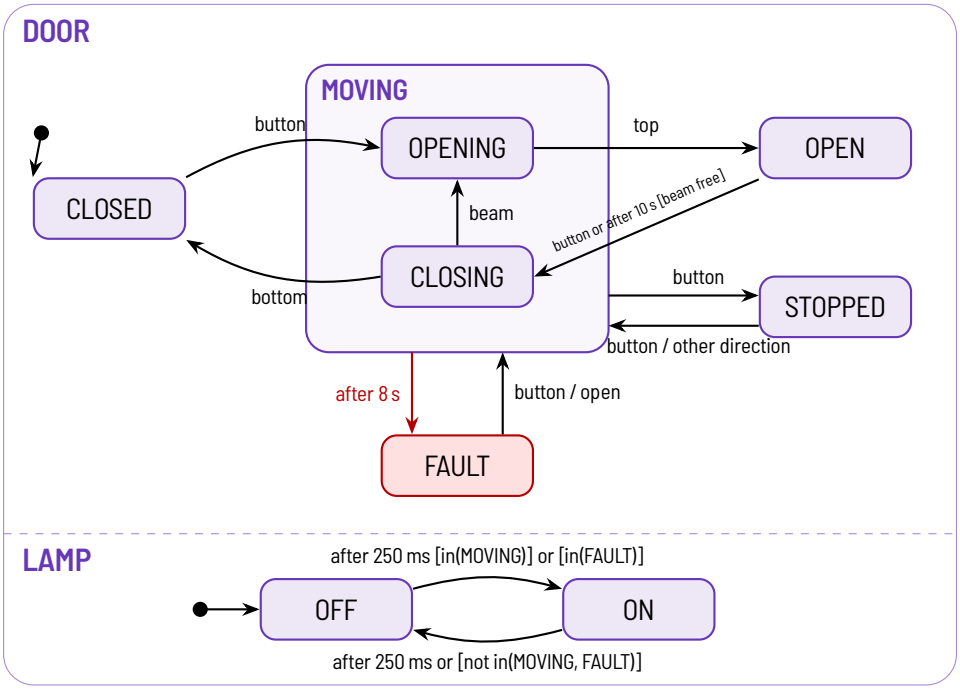


Figure 2.4.1: StateChart of the garage door: super-state MOVING, timed transitions, concurrent region LAMP

- one button for everything (like a remote control): open, stop, close; closes by itself after 10 s
- safety: a light barrier stops and reverses the closing door; a jammed door must not run the motor for ever

C: one function for every transition

```
enum DoorState { CLOSED, OPENING, OPEN, CLOSING, STOPPED, FAULT, NUM_DOOR_STATES };
DoorState door = CLOSED;
uint32_t door_entered_ms;           // for the timed transitions "after(t)"
bool last_moved_up;                 // what STOPPED remembers

bool in_moving(DoorState s) {       // super-state MOVING = {OPENING, CLOSING}
  return s == OPENING || s == CLOSING;
}

// every transition goes through here: exit actions, then entry actions
void enter(DoorState next) {
  digitalWrite(MOTOR_UP, LOW);      // exit: motor off, so UP and DOWN
  digitalWrite(MOTOR_DOWN, LOW);    // are never on at the same time
  door = next;
  door_entered_ms = millis();
  if (next == OPENING) { digitalWrite(MOTOR_UP, HIGH); last_moved_up = true; }
  if (next == CLOSING) { digitalWrite(MOTOR_DOWN, HIGH); last_moved_up = false; }
}
```

- entry and exit actions live in one place – no transition can forget to switch the motor off
- a super-state needs no own variable: `in_moving()` is a test on the sub-state

C: the transition function with hierarchy and timers

```
void door_tic(bool button, bool top, bool bottom, bool beam) {
  uint32_t t = millis() - door_entered_ms;
  // transitions of the super-state come first: they leave every sub-state
  if (in_moving(door) && t >= MOTOR_TIMEOUT_MS) { enter(FAULT); return; }

  switch (door) {
  case CLOSED:  if (button) enter(OPENING); break;
  case OPENING: if (top) enter(OPEN);
                else if (button) enter(STOPPED);
                break;
  case OPEN:    if ((button || t >= AUTO_CLOSE_MS) && !beam)   // guard [beam free]
                  enter(CLOSING);
                break;
  case CLOSING: if (beam) enter(OPENING);             // safety first: reverse
                else if (bottom) enter(CLOSED);
                else if (button) enter(STOPPED);
                break;
  case STOPPED: if (button) enter(last_moved_up ? CLOSING : OPENING); break;
  case FAULT:   if (button) enter(OPENING); break;    // acknowledge: try to open
  default:      enter(STOPPED);                       // invalid state: motor off
  }
}
```

- the order of the `ifs` is the priority of the transitions: in CLOSING the light barrier wins over everything else

C: a concurrent region

```
enum LampState { LAMP_OFF, LAMP_ON };
LampState lamp = LAMP_OFF;  uint32_t lamp_entered_ms;    // own state, own timer

void lamp_tic() {
  LampState next = lamp;
  if (door == FAULT) next = LAMP_ON;                  // condition in(FAULT)
  else if (!in_moving(door)) next = LAMP_OFF;         // condition in(MOVING)
  else if (millis() - lamp_entered_ms >= BLINK_MS)
    next = (lamp == LAMP_ON) ? LAMP_OFF : LAMP_ON;
  if (next != lamp) { lamp = next; lamp_entered_ms = millis(); }
  digitalWrite(LAMP, lamp == LAMP_ON);
}
void loop() {                                  // every 20 ms
  /* ... button edge, read TOP, BOTTOM, BEAM ... */
  door_tic(button, top, bottom, beam);
  lamp_tic();                                  // both regions get every tick
}
```

- an AND-super-state = one state variable and one tick function per region, all called in every tick
- regions communicate by reading each other's state (`in(...)`) – read only, never write

The environment as a model: the door is a custom chip

- the door does not exist: a Wokwi **custom chip** (`door.chip.c`) models it, the controller sees only its pins
- inputs UP, DOWN (motor); outputs TOP, BOTTOM (end switches), BEAM (light barrier); a small display shows the door
- controls on the chip inject situations that are dangerous or expensive to test on a real door: **obstacle** in the door, door **jammed**
- the model checks the controller: both motor lines on ⇒ "short circuit!", door closes onto the obstacle ⇒ "CRASH"

```
static void on_step(void *user_data) {             // every 10 ms of simulated time
  chip_t *c = user_data;
  bool up = pin_read(c->up), down = pin_read(c->down);
  bool obstacle = attr_read(c->obstacle_attr), jam = attr_read(c->jam_attr);
  if (up && down) printf("motor UP and DOWN at the same time: short circuit!\n");
  else if (!jam) c->pos += (up - down) * 0.01f / TRAVEL_S;    // 5 s from closed to open
  /* ... limit pos to 0..1; obstacle under the door: stop at 15 %, print CRASH ... */
  pin_write(c->top, c->pos >= 1);
  pin_write(c->bottom, c->pos <= 0);
  pin_write(c->beam, obstacle);
}
```

Testing the StateChart against the model

```
- wait-serial: 'door: closing'                    # after 10 s: auto close
- delay: 2000ms
- set-control: { part-id: door, control: obstacle, value: 1 }
- wait-serial: 'door: opening'                    # beam: reverse
- set-control: { part-id: door, control: obstacle, value: 0 }
  ...
- set-control: { part-id: door, control: jam, value: 1 }
- wait-serial: 'door: FAULT'                      # 8 s without end switch
- expect-pin: { part-id: pico, pin: GP17, value: 0 }     # motor off
- expect-pin: { part-id: pico, pin: GP18, value: 1 }     # lamp steady on
```

- a Wokwi scenario (`door/test/door.yaml`) presses the button, switches the faults on the chip and checks pins and serial output
- the StateChart is the test plan: every transition needs a step that triggers it – which ones does `door.yaml` still miss?
- the same idea at a larger scale: case study in chapter Operating Systems (climate chamber, two custom chips)

Example: a pedestrian crossing

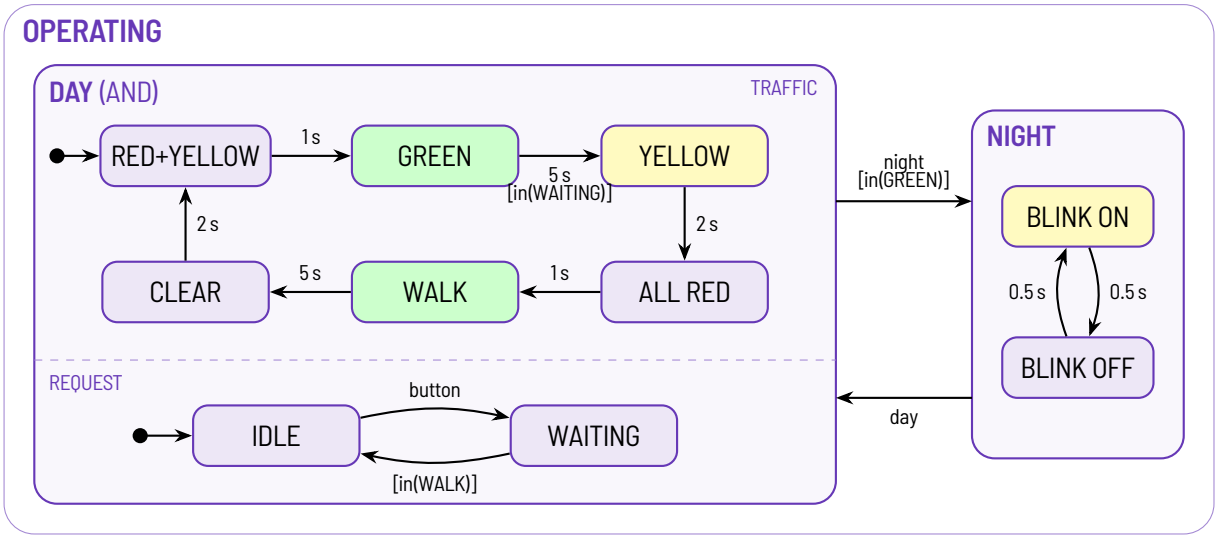


Figure 2.4.2: StateChart of the pedestrian crossing: AND-super-state DAY with two regions, OR-super-state OPERATING

- TRAFFIC (6 states) and REQUEST (2 states) run concurrently and talk only through conditions on each other's state
- the product machine would have $6 \times 2 = 12$ states – and grows with every further region
- NIGHT can be entered only from GREEN: a guard on the super-state transition, never in the middle of a pedestrian phase

C: AND- and OR-super-states

```
enum Mode { DAY, NIGHT };                          // OR-super-state OPERATING
// + one enum each for the regions TRAFFIC, REQUEST and for NIGHT (BLINK_ON/OFF)
```

```
3  void crossing_tic(bool button, bool night) {
4    switch (mode) {
5    case DAY:
6      if (night && traffic == GREEN) { mode = NIGHT; return; }  // only from a safe state
7      traffic_tic();                                            // AND: both regions run
8      request_tic(button);                                      // in every tick
9      break;
10   case NIGHT:
11     if (!night) {
12       mode = DAY;                                             // enter DAY in its initial states
13       request = IDLE;
14       set_traffic(RED_YELLOW);
15       return;
16     }
17     blink_tic();
18     break;
19   }
20 }
```

- one variable per OR-super-state, one per region; `traffic_tic()` uses `request == WAITING`, `request_tic()` uses `traffic == WALK`

Recipe for a StateChart in C

- one `enum` variable per OR-super-state and per concurrent region
- super-states without own variable: a function `in_xxx()` that tests the sub-state
- transitions of a super-state are checked before those of its sub-states
- one function `enter()` for all transitions: exit actions, set the state, store the entry time, entry actions
- timed transitions: compare `millis() - entered_ms` with the time – never block with `delay()`
- one tick function per region; all are called in every tick, regions only read each other's state
- test against a model of the environment: custom chips in Wokwi, scenarios for every transition
- larger machines: a table of function pointers, or a framework (e.g. QP/C with hierarchical state machines) or a code generator (Stateflow, itemis CREATE)

2.5 Activity Oriented Models

Activity oriented models describe what happens to the data and in which order. Data flow graphs show the dependencies between operations and leave the order open, which is ideal for signal processing and for deciding which operations can run in parallel. Flow charts add control flow. Sequence charts show the communication between components over time.

- modelling the flow of data in an embedded system → **DFG** (data flow graph)
- nodes are operations; special nodes for input, output and memory
- edges show the data dependencies on all levels of abstraction
- well suited for the specification and as input for scheduling, because no order is fixed beyond the dependencies
- DFGs can be hierarchical

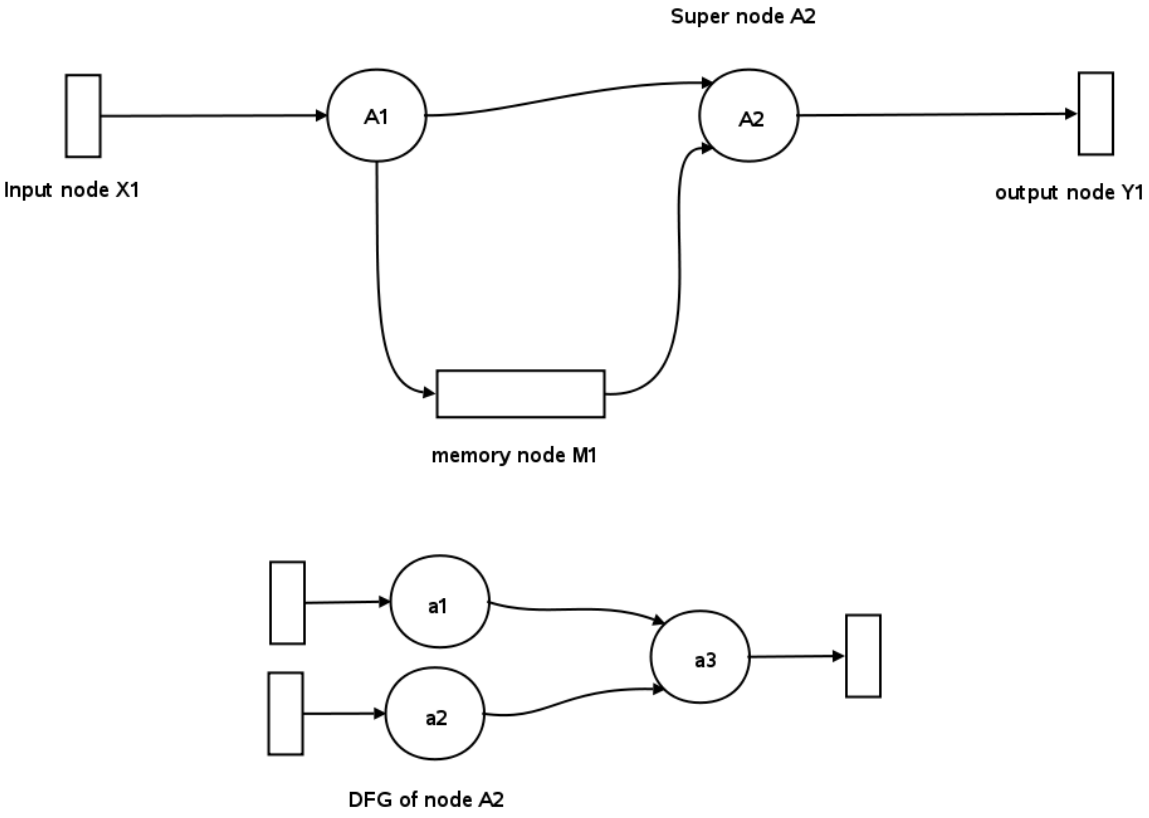


Figure 2.5.1: Data flow graph

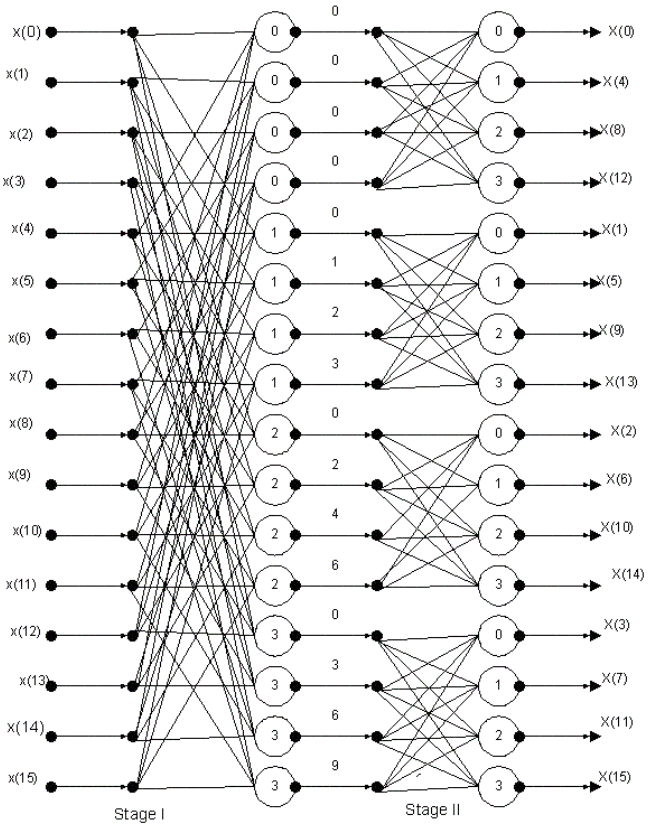


Figure 2.5.2: DFG of an FFT algorithm

- extension of a DFG with control flow → **CFG** (control flow graph)
- additional control elements (decisions, loops)
- edges transfer the control
- CFGs are drawn as flow charts

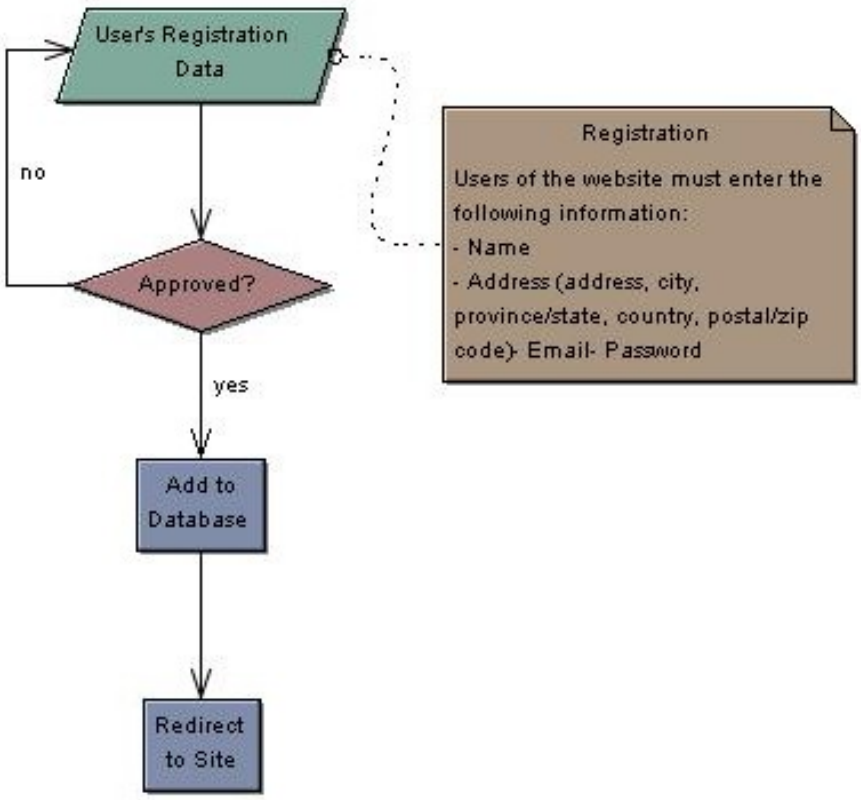
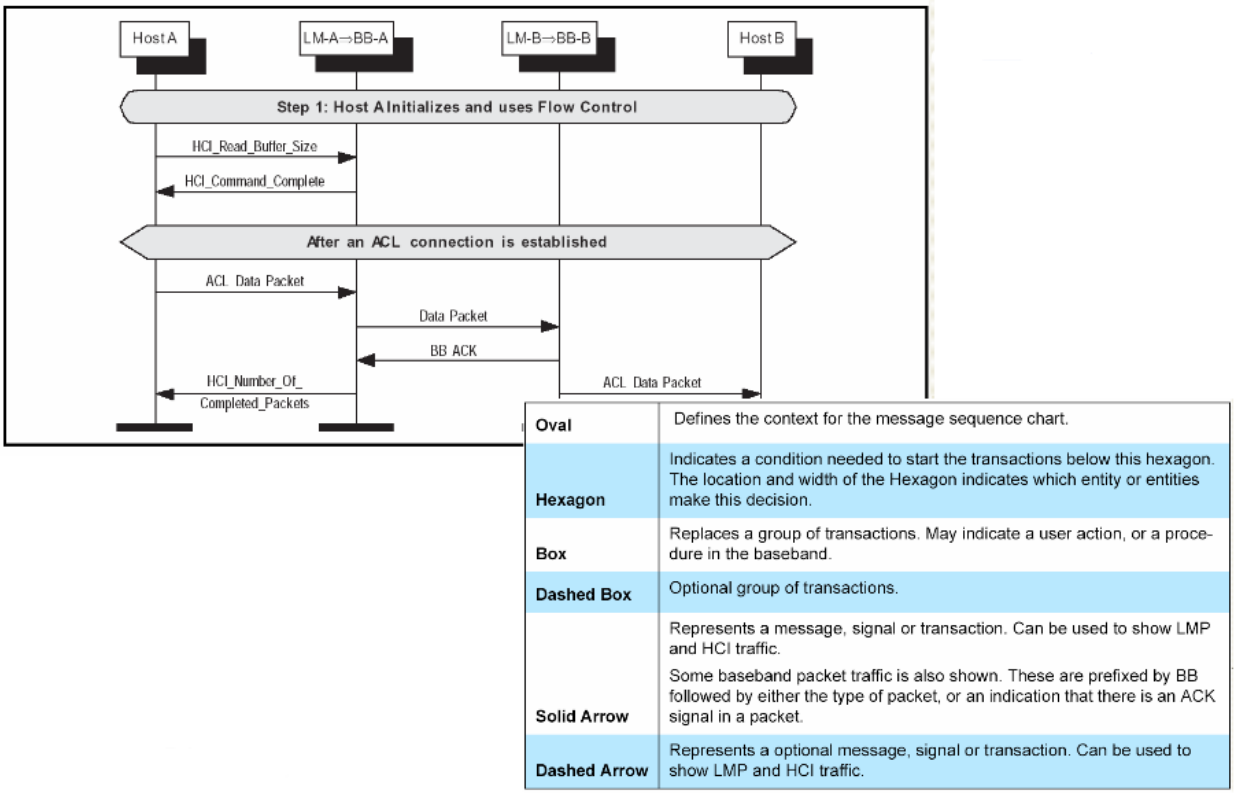


Figure 2.5.3: Flow chart of an algorithm

Message Sequence Charts (MSC)

- modelling of timing and of the communication between components
- graphical representation of schedules
  - time is a separate axis (vertical): timing is visible explicitly
  - components (activities) on the horizontal axis
- today usually drawn as UML sequence diagrams – the TCP handshake and the DHCP diagram in chapter Networking are examples

- **advantages of activity oriented models**
  - well suited to visualise schedules and communication
  - DFG and CFG can be hierarchical
- **disadvantages**
  - an MSC shows one scenario only, not all possible behaviours
  - no description of non-functional behaviour



Oval	Defines the context for the message sequence chart.
Hexagon	Indicates a condition needed to start the transactions below this hexagon. The location and width of the Hexagon indicates which entity or entities make this decision.
Box	Replaces a group of transactions. May indicate a user action, or a procedure in the baseband.
Dashed Box	Optional group of transactions.
Solid Arrow	Represents a message, signal or transaction. Can be used to show LMP and HCI traffic. Some baseband packet traffic is also shown. These are prefixed by BB followed by either the type of packet, or an indication that there is an ACK signal in a packet.
Dashed Arrow	Represents a optional message, signal or transaction. Can be used to show LMP and HCI traffic.

Figure 2.5.4: Message sequence chart

- no description of the structure (MSC)

Exercises

- Extend the coffee machine by a "cancel" button that returns to idle from every selection state. Change the table first, then C and Rust. What does the Rust compiler say if you forget a case?
- Convert the coffee machine into a Mealy machine whose outputs are pulses on the transitions. Compare the number of states.
- Give the coffee machine a "cancel" time-out: a selection that is not completed within 10 s returns to idle.
- Garage door: delete the transition CLOSING → OPENING on **beam** and run the scenario. What does the door model report? Which other transition of the StateChart protects people, and how would you test it?
- Pedestrian crossing: draw the product machine of TRAFFIC and REQUEST. Which of the 12 states can never be reached?
- Draw the state machine of the button debouncer of chapter Programming.
- Draw a message sequence chart of the MQTT example of chapter Networking (device, broker, smartphone).

3 Cyber-Physical Systems

3.1 Overview

An embedded system never works alone. It measures and influences a physical process: the temperature of a room, the speed of a motor, the position of a robot arm. Cyber-physical systems (CPS) are systems in which this coupling of computation and physics is essential. They are often distributed and connected, as in a car, a factory, a power grid or an aircraft. Their behaviour can only be understood if both parts are described together: the physics with differential equations, the software with algorithms and state machines.
Because building and testing the real system is expensive and sometimes dangerous, CPS are developed with models and simulation. This chapter introduces the idea of CPS and the steps from a model to the real system. It then shows how physical systems are modelled with the equation-based language Modelica, using the free tool OpenModelica, and with a few lines of Python.

What is a cyber-physical system?

Definition 3.1 Cyber-physical systems are integrations of computation with physical processes. Embedded computers and networks monitor and control the physical processes, usually with feedback loops where physical processes affect computations and vice versa. [Edward A. Lee, 2008]

- combination of mechanical, electrical, thermal, ... components with software and communication
- examples: driver assistance systems, industrial robots, smart grids, medical devices, building automation
- related terms: **Internet of Things** (focus on connectivity), **Industry 4.0** (production), **digital twin** (a model of a real system that is kept up to date with data from the system)

The feedback loop

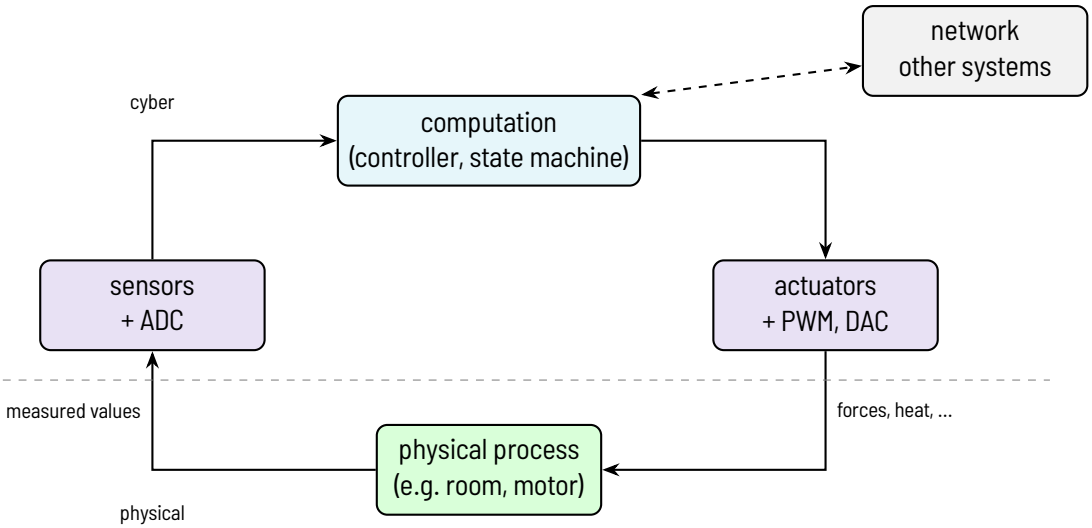


Figure 3.1.1: A cyber-physical system: computation and physical process in a closed loop

- the physical process runs continuously in time; the computer samples and acts at discrete times
- timing is part of the correctness: a controller that reacts too late can make the loop unstable

From model to product

- the system is developed and tested in steps, replacing models by real parts one after the other

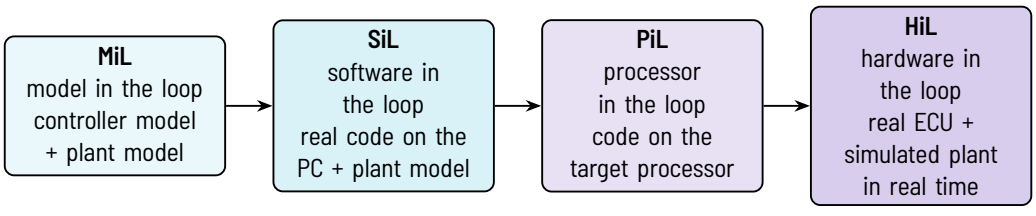


Figure 3.1.2: Test levels from the model to the real hardware

- **HiL**: the real control unit runs against a real-time plant simulator – dangerous or rare situations can be tested safely and repeatably

- our Wokwi simulations are a kind of PiL test: the real firmware on a simulated processor
- **FMI** 3.0 (2022): standard to exchange models between tools as **FMU**s for co-simulation

Simulation of physical systems

- physical systems are described by **differential equations**, often with algebraic constraints (differential-algebraic equations, DAE)
- models with lumped parameters (masses, springs, resistors) – ordinary differential equations
- continuous-time physics combined with discrete-time controllers and events (hybrid systems)
- tools
  - **block oriented**: MATLAB/Simulink, Xcos (Scilab) – signals flow in one direction from block to block
  - **equation based** (object oriented): Modelica with OpenModelica (free) or Dymola (commercial) – components are described by their physical equations
  - general programming languages: Python with `numpy`/`scipy`, Julia (ModelingToolkit)

Block oriented modelling

- the system is a diagram of blocks with inputs and outputs: gain, sum, integrator, transfer function
- every connection has a fixed direction (causality): the modeller decides what is computed from what

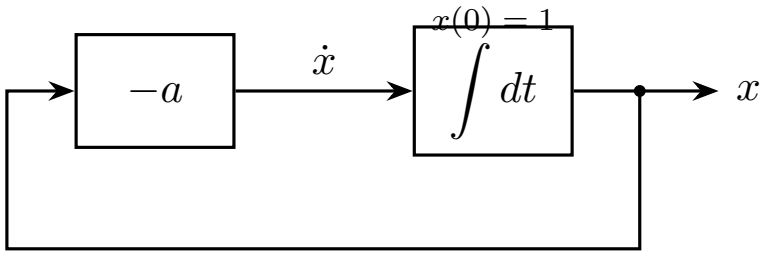


Figure 3.1.3: Block diagram of $\dot{x} = -a\,x$: the output of the integrator is fed back

- natural for control engineering and signal processing
- for physical networks (electrical circuits, mechanics) the modeller must first solve the equations for the right variables by hand – this is what equation-based tools avoid

3.2 Modelling with Modelica

Modelica is a language for describing physical systems by equations. A model states which relations hold, for example Ohm's law $u = R \cdot i$, without saying which variable is computed from which. The compiler sorts the equations, solves them symbolically where possible and generates simulation code. Components can be connected like real parts: connecting a resistor to a capacitor automatically produces Kirchhoff's laws. Modelica is standardised by the Modelica Association. OpenModelica is a free implementation with a graphical editor (OMEdit) and an extensive standard library for electrical, mechanical, thermal, fluid and control components.

Features of Modelica / OpenModelica

- **equation based** (acausal): equations instead of assignments – the tool decides the order of computation
- **object oriented**: classes, instances, inheritance, packages; parameters separate from the structure (a resistor model, used with different values)
- **multi-domain**: electrical, mechanical, thermal, hydraulic, ... components in one model
- **connectors** with potential variables (voltage, temperature) and flow variables (current, heat flow): `connect()` creates the equations "potentials equal, sum of flows zero"
- physical units for all quantities (`unit = "V"`) – the tool checks them
- graphical (drag and drop in OMEdit) and textual modelling of the same model
- export as FMU for co-simulation; code generation for controllers

OpenModelica: graphical modelling

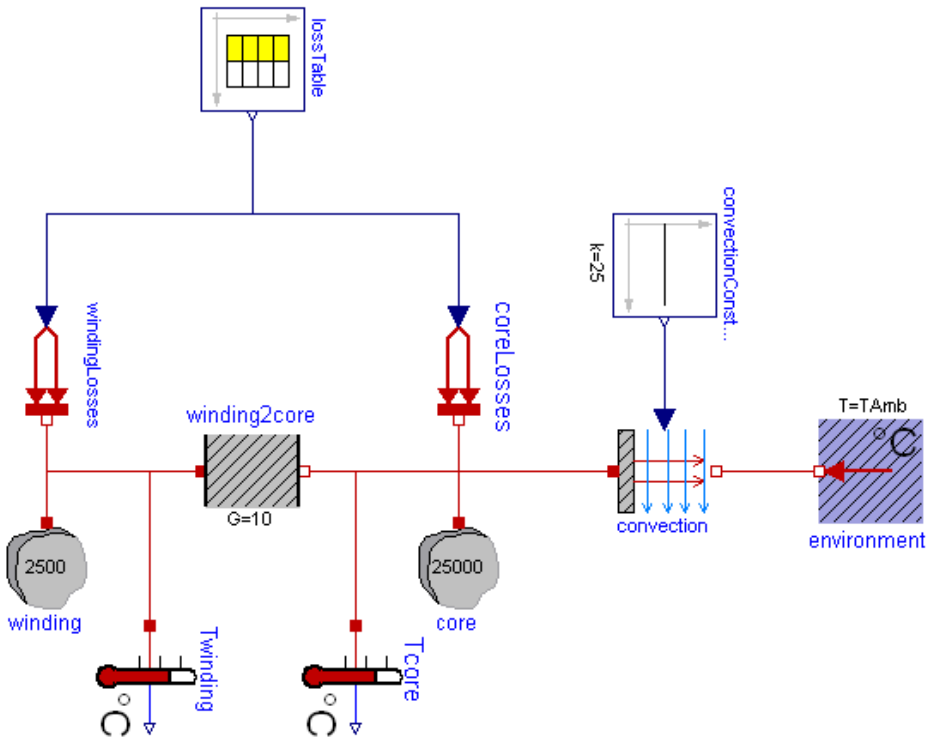


Figure 3.2.1: OpenModelica model of a heat transfer

First elements of Modelica

- data types: `Real` (64 bit floating point), `Integer`, `Boolean`, `String`, enumerations
- `parameter`: constant during a simulation, can be changed between simulations
- `der(x)`: time derivative $\dot{x}$
- `start`: initial value; `fixed = true`: the initial value must hold exactly
- example: $\dot{x} = -a \cdot x$

```
model HDE
  Real x(start = 1, fixed = true);
  parameter Real a = 1;
equation
  der(x) = -a*x;
end HDE;
```

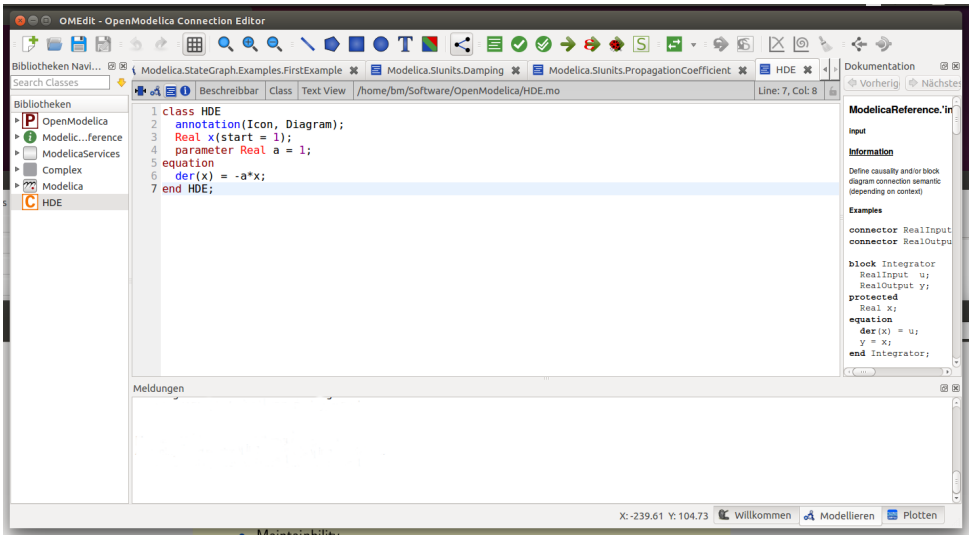


Figure 3.2.2: Model HDE in OMEdit

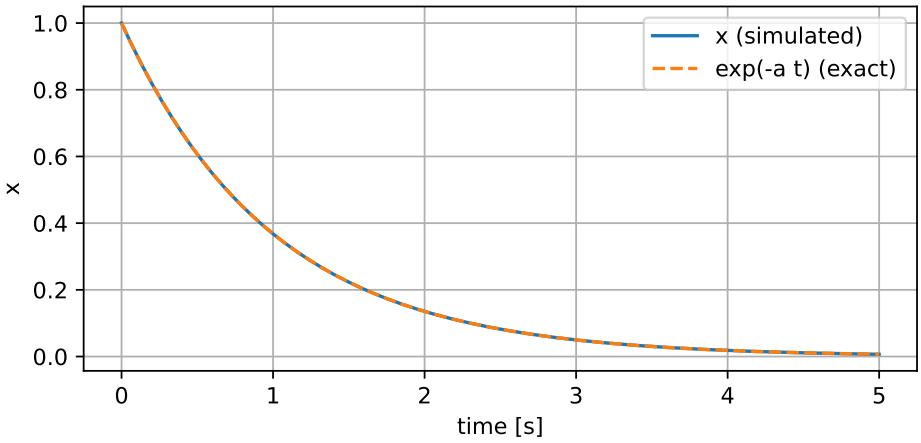
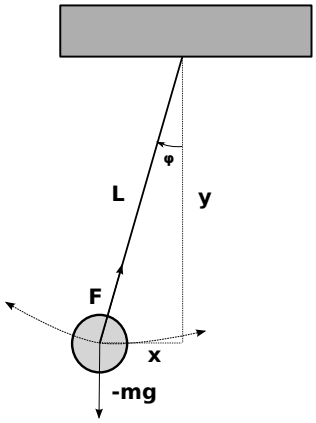


Figure 3.2.3: Solution of the model HDE: simulation and exact solution $x(t) = e^{-at}$

Differential-algebraic equations: the pendulum

- a model can contain algebraic equations (without derivatives) besides the differential equations
- example: a planar pendulum, a mass m on a rod of length L; the rod force F is unknown



$$
\begin{aligned}
m\,\dot{v}_x &= -\frac{x}{L}\,F \\
m\,\dot{v}_y &= -\frac{y}{L}\,F - m\,g \\
\dot{x} &= v_x \\
\dot{y} &= v_y \\
x^2 + y^2 &= L^2
\end{aligned}
$$

- 5 equations for 5 unknowns (x, y, v_x, v_y, F); the last equation is the constraint of the rod

The pendulum in Modelica

```
1  model Pendulum "Planar pendulum"
2    parameter Real m = 1, g = 9.81, L = 0.5;
3    Real F;
4    Real x(start = 0.5, fixed = true), y(start = 0);
5    Real vx, vy;
6  equation
7    m*der(vx) = -(x/L)*F;
8    m*der(vy) = -(y/L)*F - m*g;
9    der(x) = vx;
10   der(y) = vy;
11   x^2 + y^2 = L^2;   // algebraic equation (constraint of the rod)
12 end Pendulum;
```

- the equations are written as in the physics book; OpenModelica reduces the DAE (index reduction) and chooses the state variables itself

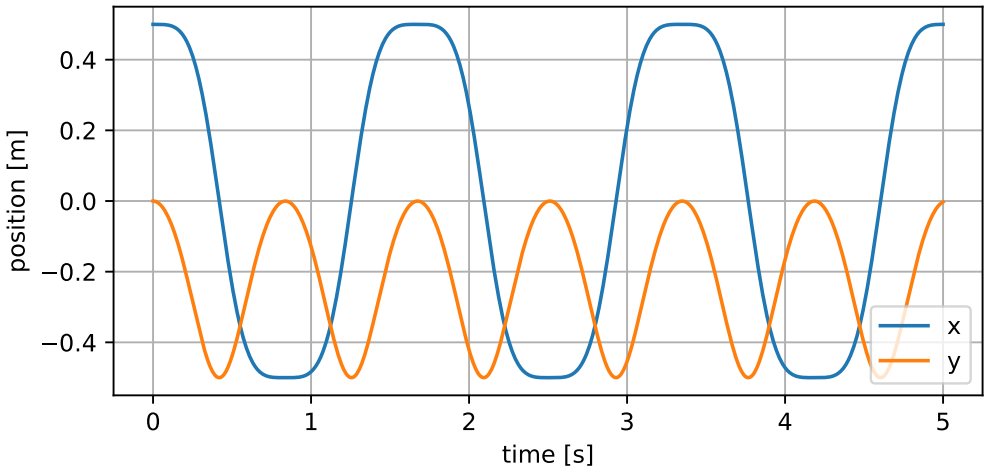


Figure 3.2.4: Horizontal and vertical position of the pendulum, started with the rod horizontal

- y is always ≤ 0: the mass swings below the pivot; the period is longer than for small angles

Classes and instances

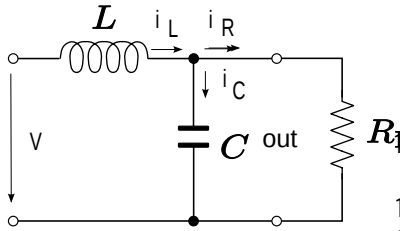
- every object has a class that defines its data and its behaviour
- a class has variables (constants, parameters, computed values), equations, and can contain instances of other classes

```
class Point "Point in a three-dimensional space"
  Real x;
  Real y;
  Real z;
end Point;

class Triangle
  Point point1;
  Point point2;
  Point point3;
end Triangle;

class Triangle2 "with modification of an instance"
  Point point1(x = 1, y = 2, z = 3);
  Point point2;
  Point point3;
end Triangle2;
```

An electrical example




```
model RLC1 "A resistor-inductor-capacitor circuit"
  type Voltage = Real(unit = "V");
  type Current = Real(unit = "A");
  parameter Voltage Vb = 24 "Battery voltage";
  parameter Real L(unit = "H") = 1;
  parameter Real R(unit = "Ohm") = 100;
  parameter Real C(unit = "F") = 1e-3;
  Voltage V;
  Current i_L, i_R, i_C;
equation
  V = i_R*R;
  C*der(V) = i_C;
  L*der(i_L) = Vb - V;
  i_L = i_R + i_C;
end RLC1;
```

- damped oscillation ($\omega_0 = 1/\sqrt{LC} \approx 31.6\,\mathrm{s}^{-1}$, i.e. about 5 Hz); final state $i_L = i_R = V_b/R = 0.24\,\mathrm{A}$, $i_C = 0$

The same equations in Python

```
from scipy.integrate import solve_ivp

Vb, L, R, C = 24.0, 1.0, 100.0, 1e-3

def rlc(t, s):              # state: capacitor voltage V, inductor current i_L
    V, iL = s
    return [(iL - V / R) / C,       # C dV/dt   = i_C = i_L - V/R
            (Vb - V) / L]           # L di_L/dt = Vb - V
```

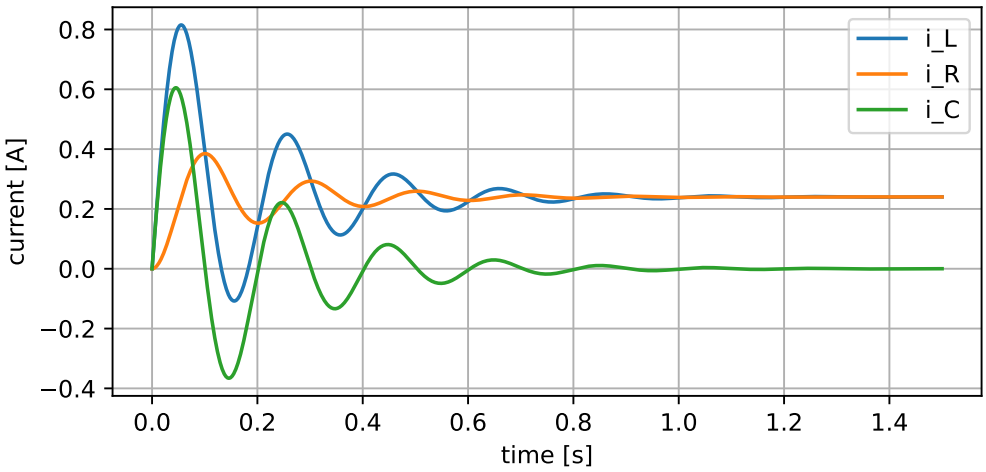


Figure 3.2.5: Currents in the RLC circuit after switching on the battery

```
9
10 sol = solve_ivp(rlc, (0, 1.5), [0.0, 0.0], max_step=1e-3)
11 print("final i_L =", sol.y[1][-1])   # 0.240 A
```

- here the equations had to be solved for the derivatives by hand – for the RLC circuit easy, for the pendulum DAE not (we used the angle as the only state)
- this is exactly the work Modelica does automatically for large models
- the plots of this chapter were created with this script (`examples/cps/simulate.py`, runs on Replit)

Exercises

- Simulate `HDE`, `Pendulum` and `RLC1` in OpenModelica and compare with the plots of the Python script.
- Change R in the RLC circuit to $10\,\Omega$ and $1000\,\Omega$. When does the circuit stop oscillating? Compare with the condition $R = \frac{1}{2}\sqrt{L/C}$.
- Build the RLC circuit graphically from the Modelica standard library (`Modelica.Electrical.Analog`) and compare the generated equations with `RLC1`.
- Add a two-point controller to a heated room model: heating on below 20 °C, off above 21 °C. How does the hysteresis influence the switching frequency?

3.3 Case Study: a Digital Twin

A model is most useful when it runs next to the real system and is fed with the same data. It then knows what the system *should* do and can compare this with what it *does*. This is the idea of the digital twin. This section builds a small but complete one. The physical part is a water tank with a pump, a consumer and a possible leak; it runs in Wokwi as a custom chip, controlled by an ESP32-C3. The cyber part is a Python program on a server in the internet. It receives the measurements over an encrypted connection, runs a model of the tank, estimates the leak flow that no sensor measures, and sends back setpoints and alarms. A web dashboard shows both sides. The example is in `examples/cps/twin`.

Digital twin

- a **model** of one particular real system that is **kept up to date** with data from it, and that may act back on it
  - model only, no data link: simulation (as in the previous section)
  - data flows from the system to the model: **digital shadow**
  - data flows in both directions: **digital twin**
- uses
  - **virtual sensors**: estimate what is not measured (wear, leak flow, temperature inside a winding)
  - **fault detection**: the system does something the model does not expect
  - **prediction** and what-if: run the model ahead of time, try a setting on the model first
  - **supervision**: setpoints, schedules, alarms for many devices from one place
- examples: wind turbines, aircraft engines (maintenance), production lines, water and power grids

The setup

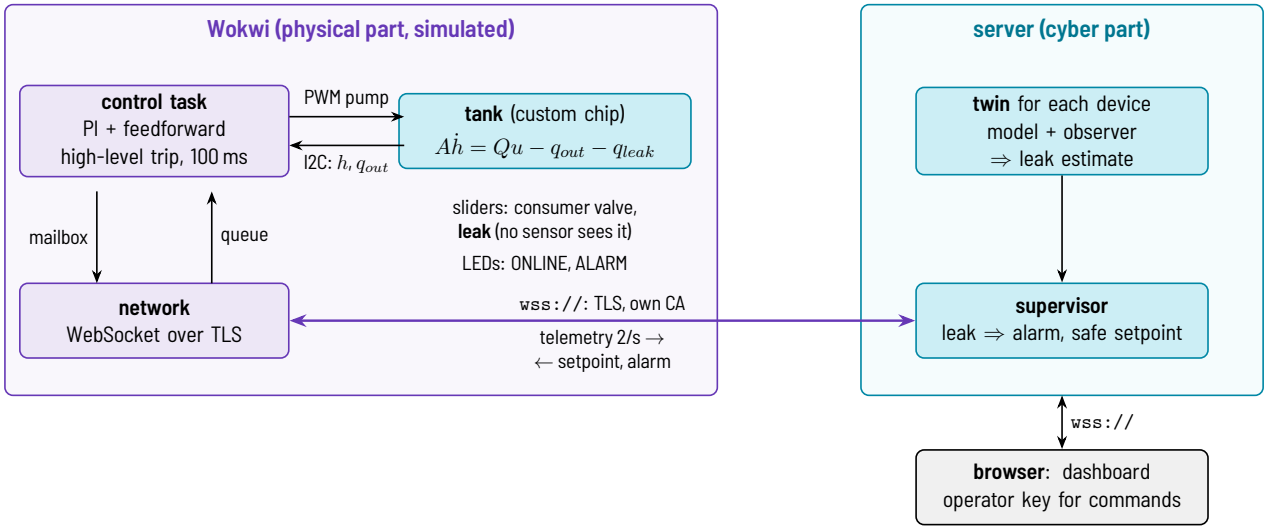


Figure 3.3.1: The physical part runs in Wokwi, the cyber part on a server in the internet

The physical part: tank and controller

- tank model in the custom chip (10 ms steps), area $A = 0.04\,\text{m}^2$, pump $Q = 2\,\text{L/s}$ at full speed:

$$A\,\frac{dh}{dt} = Q\,u \;-\; \underbrace{v\,k_{out}\sqrt{h}}_{q_{out},\ \text{measured}} \;-\; \underbrace{l\,k_{leak}\sqrt{h}}_{q_{leak},\ \text{not measured}}$$

- consumer valve v and leak l are sliders on the chip; the chip draws the tank
- controller on the ESP32-C3, every 100 ms: feedforward q_{out}/Q (replace what the consumer takes) + PI on the level
- **what stays on the device**: the control loop and the high-level trip ($h > 0.95\,\text{m} \Rightarrow$ pump off)
  - they run in their own FreeRTOS task and never wait for the network

    - TLS handshake, lost Wi-Fi, server down: the tank keeps its level – only the ONLINE LED goes off
- **what the server decides**: the setpoint and the alarm – slow decisions that need the model and the overview

The cyber part: model and observer

- the twin knows A and Q, and gets h, q_{out} and u from the device every 0.5 s
- plain model without measurements: $\hat{h}_{k+1} = \hat{h}_k + \Delta t\,(Qu - q_{out})/A$ – drifts away as soon as water goes somewhere unknown
- **observer**: predict with the model, then correct with the measurement; the remaining error is explained by an unknown outflow $\hat{q}_{leak}$

$$\begin{aligned} h^- &= \hat{h}_k + \Delta t\,(Qu - q_{out} - \hat{q}_{leak})/A \qquad r = h_{meas} - h^- \\ \hat{h}_{k+1} &= h^- + L_1\, r \qquad\qquad \hat{q}_{leak} \leftarrow \hat{q}_{leak} - L_2\, r \end{aligned}$$

- $L_1 = 0.3$, $L_2 = 0.004$: estimate noise about 0.015 L/s, 90 % of a step after about 5 s (a Kalman filter computes such gains from the noise levels)
- supervisor: $\hat{q}_{leak} > 0.15$ L/s for 3 s $\Rightarrow$ alarm, setpoint 250 mm (less pressure, less leak flow) until the operator acknowledges
- the time base is the **device** clock (Δt from the time stamps): network delays and a simulation slower than real time do not disturb the model

A secure connection

```
const char *CA_CERT = R"(-----BEGIN CERTIFICATE-----
MIIB3zCCAYagAwIBAgIUGvxo7no1ov/HoARhayx7NHtAbGUwCgYIKoZIzj0EAwIw ...
-----END CERTIFICATE-----)";
ws.setExtraHeaders("Authorization: Bearer ec-lecture-2026");
ws.beginSslWithCA("srv2003097.hstgr.cloud", 8443, "/device?id=tank1",
                  CA_CERT, "twin");
```

- **TLS**: telemetry and commands are encrypted and cannot be changed on the way
- **own certificate authority** (`make_pki.sh`, P-256): the device trusts only certificates signed by this CA (pinning) – not anybody with some public certificate; the server certificate can be renewed without new firmware
- the **server** checks the device: token in the HTTP header (here public in the sketch – in a product a secret or a client certificate per device: mutual TLS); commands from the browser need an **operator key**
- JSON up: `{"t":84403,"h":0.242,"q":883,"u":0.62,"sp":0.25}`
  JSON down: `{"sp":0.25,"alarm":1,"leak":0.368}`

Result: the twin finds the leak

Lessons for cyber-physical systems

- split by time scale and by safety: fast loops and protection on the device, slow decisions with the model and the overview in the cyber part

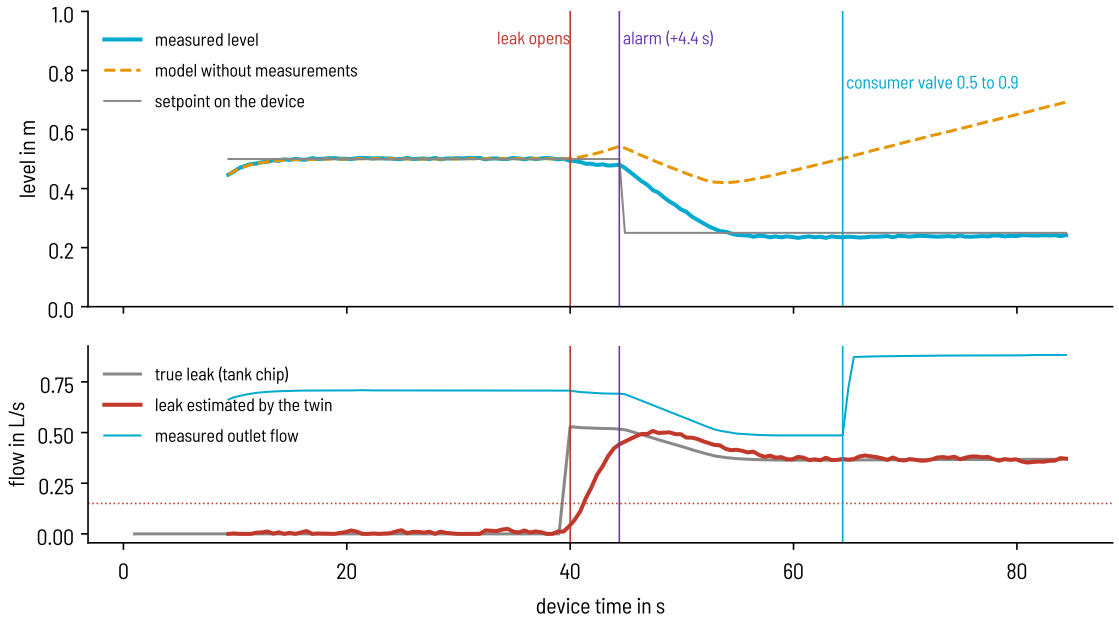


Figure 3.3.2: End-to-end run (Wokwi scenario against the server): the leak opens at 40 s, the twin raises the alarm 4.4 s later and lowers the setpoint. The model without measurements (dashed) drifts away. More consumption at 64 s is measured and causes no false alarm.

- the device must have a defined behaviour **without** the cyber part (here: keep the last setpoint, report offline) – the network will fail
- a model alone is not a twin: without measurements it drifts; without a model the measurements cannot tell a leak from a consumer
- a twin is only as good as its parameters: validate A, Q and the sensors on the real system, and track slow changes (pump wear)
- security is part of the design: every device on the internet is attacked – encryption, authentication of *both* sides, secrets per device, updates
- test levels again: `test/sil_device.py` tests the server with a tank and controller in Python (SiL), the Wokwi scenario runs the real firmware against the real server (PiL)

Exercises: digital twin

- run the Wokwi project with your own device name and watch your twin on the dashboard. Open the leak slowly in steps of 0.05. Which is the smallest leak the twin detects? Explain the limit with the noise of the estimate.
- the pump wears and delivers only 80 % of Q (change `Q_PUMP` in the chip). What does the twin conclude? Extend the observer so that it estimates the pump factor as well. Can it tell a weak pump from a leak?
- add a prediction to the server: when will the tank be empty if the pump fails now? Show it on the dashboard.
- stop the server during a run. What does the device do, what should it do after 10 minutes without a server? Implement your choice.
- replace the shared device token by mutual TLS: create a client certificate per device with the CA and let the server check it.

4 Processors

4.1 Processors for Embedded Systems

Embedded systems use a wide range of processors, from 8 bit microcontrollers that cost a few cents to multi-core application processors that run Linux. The choice depends on the computing power needed, on the memory, on the peripherals, on the energy budget and, in large series, on the price. This chapter first describes the classes of processors and the basic architecture concepts. Then it looks at the families used in this lecture: the 8 bit AVR of the Arduino Uno, the ARM Cortex-M0+ of the Raspberry Pi Pico and the RISC-V core of the ESP32-C3. All three can be simulated in Wokwi.

Classes of processors

- **microcontroller** (MCU): CPU, flash, RAM and peripherals on one chip
  - runs without an operating system or with an RTOS; starts in milliseconds; consumes milli-watts or less
  - 8 bit (AVR, PIC, 8051) for very simple, cheap devices; 32 bit (ARM Cortex-M, RISC-V) for almost everything else
- **application processor** (MPU): fast cores with memory management unit, external DRAM, usually Linux or Android
  - ARM Cortex-A (smartphones, Raspberry Pi), increasingly RISC-V application cores
- **digital signal processor** (DSP): optimised for filter and transformation algorithms (audio, radio, motor control)
- **FPGA**: configurable logic, for highly parallel or very fast functions; often combined with processor cores (system on chip)
- modern chips combine several of these: e.g. Cortex-A cores for Linux plus a Cortex-M core for real time on the same die

Processors of this lecture (all simulated in Wokwi)

	ATmega328P	RP2040	ESP32-C3	STM32F103
board	Arduino Uno	Raspberry Pi Pico	ESP32-C3-DevKitM-1	"Blue Pill"
architecture	AVR, 8 bit	ARM Cortex-M0+, 32 bit	RISC-V RV32IMC, 32 bit	ARM Cortex-M3, 32 bit
cores, clock	1, 16 MHz	2, 125–200 MHz	1, 160 MHz	1, 72 MHz
program memory	32 KB flash	external QSPI flash (2 MB)	flash in the package (4 MB)	64 KB flash
RAM	2 KB	264 KB	400 KB	20 KB
special	EEPROM, 5 V	PIO, USB	Wi-Fi, Bluetooth LE	CAN, USB

- the successor of the RP2040, the RP2350 (Raspberry Pi Pico 2), offers two Cortex-M33 **or** two RISC-V cores on the same chip – selectable at boot

Von Neumann and Harvard architecture

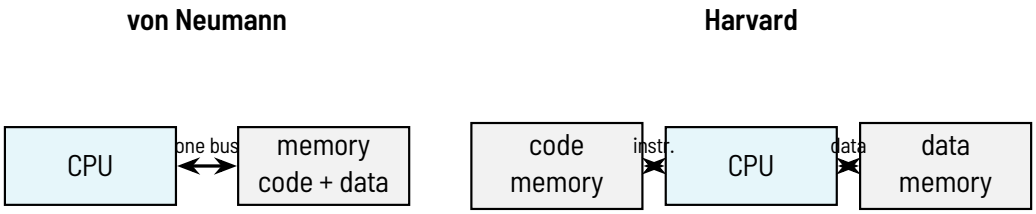


Figure 4.1.1: One bus for code and data, or separate buses

- **Harvard**: fetching the next instruction and accessing data at the same time – higher throughput
  - AVR: pure Harvard; constants in flash need special instructions (`PROGMEM`, `lpm`)
- **modified Harvard** (Cortex-M, RISC-V MCUs): separate instruction and data buses or caches, but **one** common address space – a pointer can point to flash or RAM

RISC and CISC

- **CISC** (complex instruction set computer, e.g. x86): many instructions, memory operands in arithmetic instructions, variable length
- **RISC** (reduced instruction set computer): all microcontroller architectures of this lecture
  - **load/store architecture**: arithmetic only on registers; memory accessed only with load and store
  - many general purpose registers (AVR 32 × 8 bit, Cortex-M 13 × 32 bit + SP, LR, PC, RISC-V 31 × 32 bit + zero register)
  - simple, mostly fixed length instructions ⇒ easy to decode and to pipeline
  - compressed instruction sets for small code: ARM Thumb (16 bit), RISC-V "C" extension
- **pipeline**: fetch, decode, execute overlap – ideally one instruction per clock
  - Cortex-M0+: 2 stages; Cortex-M3/M4: 3 stages; Cortex-M7: 6 stages, two instructions per clock
  - a jump empties the pipeline: branches cost extra cycles

The same C function on four processors

```
1  uint32_t sum(const uint16_t *a, int n) {
2      uint32_t s = 0;
3      for (int i = 0; i < n; i++) {
4          s += a[i];
5      }
6      return s;
7  }
```

processor	instructions in the loop	code size	reason
AVR (8 bit)	12	40 bytes	32 bit sum and 16 bit index need several 8 bit operations
Cortex-M0+	7	24 bytes	16 bit Thumb instructions
Cortex-M4	6	24 bytes	load with scaled index: `ldrh r4, [r0, r3, lsl #1]`
RISC-V RV32IMC	7	28 bytes	mix of 16 and 32 bit instructions

- compiled with `-Os` (optimise for size); listings and script in `examples/processors`, or try it on `https://godbolt.org`

The loop in assembly: AVR and Cortex-M0+

```
1  ; AVR: s in r22..r25 (4 x 8 bit), pointer in Z (r31:r30), i in r19:r18
2  .L3:  ld   r26,Z+            ; low byte of a[i], Z++
3        ld   r27,Z+            ; high byte of a[i], Z++
4        add  r22,r26           ; s += a[i], byte by byte with carry
5        adc  r23,r27
6        adc  r24,__zero_reg__
7        adc  r25,__zero_reg__
8        subi r18,-1            ; i++ (16 bit)
9        sbci r19,-1
10 .L2:  cp   r18,r20           ; i < n ?
11       cpc  r19,r21
12       brlt .L3
```

```
1  @ Cortex-M0+: a in r0, n in r1, i in r3, s in r2
2  .L3:  lsls  r4, r3, #1       @ byte offset = i * 2
3        ldrh  r4, [r0, r4]     @ load halfword a[i]
```

```
4       adds  r3, r3, #1        @ i++
5       adds  r2, r2, r4        @ s += a[i]
6       b     .L2
7 .L2:  cmp   r3, r1            @ i < n ?
8       blt   .L3
```

The loop in assembly: RISC-V

```
1 # RISC-V: a in a4 (copied from a0), n in a1, i in a5, s in a0 (= return value)
2 .L2:  blt   a5,a1,.L3         # i < n ? compare and branch in one instruction
3       ret
4 .L3:  slli  a3,a5,1           # byte offset = i * 2
5       add   a3,a4,a3          # address = a + offset
6       lhu   a3,0(a3)          # load halfword unsigned
7       addi  a5,a5,1           # i++
8       add   a0,a0,a3          # s += a[i]
9       j     .L2
```

- RISC-V has no status flags: compare and branch are one instruction (`blt`)
- only one addressing mode (register + 12 bit offset) – the address is computed with normal instructions
- register names of the calling convention: `a0`–`a7` arguments and return values, `sp` stack pointer, `ra` return address, `zero` always 0
- more about assembly programming in chapter Programming

4.2 Microcontroller Families

Most new designs today use one of two 32 bit architectures: ARM Cortex-M, licensed by Arm to almost every semiconductor company, or RISC-V, an open instruction set that anybody may implement without license fees. 8 bit microcontrollers are still produced in huge numbers for very simple and cost-sensitive devices. Because the cores are similar, the differences between microcontrollers are mostly in the peripherals, the memory, the power consumption and the software support.

8 bit AVR (ATmega328P)

- 32 registers of 8 bit, most instructions in one clock cycle, 16 MHz on the Arduino Uno
- pure Harvard architecture: 32 KB flash for code, 2 KB SRAM for data, 1 KB EEPROM
- 5 V operation, robust I/O pins, very low sleep current (well below 1 μA in power-down)
- limits: 16 or 32 bit arithmetic needs several instructions, no hardware division, little RAM
- still used for simple control tasks; and it made embedded programming popular through Arduino

ARM Cortex-M

- Arm designs the cores and licenses them; the chip manufacturers (ST, NXP, Raspberry Pi, Nordic, ...) add memory and peripherals
- the same core, the same tools and the same interrupt controller across thousands of chips

core	architecture	features
Cortex-M0 / M0+	ARMv6-M	smallest and most efficient; Thumb subset, no division instruction
Cortex-M3	ARMv7-M	full Thumb-2 instruction set, hardware division
Cortex-M4	ARMv7E-M	+ DSP instructions (SIMD, MAC), optional single precision FPU
Cortex-M7	ARMv7E-M	superscalar, caches, optional double precision FPU, up to about 1 GHz
Cortex-M23 / M33	ARMv8-M	+ TrustZone security extension (M33 also DSP and FPU)
Cortex-M55 / M85	ARMv8.1-M	+ Helium vector extension for signal processing and machine learning

- common to all: NVIC interrupt controller, SysTick timer, vector table, debug via SWD

RISC-V

- open instruction set architecture (since 2010, UC Berkeley; today RISC-V International) – no license fees
- modular: a small base set plus standard extensions
  - **RV32I**: 32 bit integer base, 32 registers (`x0` is always 0)
  - **M** multiplication/division, **A** atomic operations, **F/D** floating point, **C** compressed 16 bit instructions, **V** vectors
  - ESP32-C3 = RV32IMC
- many implementations: commercial (SiFive, Andes, Espressif) and open source (e.g. the Hazard3 cores in the RP2350)
- the same compilers (GCC, LLVM) and the same Rust targets as for ARM
- peripherals and interrupt controllers are not standardised as uniformly as on Cortex-M yet

Other architectures

- **Xtensa** (Cadence): configurable cores in the ESP32 and ESP32-S3; Espressif's newer chips use RISC-V
- **PIC**, **8051**: 8 bit, still common in cost-sensitive mass products
- **MSP430**: 16 bit, extremely low power (metering, sensors)
- manufacturer-specific automotive cores (e.g. Infineon TriCore, Renesas RH850): safety features such as cores running in lock-step

4.3 Inside a Microcontroller

The processor core is only a small part of a microcontroller. Around it are memories of different kinds, a bus system that connects the core with the peripherals, a clock system and a power management unit. How these parts work determines how fast a program really runs, how exactly its timing can be predicted and how long a battery lasts.

Block diagram of a microcontroller

- the bus matrix lets the CPU and the DMA access different memories at the same time

Memories

- **flash**: non-volatile program memory; erased in blocks, limited number of write cycles (typically 10 000–100 000)

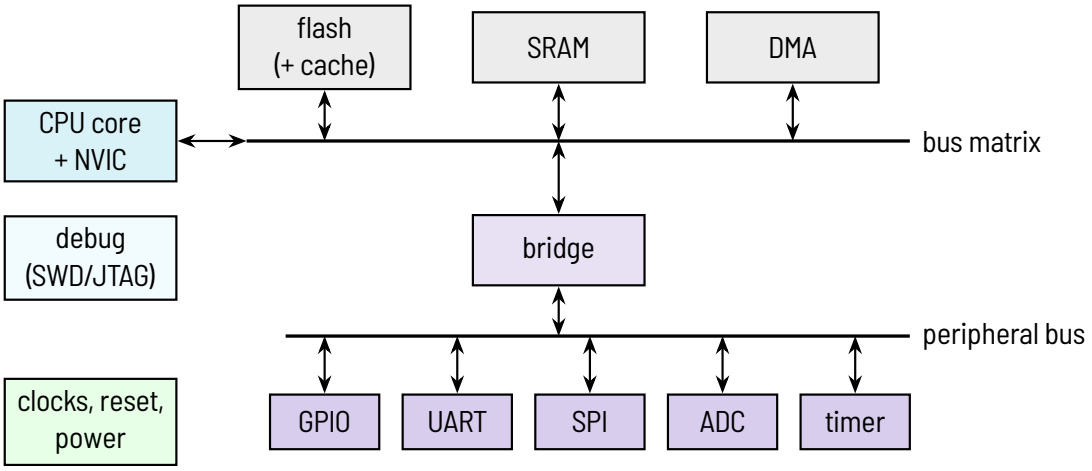


Figure 4.3.1: Typical structure of a 32 bit microcontroller

  - at high clock rates flash is too slow ⇒ wait states, prefetch buffers or a cache
  - RP2040 and ESP32 execute code directly from an external QSPI flash through a cache ("execute in place", XIP) – a cache miss costs many cycles
- **SRAM**: fast, volatile working memory for data, stack and time-critical code
- **EEPROM** or flash emulation: small, byte-wise changeable non-volatile data (settings, calibration)
- caches make programs faster on average, but timing less predictable ⇒ put real-time critical code into RAM (e.g. `IRAM_ATTR` on the ESP32, `__not_in_flash_func` on the RP2040)
- **MPU** (memory protection unit): protects memory regions (e.g. catches stack overflows); an MMU with virtual memory exists only on application processors

Memory map of the RP2040

address	content
0x0000 0000	boot ROM (16 KB)
0x1000 0000	external flash via XIP cache (code, constants)
0x2000 0000	SRAM (264 KB: data, stack, heap)
0x4000 0000	peripherals on the APB bus (UART, SPI, I2C, ADC, PWM, timer, ...)
0x5000 0000	peripherals on the AHB bus (DMA, USB, PIO)
0xd000 0000	SIO: single-cycle I/O (GPIO, hardware divider, spinlocks)
0xe000 0000	Cortex-M0+ internal registers (NVIC, SysTick)

- one 32 bit address space for everything: memory-mapped I/O (see chapter Peripherals and Interfaces)
- the linker places code and data into these regions according to the linker script (`memory.x` in our Rust examples)

Interrupts and exceptions

- the **vector table** at the start of the program contains the initial stack pointer and the addresses of all handlers (reset, faults, interrupts)
- Cortex-M: the hardware saves 8 registers on the stack before the handler starts, so a handler is an ordinary C or Rust function

- interrupt latency: 15 clock cycles on the Cortex-M0+, 12 on the Cortex-M3/M4
- **NVIC**: priorities, nesting, and **tail chaining** – a pending interrupt starts directly after the current one without restoring and saving the registers again
- **faults** (HardFault, MemManage, BusFault, UsageFault): invalid memory accesses, undefined instructions, division by zero – the fault handler is the place to log the error and reset
- RISC-V: similar concepts, but the interrupt controller depends on the implementation (CLIC, PLIC or vendor-specific)

Clock and power

- clock sources: internal RC oscillator (fast start, inaccurate), external crystal (accurate), PLL to multiply the frequency
- dynamic power $P \approx C \cdot U^2 \cdot f$: lower voltage and frequency save energy
- peripherals that are not used should have their clock switched off (clock gating)
- sleep modes: from "CPU stopped, peripherals running" to "everything off except a wake-up timer"
  - ESP32-C3 deep sleep: about 5 μA; Wi-Fi transmission: well above 100 mA
  - a battery device spends most of its time sleeping: average current = sum of (current $\times$ time share)
- example: 2000 mAh battery, 10 μA sleep and 1 second at 100 mA every 10 minutes $\Rightarrow$ average ≈ 0.18 mA $\Rightarrow$ about 15 months

Arithmetic: integer, fixed point, floating point

- without FPU (AVR, Cortex-M0+, ESP32-C3), floating point is done in software: tens to hundreds of cycles per operation
- **fixed point**: integers with an implied scale, e.g. Q15 = value $\times 2^{15}$ in 16 bit
  - fast on every processor, but the programmer must watch range and rounding
- FPU (Cortex-M4F, M7, M33, ESP32-S3): single precision float in one or a few cycles – `double` is still done in software if the FPU is single precision only
- **DSP features**: multiply-accumulate (MAC) in one cycle, saturating arithmetic, SIMD (two 16 bit or four 8 bit operations in one 32 bit instruction)
- dedicated DSPs add zero-overhead loops, circular addressing and several MAC units (VLIW) – today often replaced by Cortex-M4/M7 or vector extensions

4.4 Application Processors and Selection

When an application needs a graphical user interface, a camera, a lot of memory or complex network software, a microcontroller is no longer enough. Application processors run an operating system with virtual memory such as Linux. The last part of this chapter summarises their features and gives a checklist for choosing a processor.

Application processors

- ARM Cortex-A (and RISC-V application cores) with
  - **MMU**: virtual memory, every process has its own protected address space – required by Linux

    - several cores, GHz clock rates, multi-level caches, out-of-order execution in larger cores
    - external DRAM (hundreds of MB to several GB), storage on eMMC or SD card
- examples: Raspberry Pi 5 (4 × Cortex-A76, 2.4 GHz), NXP i.MX 8/9, TI AM62x, STM32MP1/MP2 (Cortex-A + Cortex-M on one chip)
- often with GPU, video codecs and neural network accelerators (NPU)
- non-deterministic timing (caches, DRAM, branch prediction) ⇒ hard real time on a separate core or MCU

Choosing a processor: checklist

- **function**: required peripherals (number of UARTs, ADC resolution, CAN, USB, Ethernet, radio)
- **performance**: computing power, FPU or DSP needed? Real-time requirements?
- **memory**: flash and RAM with reserve for later updates (rule of thumb: start with 50 % usage at most)
- **energy**: battery or mains? Sleep current, wake-up time
- **environment**: temperature range, automotive or industrial qualification, safety certification
- **software**: compilers, SDK, RTOS and Rust support, example code, community
- **business**: price at the planned quantity, availability for 10+ years, second source
- in practice the software ecosystem and the availability decide more often than the core itself

Exercises

- Compile `sum.c` on `https://godbolt.org` for the Cortex-M0+ with `-O0`, `-Os` and `-O2`. Compare the number of instructions in the loop.
- Change the type of `s` to `uint16_t`. Which processor profits most, and why?
- Change `a` to `const float *` and `s` to `float`. Which instructions does the compiler generate for the Cortex-M0+, the Cortex-M4 (`-mfloat-abi=hard -mfpu=fpv4-sp-d16`) and the ESP32-C3?
- Estimate the battery lifetime of a sensor that wakes up every minute for 50 ms at 20 mA and sleeps at 5 μA (battery 1000 mAh).
- Choose a microcontroller for a battery-powered weather station that sends data via Wi-Fi every 15 minutes. Justify your choice with the checklist.

5 Programming

5.1 From Source Code to Firmware

Embedded software is developed on a PC and runs on a different processor: it is cross-compiled. The toolchain translates the source code into machine code for the target, the linker places code and data at the addresses of the target's memory, and a programming tool writes the result into the flash. This chapter starts with these steps. Then it looks at the languages used in this lecture, from assembly through C and Rust to MicroPython, and ends with systematic debugging. Several examples run on the Raspberry Pi Pico in Wokwi again.

The build process

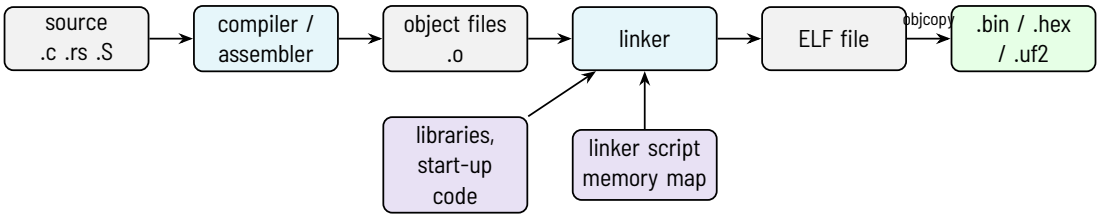


Figure 5.1.1: From source code to the image in the flash

- **cross compiler**, named after the target: `arm-none-eabi-gcc` (ARM, no OS), `riscv32-esp-elf-gcc`; in Rust a target triple such as `thumbv6m-none-eabi`
- the **linker** combines all object files and assigns addresses according to the linker script (`memory.x` in our Rust examples: flash at 0x1000 0000, RAM at 0x2000 0000)
- the **ELF** file contains code, data, symbols and debug information – the debugger needs it; the flash only needs the raw image

Sections and start-up

- `.text`: machine code (flash); `.rodata`: constants (flash)
- `.data`: initialised variables – the initial values are in flash, the variables in RAM
- `.bss`: variables initialised with zero (RAM only)
- the **start-up code** runs before `main`: set the stack pointer (from the vector table), copy `.data` from flash to RAM, fill `.bss` with zeros, set up clocks, then call `main`
- in Rust `cortex-m-rt` provides this code and the attribute `#[entry]`

```
$ llvm-size -A target/thumbv6m-none-eabi/release/gpio_blink   # addresses in hex
section        size        addr
.vector_table   192   0x10000100
.boot2          256   0x10000000      <- 2nd stage bootloader of the RP2040
.text          2940   0x100001c0
.rodata         360   0x10000b3c
.bss              4   0x20000000
```

- the same blink program with the Arduino core: about 314 KB code – the core always links USB, serial and file system support

Getting the program onto the chip

- **bootloader in ROM** or flash: receives the image over USB or UART
  - RP2040: hold BOOTSEL, the Pico appears as a USB drive – copy a `.uf2` file onto it
  - ESP32: ROM bootloader via the serial port (`esptool`, `espflash`)
- **debug probe** via SWD or JTAG (see section Debugging): programming and debugging with the same cable
- in production: programming on the panel, in-circuit test, unique keys and serial numbers per device
- **over the air** updates: the running firmware downloads a new image into a second flash partition and switches after a check – needs a bootloader that can roll back

5.2 Programming in Assembly

Today almost nobody writes complete programs in assembly. Compilers generate good code, and C and Rust are portable and much easier to maintain. Still, every embedded developer must be able to read assembly. You need it to understand what the compiler made of your code, to analyse a crash in the debugger, and to write the few parts that cannot be expressed in a high level language, such as start-up code, context switches and special instructions. We use the Cortex-M0+ of the RP2040 as our example. Its instruction set (Thumb, ARMv6-M) is small enough to be learned in a few hours.

Registers of the Cortex-M0+

- 16 registers of 32 bit
  - `r0`–`r12`: general purpose (most 16 bit instructions can only use `r0`–`r7`)
  - `r13` = `sp`: stack pointer (two of them: main and process stack)
  - `r14` = `lr`: link register, return address of a function call
  - `r15` = `pc`: program counter
- **xPSR** status register with the flags of the last arithmetic instruction
  - **N** negative, **Z** zero, **C** carry (unsigned overflow), **V** overflow (signed)
  - only instructions with an `s` suffix (`adds`, `subs`) and comparisons (`cmp`) set the flags
- all data processing works on registers – memory only via `ldr`/`str` (load/store architecture)

Instructions (selection)

group	examples	meaning
move	`movs r0, #5  mov r1, r2`	r0 = 5 (8 bit immediate), r1 = r2
arithmetic	`adds r0, r1, r2  subs r1, r1, #1  muls r0, r1`	r0 = r1 + r2, r1 = r1 - 1, r0 = r0 · r1
logic, shift	`ands r0, r1  orrs  eors  lsls r0, r0, #2`	bitwise AND, OR, XOR; shift left
compare	`cmp r1, #0  cmp r3, r1`	compute r1 - 0, set flags only
load	`ldr r0, [r1]  ldrh r3, [r0, #2]  ldrb r2, [r0, r1]`	32 / 16 / 8 bit from memory
store	`str r0, [r1, #4]  strh  strb`	32 / 16 / 8 bit to memory
constant	`ldr r0, =0xd000001c`	32 bit constant from a literal pool in flash
stack	`push {r4, lr}  pop {r4, pc}`	save registers / restore and return
branch	`b label  beq  bne  blt  bhs`	jump, conditional on the flags
call, return	`bl function  bx lr`	call (return address in lr), return

- conditions: `eq`/`ne` (Z), signed `lt`/`ge`/`gt`/`le`, unsigned `lo`/`hs`/`hi`/`ls`

Calling convention (AAPCS)

- the rules that let C, Rust and assembly functions call each other
- arguments in `r0`–`r3`, further arguments on the stack
- return value in `r0` (64 bit values in `r0`+`r1`)
- `r0`–`r3` and `r12` may be changed by the called function (caller-saved)
- `r4`–`r11` must have the same value after the call (callee-saved) – save them with `push` if you use them
- the stack grows downwards and is 8 byte aligned at function calls
- the RISC-V equivalent: arguments in `a0`–`a7`, return value in `a0`, callee-saved `s0`–`s11`, return address in `ra`

Our own assembly function

```
@ uint32_t asm_sum(const uint16_t *a, int n)    file sum_m0.S
@ AAPCS: a in r0, n in r1, result in r0. Only r0..r3 are used.
    .syntax unified
    .cpu cortex-m0plus
    .thumb
    .global asm_sum
    .type   asm_sum, %function
    .thumb_func
asm_sum:
    movs  r2, #0          @ s = 0
    cmp   r1, #0
    ble   2f              @ n <= 0: nothing to do
1:  ldrh  r3, [r0]        @ r3 = *a
    adds  r0, r0, #2      @ a++ (2 bytes)
    adds  r2, r2, r3      @ s += r3
    subs  r1, r1, #1      @ n--, sets the Z flag
    bne   1b              @ loop while n != 0
2:  movs  r0, r2          @ return s
    bx    lr
```

- 5 instructions in the loop instead of 7 from the compiler (chapter Processors): counting down saves the compare

Calling it from C and from Rust

```
extern "C" uint32_t asm_sum(const uint16_t *a, int n);   // C linkage

const uint16_t data[] = {1, 2, 3, 1000, 60000, 65535};
Serial1.printf("asm_sum = %lu\n", (unsigned long)asm_sum(data, 6));

uint32_t sp;
asm volatile("mov %0, sp" : "=r"(sp));   // inline assembly (GCC syntax)
```

```
global_asm!(".global asm_sum", /* ... the same instructions ... */);
unsafe extern "C" {
    fn asm_sum(a: *const u16, n: i32) -> u32;
}
let s_asm = unsafe { asm_sum(data.as_ptr(), data.len() as i32) };

let mut x: u32 = 40;
unsafe { asm!("adds {0}, {0}, {1}", inout(reg) x, in(reg) 2u32) };
```

- output in Wokwi (both languages): `asm_sum = 126541`, identical to the C and Rust versions
- Rust marks every call into assembly as `unsafe`: the compiler cannot check it

5.3 C for Embedded Systems

C is still the most widely used language for microcontrollers. It is close to the hardware, every chip vendor supports it, and almost all existing code is written in it. On the other hand C checks very little. Out of bounds

accesses, integer overflows and use of freed memory compile without complaint and lead to errors that are hard to find. Embedded C is therefore usually written following coding rules, compiled with all warnings enabled and checked with static analysis tools.

Data types and qualifiers

- `int` has 16 bit on the AVR and 32 bit on ARM ⇒ use the fixed width types of `<stdint.h>`: `uint8_t`, `int16_t`, `uint32_t`, …
- `volatile`: every access really happens – for hardware registers and variables shared with interrupts
- `const`: read only; on most MCUs `const` data stays in flash and saves RAM
- `static` at file level: visible only in this file – the C way of encapsulation
- integer pitfalls
  - signed overflow is undefined behaviour: the compiler may assume it never happens
  - shifting by the width of the type or more is undefined: `1 << 32`
  - integer promotion: `uint8_t a = 200, b = 100; if (a + b > 255)` is true, because the sum is calculated as `int`

Registers as structures

```
typedef struct {
    volatile uint32_t cpuid;          /* 0x000 */
    volatile uint32_t gpio_in;        /* 0x004 */
    volatile uint32_t gpio_hi_in;     /* 0x008 */
    uint32_t _reserved;               /* 0x00c */
    volatile uint32_t gpio_out;       /* 0x010 */
    volatile uint32_t gpio_out_set;   /* 0x014 */
    volatile uint32_t gpio_out_clr;   /* 0x018 */
    volatile uint32_t gpio_out_xor;   /* 0x01c */
    volatile uint32_t gpio_oe;        /* 0x020 */
    volatile uint32_t gpio_oe_set;    /* 0x024 */
} sio_hw_t;

#define sio_hw ((sio_hw_t *)0xd0000000u)

/* The compiler checks the offsets for us */
_Static_assert(__builtin_offsetof(sio_hw_t, gpio_out_xor) == 0x01c, "offset");

void led_toggle(unsigned pin) { sio_hw->gpio_out_xor = 1u << pin; }
```

- the style of the Pico SDK and of the CMSIS headers of all ARM vendors; generated from the chip description (SVD file)

Bit manipulation

```
reg |=  (1u << 5);            // set bit 5
reg &= ~(1u << 5);            // clear bit 5
reg ^=  (1u << 5);            // toggle bit 5
if (reg & (1u << 5)) { }      // test bit 5

/* write a field of several bits (read - modify - write) */
static inline uint32_t set_field(uint32_t reg, unsigned shift, uint32_t mask, uint32_t
    value) {
```

```
    return (reg & ~(mask << shift)) | ((value & mask) << shift);
}
ctrl = set_field(ctrl, 4, 0x3u, 2u);    /* bits 5..4 := 2 */
```

- always use unsigned constants (`1u`) – `1 << 31` overflows a signed `int`
- read-modify-write is not atomic: protect it against interrupts, or use SET/CLR registers if the hardware offers them
- C bit fields (`uint32_t mode : 2;`) are not portable for registers: the order of the bits is implementation-defined

Rules for reliable C

- compile with all warnings and treat them as errors: `-Wall -Wextra -Werror`
- no dynamic memory after start-up (`malloc`/`free`): fragmentation and failing allocations at run time
- no recursion: the stack usage must be known
- check every return value and every array index
- coding standards: **MISRA C** (automotive and industry), CERT C (security) – they forbid the dangerous constructs of the language
- static analysis (e.g. `cppcheck`, `clang-tidy`, commercial tools), sanitizers and unit tests on the PC
- these rules catch many errors – Rust makes most of them impossible by construction (next section)

5.4 Rust for Embedded Systems

Rust is a systems programming language with the performance of C and without a garbage collector, but with memory safety checked by the compiler. A Rust program without `unsafe` blocks has no dangling pointers, no out of bounds accesses and no data races. For embedded systems it adds more: the type system can express hardware states, as we saw with the GPIO pins in chapter 6. Cargo makes libraries and builds easy, and a large ecosystem of hardware abstraction crates exists. Rust is now used in the Linux kernel, and a qualified compiler (Ferrocene) is available for safety-critical products. This section summarises the concepts we used in the previous chapters and shows how to test embedded code on the PC.

Ownership and borrowing in five rules

- every value has exactly one **owner**; when the owner goes out of scope, the value is dropped (memory, peripherals, locks are released)
- assignment or passing to a function **moves** the value: the old variable can no longer be used
- instead of moving, a value can be **borrowed**: `&T` (read, many at a time) or `&mut T` (write, only one at a time)
- a reference can never live longer than the value it points to
- consequence: no use after free, no double free, and no two parts of a program changing the same data at the same time

```
let led = pins.gpio15.into_push_pull_output();
spawner.spawn(blink(led).unwrap());    // led is moved into the task
led.set_high();                        // error[E0382]: borrow of moved value: `led`
```

Embedded Rust: no_std and the layers

- `#![no_std]`: only the `core` library (no heap, no files, no threads); `alloc` can be added with an allocator
- `#![no_main]` and `#[entry]`: the start-up code calls our function; it never returns (`-> !`)
- a **panic handler** decides what happens on a panic: halt, print a backtrace, reset

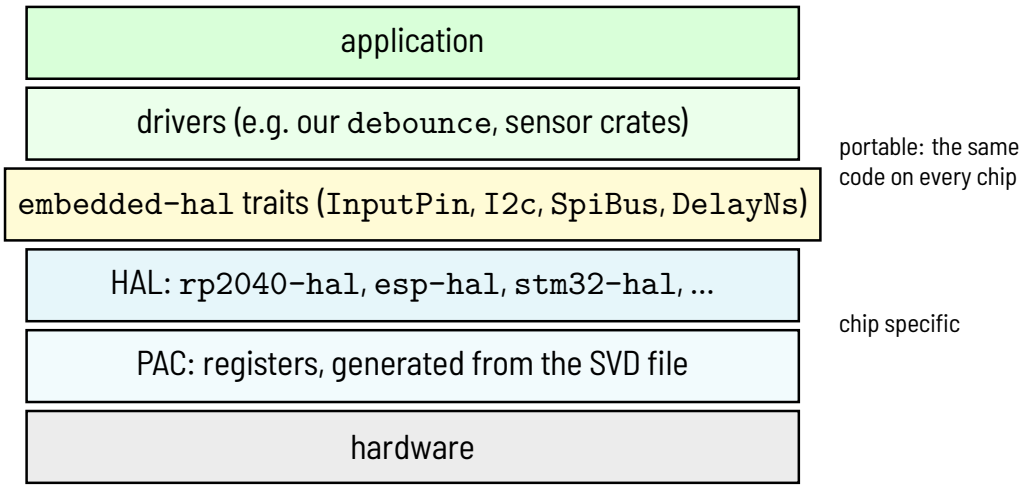


Figure 5.4.1: Layers of the embedded Rust ecosystem

A hardware independent driver

```
#![no_std]
use embedded_hal::digital::InputPin;
pub enum Edge { None, Pressed, Released }
pub struct Debouncer { stable: bool, counter: u8, threshold: u8 }  // threshold = samples

impl Debouncer {
    pub fn update(&mut self, raw: bool) -> Edge {
        if raw == self.stable {
            self.counter = 0;
            return Edge::None;
        }
        self.counter += 1;
        if self.counter < self.threshold {
            return Edge::None;
        }
        self.counter = 0;
        self.stable = raw;
        if raw { Edge::Pressed } else { Edge::Released }
    }

    pub fn poll<P: InputPin>(&mut self, pin: &mut P) -> Result<Edge, P::Error> {
        Ok(self.update(pin.is_low()?))    // any pin type that implements InputPin
```

```
    }
}
```

Testing embedded code on the PC

```
#[test]
fn bouncing_gives_one_edge() {
    let mut d = Debouncer::new(3);
    // contact bounce: short pulses before the level is stable
    let samples = [true, false, true, false, true, true, true, true, true];
    assert_eq!(run(&mut d, &samples), (1, 0));
}

/// A fake pin for tests: implements the same trait as a real GPIO pin.
struct FakePin(bool);
impl InputPin for FakePin { /* is_high / is_low return self.0 */ }
```

```
$ cargo test
test tests::bouncing_gives_one_edge ... ok
test tests::clean_press_and_release ... ok
test tests::poll_reads_active_low_pin ... ok
test tests::short_glitch_is_ignored ... ok
test result: ok. 4 passed; 0 failed
```

- the same library runs unchanged on the Pico (`button_lib.rs`) – logic is tested quickly on the PC (or Replit), only the hardware access needs the target

Rust tools and practices

- `rustup target add thumbv6m-none-eabi` / `riscv32imc-unknown-none-elf`: cross compilation without a separate toolchain
- `cargo`: build, dependencies (`Cargo.toml`, `Cargo.lock` for reproducible builds), tests, documentation
- `cargo clippy` (lints), `cargo fmt` (formatting)
- `probe-rs`: flashing and debugging via SWD/JTAG; `defmt`: compact logging, formatted on the PC
- errors are values: `Result<T, E>` and `Option<T>`, propagated with `?`; `unwrap()` only where an error really cannot occur
- `unsafe` for what the compiler cannot check (registers, assembly, FFI to C) – keep it small and encapsulated in safe functions
- C and Rust can be mixed: `extern "C"` in both directions

5.5 Python and MicroPython

Python plays two roles in embedded development. On the PC it is the standard language for tools: test scripts, data analysis, communication with devices, build automation. On the microcontroller itself, MicroPython (and

its variant CircuitPython) implements Python 3 in about 256 KB of flash. The program is interpreted, so it is much slower than C or Rust and the garbage collector causes pauses. But changes can be tried out at once in the interactive prompt. This makes MicroPython excellent for prototypes, teaching and devices where the computing time does not matter.

Python 3 on the PC: working with binary data

```
import struct

# 14 bytes from register 0x3B: accel X/Y/Z, temperature, gyro X/Y/Z
raw = bytes.fromhex("0000 0000 4000 f24c 0000 0083 ff7d")

ax, ay, az, temp, gx, gy, gz = struct.unpack(">7h", raw)    # > = big endian, h = int16

accel_g = [v / 16384 for v in (ax, ay, az)]       # +-2 g range: 16384 LSB/g
temp_c = temp / 340 + 36.53                       # formula from the datasheet
print(f"acceleration: {accel_g} g")               # [0.0, 0.0, 1.0] g
print(f"temperature:  {temp_c:.1f} C")            # 26.2 C

packet = struct.pack("BB", 0x6B, 0x00)            # build an I2C write
```

- typical uses: serial port (`pyserial`), MQTT clients (`paho-mqtt`), test automation (`pytest`), plots (`matplotlib`), Jupyter notebooks for measurements
- always Python 3 – Python 2 has not been maintained since 2020

MicroPython on the Pico

```
from machine import ADC, PWM, Pin
import time

pot = ADC(26)
pwm = PWM(Pin(15))
pwm.freq(1000)                  # 1 kHz

while True:
    value = pot.read_u16()      # 0 .. 65535 (12 bit ADC, scaled to 16 bit)
    pwm.duty_u16(value)         # duty cycle 0 .. 100 %
    print(f"ADC {value:5d}  ->  duty {value * 100 // 65535:3d} %")
    time.sleep_ms(200)
```

- the module `machine` wraps the peripherals: `Pin`, `ADC`, `PWM`, `I2C`, `SPI`, `UART`, `Timer`
- interrupts: `button.irq(trigger=Pin.IRQ_FALLING, handler=f)`; registers: `machine.mem32[0x = 1 << 15`
- runs directly in the browser: new MicroPython Pico project on `https://wokwi.com`
- limits: 10 to 100 times slower than C, garbage collection pauses, more RAM – time-critical parts can be written in C as native modules

5.6 Debugging

Debugging embedded systems is harder than debugging PC programs. The program runs on another device, often without a screen. Timing matters and changes when you observe it, and the error may lie in the hardware, in the software or between them. A systematic procedure and the right tools save a lot of time. This section describes both, with a real example from the preparation of this lecture.

A systematic approach

- **reproduce**: find a way to make the error appear reliably – an error that cannot be reproduced cannot be verified as fixed
- **observe**: collect facts (output, signals, register values) instead of guessing
- **hypothesis**: one possible cause that explains all observations
- **experiment**: a test that confirms or refutes the hypothesis – change one thing at a time
- **isolate**: halve the search space (comment out half of the code, `git bisect` to find the commit that introduced the error)
- **fix and secure**: correct the cause, not the symptom; add a test that would have found the error
- write down what you have tried – especially when the error takes days

A real case: the flaky button

- symptom: the Rust debouncing example (chapter 6) toggled the LED in some simulation runs and not in others; the C version always worked
- observation 1: the debouncer library passes all unit tests ⇒ the logic is probably correct
- observation 2 (GDB): the breakpoint where a new level is accepted is never reached
- observation 3 (diagnostic output with timestamps): dozens of "pressed"/"released" events within **one** millisecond – the loop runs much faster than once per millisecond
- hypothesis: `delay.delay_ms(1)` does not wait 1 ms
- experiment: measure the delays with the RP2040 hardware timer
  - `delay 1 ms: SysTick 4 us, TIMER 1000 us`
  - `delay 500 ms: SysTick 365788 us, TIMER 500001 us`
- cause: the SysTick-based delay does not work correctly in the simulator; fix: use the RP2040 timer as delay provider (it implements the same `DelayNs` trait)

Tools without a debugger

- **output** via UART or USB (`printf`, `println!`, logging frameworks)
  - simple and powerful, but changes the timing: a line at 115200 baud takes milliseconds
  - faster alternatives: binary logging (`defmt`), RTT (output through the debug probe, no UART needed)
- **GPIO toggles** + logic analyzer or oscilloscope: measure timing in microseconds with almost no influence
- **assertions**: `assert()` / `assert!()` check assumptions and stop early instead of failing later
- **fault handlers**: on a HardFault or panic print the program counter and a backtrace (`esp-backtrace` in our ESP32 examples)
- **watchdog** and reset reason: find out that and why the device restarted

GDB: the debugger

command	function
`target remote localhost:3333`	connect to the debug server (probe, OpenOCD, probe-rs, Wokwi)
`break func` / `break file.rs:42`	breakpoint
`continue`, `next`, `step`, `finish`	run, next line, into a function, out of the function
`stepi`, `display/i $pc`	one machine instruction, show the next instruction
`backtrace`	call stack
`print var`, `info registers`	variables and registers
`x/6hu addr`	memory: 6 halfwords, unsigned decimal
`watch var`	watchpoint: stop when the variable is written

- GDB needs the ELF file with debug information (`debug = 2` in our `Cargo.toml`)
- graphical front ends: VS Code (Cortex-Debug, probe-rs extension), Eclipse-based IDEs

GDB session with the Wokwi simulator

```
$ wokwi-cli -g 3333 .                       # simulation waits for GDB
$ arm-none-eabi-gdb target/thumbv6m-none-eabi/release/asm_sum
(gdb) target remote localhost:3333
(gdb) break asm_sum
(gdb) continue
Breakpoint 1, 0x100001fe in asm_sum ()
(gdb) info registers r0 r1
r0             0x2003ffd4          537133012      <- address of the array
r1             0x6                 6              <- n
(gdb) x/6hu $r0
0x2003ffd4:     1       2       3       1000    60000   65535
(gdb) stepi 4
(gdb) display/i $pc
1: x/i $pc
=> 0x10000206 <asm_sum+10>:     adds    r2, r2, r3
(gdb) finish
0x10000354 in asm_sum::__cortex_m_rt_main () at src/bin/asm_sum.rs:67
67      let s_asm = unsafe { asm_sum(data.as_ptr(), data.len() as i32) };
```

- in VS Code: "Wokwi: Start Simulator and Wait for Debugger", then start a GDB debug session – breakpoints in the source code

Debugging in the browser: a wrong result

- JSLinux "Alpine Linux" (x86_64) at `https://bellard.org/jslinux/` contains GCC and GDB – nothing to install
- program `examples/jslinux/average.c`: prints `average = 8 (expected 200)`

```
$ gcc -g -O0 -o average average.c && gdb -q ./average
(gdb) set can-use-hw-watchpoints 0      # software watchpoint, safe in an emulator
(gdb) break average
(gdb) run
(gdb) next                              # sum = 0 is now set
(gdb) watch sum                         # stop whenever sum changes
(gdb) continue
Old value = 0 '\000'
New value = 200 '\310'
(gdb) continue
```

```
Old value = 200 '\310'
New value = 154 '\232'                    # 200 + 210 = 410 > 255: uint8_t overflows
```

Debugging in the browser: a crash

- program `examples/jslinux/config.c`: prints two settings, then crashes

```
$ gcc -g -O0 -o config config.c && gdb -q ./config
(gdb) run
rate      = 100
threshold = 42
Program received signal SIGSEGV, Segmentation fault.
0x... in main () at config.c:23
23      printf("limit     = %d\n", find("limit")->value);
(gdb) backtrace
#0  0x... in main () at config.c:23
(gdb) print find("limit")                 # GDB can call functions of the program
$1 = (struct setting *) 0x0
(gdb) print *find("rate")
$2 = {key = 0x... "rate", value = 100}
```

- `find()` returns `NULL` for an unknown key, and `main()` uses the result without checking it – the classic null pointer dereference
- fix it, and think about how Rust's `Option` would have prevented it

Debugging a Linux program without hardware

- on embedded Linux the debugger works on a process, not on the whole chip
- QEMU user mode runs the ARM program on the PC and offers a GDB server (`-g`); the debugger of the Yocto SDK connects to it (chapter Operating Systems, `examples/linux`)

```
$ qemu-aarch64 -L $SDKTARGETSYSROOT -g 1234 ./sensor_avg &      # waits for GDB
$ $GDB ./sensor_avg                                            # aarch64-poky-linux-
    gdb
(gdb) set sysroot /path/to/sdk/sysroots/cortexa57-poky-linux
(gdb) target remote :1234
(gdb) break sensor_avg.c:18
(gdb) continue
(gdb) display sum
(gdb) continue
1: sum = 200 '\310'           <- after the first value
(gdb) continue
1: sum = 154 '\232'           <- 200 + 210 = 410 does not fit into uint8_t: 410 - 256
(gdb) continue
1: sum = 88 'X'
```

With `-Og` the variable `sum` lives in a register, so a watchpoint on it fails; `display` at a breakpoint works.

On a real Linux board: gdbserver

- the program runs on the board under `gdbserver`; GDB runs on the PC with the symbols and the sources

```
board$ gdbserver :1234 ./sensor_avg
pc$    $GDB ./sensor_avg
(gdb)  target remote 192.168.7.2:1234
```

- the board needs `gdbserver` in the image (Yocto: `IMAGE_INSTALL:append = " gdbserver"`; the prebuilt `core-image-sato-sdk` contains it)
- the binary on the board may be stripped; GDB on the PC uses the unstripped copy and the SDK sysroot for the libraries
- crashes after the fact: enable core dumps on the board, analyse them on the PC with `$GDB ./program core`
- kernel problems: messages with `dmesg`, kernel debugging via QEMU's GDB stub (`-s -S`) or JTAG

Hardware debugging: JTAG and SWD

- on real hardware the debugger talks to the debug unit in the chip through a **debug probe**
- **JTAG** (IEEE 1149.1): signals TCK (clock), TMS (mode), TDI (data in), TDO (data out)
  - originally designed to test soldered boards: **boundary scan** reads and sets every pin of a chip through a shift register chain – without the chip's software
  - a 16-state machine (TAP controller), driven by TMS, selects instruction and data registers
  - several chips can be chained
- **SWD** (serial wire debug, ARM): the same debug functions with only two pins (SWDIO, SWCLK) – RP2040, STM32 and all Cortex-M
- functions: halt, single step, read and write registers and memory, hardware breakpoints and watchpoints, programming the flash
- probes: Raspberry Pi Debug Probe and other CMSIS-DAP probes, SEGGER J-Link, ST-LINK; the ESP32-C3 has a USB-JTAG unit built in
- software: OpenOCD or `probe-rs` as GDB server

Exercises

- Look at the sections of one of your programs with `size` or `llvm-size`. Add a global array `int a[100] = {1};` – which section grows? And with `= {0}`?
- Write `asm_max(const int32_t *a, int n)` in Thumb assembly (signed maximum) and call it from C or Rust. Test with negative numbers.
- Step through `asm_sum` with GDB and watch the flags in `xpsr` after `subs`.
- Add a test to the `debounce` crate: a press that is shorter than the threshold must not produce an edge. Run `cargo test` on Replit.
- Port `adc_pwm.py` to C and to Rust (chapter 6 contains all parts). Compare code size and effort.
- In C: why is `(uint8_t)200 + (uint8_t)100 > 255` true? Check it on the PC.
- Linux: debug `examples/linux/sensor_avg.c` with QEMU user mode and the GDB of the Yocto SDK. Fix the bug and check the result on the emulated board.

6 Peripherals and Interfaces

6.1 Peripherals of a Microcontroller

A microcontroller is more than a processor core. Most of the chip area is used by peripherals: blocks of hardware that talk to the outside world on their own, such as GPIO ports, timers, PWM generators, analog-to-digital converters and serial interfaces. The processor only configures them and exchanges data with them. It does so by reading and writing registers that appear at fixed addresses in the memory map.
In this chapter we use the Raspberry Pi Pico with the RP2040 microcontroller as our reference platform. It is cheap, very well documented and, most importantly for the lecture, it can be simulated completely in the browser with Wokwi. All examples exist in two versions: in C using the Pico SDK (running in the browser) and in Rust using the rp2040-hal crate (built locally and simulated in Wokwi for VS Code). The concepts carry over to any other microcontroller; only register names and addresses change.

The reference platform: RP2040 / Raspberry Pi Pico

- two ARM Cortex-M0+ cores, 125 MHz (up to 200 MHz), 264 KB SRAM, program in external QSPI flash (2 MB on the Pico)
- 30 GPIO pins (26 usable on the Pico), 3.3 V logic, **not** 5 V tolerant
- peripherals
  - 2 × UART, 2 × I²C, 2 × SPI, USB 1.1 (device and host)
  - 8 PWM slices with 2 outputs each (16 PWM channels)
  - 12 bit ADC with 4 external inputs and an internal temperature sensor
  - 64 bit microsecond timer with 4 alarms, watchdog, real-time clock
  - 12 DMA channels, 2 PIO blocks (programmable I/O state machines)

Memory-mapped I/O

- every peripheral is a block of **registers** in the address space
- the CPU accesses them with normal load/store instructions
- typical kinds of registers
  - **control** registers: mode, enable, speed, ...
  - **status** registers: flags like "data received", "FIFO full"
  - **data** registers: the values transferred to or from the outside world
- a register is not memory: reading may clear a flag, writing may start an action
  - in C every access has to be `volatile`, otherwise the compiler may remove or merge it

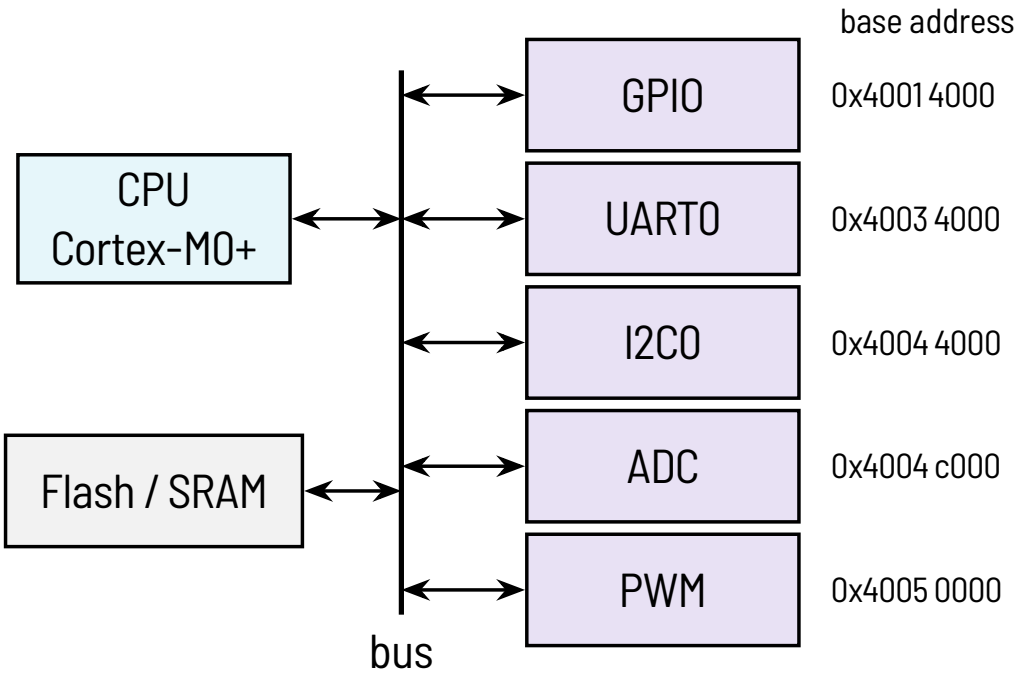


Figure 6.1.1: Peripherals are register blocks in the memory map (RP2040 addresses)

Before a peripheral can be used

- release it from **reset** (most blocks start in reset to save power)
- make sure it gets a **clock** (e.g. `clk_peri` for UART and SPI)

- connect it to the pins: **pin multiplexing**
  - a pin has many possible functions, but only one at a time
  - on the RP2040: `FUNCSEL` field in the register `GPIOn_CTRL`
  - e.g. GP0 can be SPI0 RX, UART0 TX, I2C0 SDA, PWM0 A, SIO (software GPIO), PIO, ...
- configure the peripheral, then enable it
- the SDK (C) and the HAL (Rust) do these steps for us – but they still happen

Polling, interrupts and DMA

- **polling**: the program checks a status flag in a loop
  - simple and predictable, but the CPU is busy while waiting
- **interrupt**: the peripheral signals an event, the CPU interrupts the running program and executes an interrupt service routine (ISR)
  - the CPU can sleep or do other work in between
  - shared data between ISR and main program needs care
- **DMA** (direct memory access): a separate controller copies data between peripheral and memory
  - the CPU only sets up the transfer and is informed at the end
  - used for fast or continuous data streams (ADC sampling, displays, audio)

Our simulated circuit (Wokwi)

- one circuit for all examples of this chapter: `examples/interfaces/diagram.json`

Pico pin	connected to	peripheral
GP0 / GP1	serial monitor	UART0 TX / RX
GP4 / GP5	MPU6050 acceleration sensor	I2C0 SDA / SCL
GP14	push button to GND	GPIO input
GP15	LED with 220 Ω resistor	GPIO output, PWM7 B
GP17 / GP18 / GP19	74HC595 shift register + LED bar	GPIO latch, SPI0 SCK / TX
GP26	potentiometer	ADC0

- a logic analyzer records 8 of these signals (export as VCD file)
- C: paste sketch and `diagram.json` into a new Pico project on `https://wokwi.com`
- Rust: `cargo build --release`, then start the simulation with the Wokwi extension in VS Code

6.2 Digital Inputs and Outputs (GPIO)

General purpose input/output (GPIO) is the simplest interface: the software decides whether a pin drives a level (output) or reads one (input). Even this simple case shows the typical structure of every peripheral. There is a multiplexer that connects the pin to one of several internal functions, a pad with electrical options such as pull resistors and drive strength, and registers to control everything.

Structure of a GPIO pin

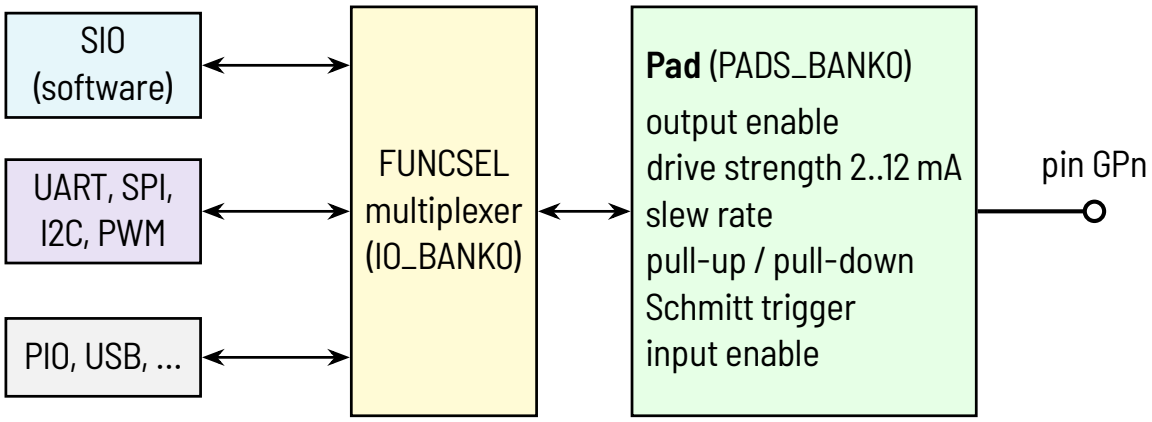


Figure 6.2.1: Path from the peripherals to a GPIO pin of the RP2040

- FUNCSEL = 5 connects the pin to the **SIO** (single-cycle I/O): the CPU controls it directly
- the pad settings are independent of the selected function

SIO registers for GPIO (base address 0xd000 0000)

Offset	Register	Meaning (one bit per pin)
0x004	GPIO_IN	input levels
0x010	GPIO_OUT	output values
0x014, 0x018, 0x01c	GPIO_OUT_SET, _CLR, _XOR	set / clear / toggle output bits
0x020	GPIO_OE	output enable (1 = output)
0x024, 0x028, 0x02c	GPIO_OE_SET, _CLR, _XOR	set / clear / toggle enable bits

- why SET, CLR and XOR registers?
  - `GPIO_OUT |= (1 << 15)` is a read-modify-write: three instructions
  - if an interrupt (or the second core) changes another bit in between, that change is lost
  - writing `1 << 15` to GPIO_OUT_SET changes only bit 15, in one bus access
- the other peripherals of the RP2040 offer the same idea for every register: aliases at +0x1000 (XOR), +0x2000 (SET), +0x3000 (CLR)

Blinking with raw register accesses (C)

```c
#define IO_BANK0_ADDR   0x40014000u
#define SIO_ADDR        0xd0000000u

// GPIOn_CTRL: function select of pin n (8 bytes per pin, CTRL at offset 4)
#define GPIO_CTRL(n)    (*(volatile uint32_t *)(IO_BANK0_ADDR + 0x04u + 8u * (n)))
#define SIO_GPIO_OE_SET (*(volatile uint32_t *)(SIO_ADDR + 0x024u))
#define SIO_GPIO_OUT_XOR (*(volatile uint32_t *)(SIO_ADDR + 0x01cu))

#define FUNCSEL_SIO     5u
#define LED             15u

void setup() {
  GPIO_CTRL(LED)  = FUNCSEL_SIO;    // pin is controlled by software (SIO)
  SIO_GPIO_OE_SET = 1u << LED;      // enable output driver (atomic set)
```

```
}

void loop() {
  SIO_GPIO_OUT_XOR = 1u << LED;     // toggle - no read-modify-write needed
  delay(500);
}
```

The same with the Pico SDK (C)

```
#include "hardware/gpio.h"

const uint LED = 15;

void setup() {
  gpio_init(LED);                // FUNCSEL = SIO, output disabled, value 0
  gpio_set_dir(LED, GPIO_OUT);   // enable the output driver
}

void loop() {
  gpio_put(LED, 1);
  sleep_ms(500);
  gpio_put(LED, 0);
  sleep_ms(500);
}
```

- the SDK functions are thin wrappers around the registers shown before
- `gpio_put()` writes to GPIO_OUT_SET or GPIO_OUT_CLR

The same in Rust (rp2040-hal)

```
#[entry]
fn main() -> ! {
    // Take ownership of the peripherals - exactly once.
    let mut pac = pac::Peripherals::take().unwrap();
    // ... clock setup (12 MHz crystal -> PLL -> 125 MHz) ...
    let sio = hal::Sio::new(pac.SIO);
    let pins = rp_pico::Pins::new(pac.IO_BANK0, pac.PADS_BANK0, sio.gpio_bank0, &mut
    pac.RESETS);

    // The type system knows that this pin is now a push-pull output.
    let mut led = pins.gpio15.into_push_pull_output();

    loop {
        led.set_high().unwrap();
        delay.delay_ms(500);
        led.set_low().unwrap();
        delay.delay_ms(500);
    }
}
```

What Rust adds: ownership and type states

- `Peripherals::take()` returns the peripherals only on the first call
  - there is exactly one owner of each register block, so two parts of the program cannot configure the same peripheral by accident
- the configuration of a pin is part of its **type**
  - `pins.gpio15` has the type `Pin<Gpio15, FunctionNull, PullDown>`
  - `into_push_pull_output()` consumes it and returns `Pin<Gpio15, FunctionSioOutput, PullDown>`
  - `set_high()` only exists for output pins: writing to an input pin is a **compile error**, not a bug found on the hardware
  - after `into_function::<FunctionUart>()` the pin can only be handed to a UART
- all of this is checked by the compiler and costs nothing at run time
- raw register access is still possible, but must be marked `unsafe` (see `gpio_blink_registers.rs`)

Output stages

- **push-pull**: one transistor connects the pin to the supply, the other to GND
  - drives both levels actively, fast edges in both directions
  - never connect two push-pull outputs: if one drives high and the other low, a large current flows and can destroy the pins

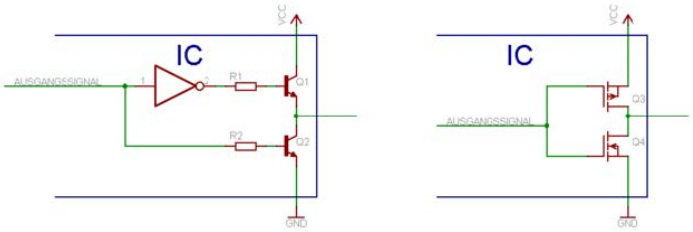


Figure 6.2.2: Push-pull output

- **open drain** (open collector for bipolar transistors): only the transistor to GND exists
  - the pin can only pull the line low; a resistor pulls it high
  - several outputs may share one line: it is low if any output pulls it low ("wired AND")
  - the rising edge is slow, as the pull-up resistor charges the line capacitance ($\tau = RC$)
  - the high level may differ from the chip's supply (level shifting, e.g. 3.3 V to 5 V)
  - used for I^{2}C, interrupt lines, reset lines

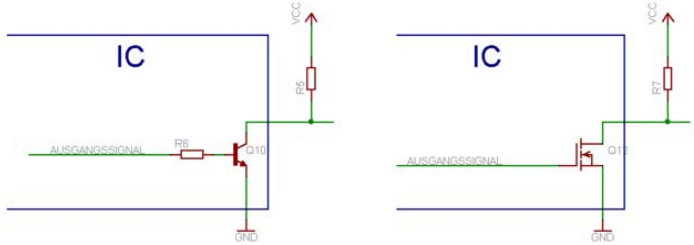


Figure 6.2.3: Open-drain output with external pull-up resistor

- **tri-state**: a push-pull stage with an additional "off" state (high impedance, Z)
  - in state Z the pin neither drives high nor low and behaves as if it were not connected
  - many devices can share a bus as long as only one of them drives at a time
  - on the RP2040: output enable = 0 means Z
  - open drain can be emulated: output value fixed at 0, switch only the output enable

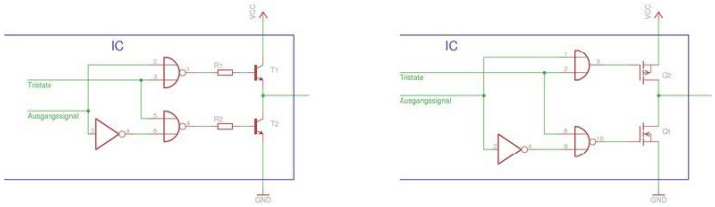


Figure 6.2.4: Tri-state output

Inputs

- an unconnected ("floating") input reads random values: it picks up noise
  - always define the idle level with a pull-up or pull-down resistor
  - RP2040: internal pull-up / pull-down of about 50–80 kΩ in the pad
- a typical button connects the pin to GND and uses the pull-up: **pressed = low**
- **Schmitt trigger**: two switching thresholds (hysteresis), so a slowly changing or noisy signal gives one clean edge instead of many
- electrical limits matter
  - 3.3 V maximum on any pin of the RP2040 – a 5 V signal needs a level shifter or voltage divider
  - drive strength 2, 4 (default), 8 or 12 mA per pin – enough for an LED, never for a motor or relay

Contact bounce

- mechanical contacts do not close cleanly: they bounce for some 100 μs up to about 20 ms
- the software sees many edges for one press – Wokwi simulates this as well

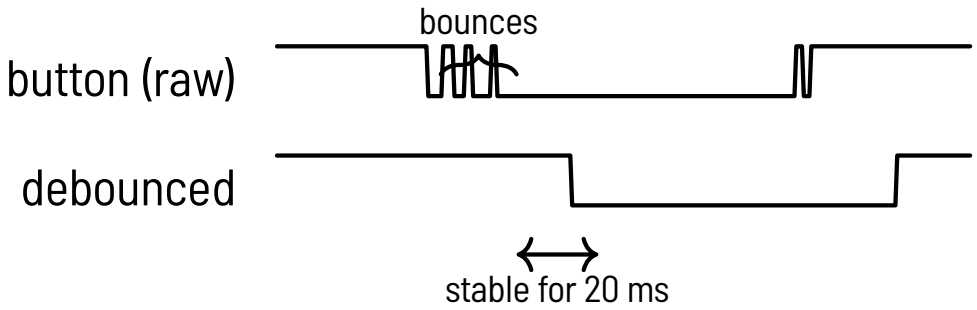


Figure 6.2.5: Bouncing button and debounced signal

- hardware solution: RC low-pass filter followed by a Schmitt trigger
- software solution: accept a new level only if it was stable for a certain time

Debouncing in software (Rust)

```rust
let mut led = pins.gpio15.into_push_pull_output();
// Button connects the pin to GND -> use the internal pull-up, pressed = low.
let mut button = pins.gpio14.into_pull_up_input();

let mut stable_pressed = false; // debounced state
let mut counter = 0; // how long the raw level differs from the stable state

loop {
    let raw_pressed = button.is_low().unwrap();
    if raw_pressed != stable_pressed {
        counter += 1;
        if counter >= STABLE_MS {
            stable_pressed = raw_pressed;
            counter = 0;
            if stable_pressed {
                led.toggle().unwrap(); // react on the press edge only
            }
        }
    } else {
        counter = 0;
    }
    delay.delay_ms(1); // sample every 1 ms
}
```

- C version: `examples/interfaces/c/button_debounce`

6.3 Interrupts

Polling a button every millisecond works, but it keeps the processor busy and the reaction time depends on the loop. With an interrupt the hardware itself watches the pin. When the selected edge occurs, the processor finishes the current instruction, saves its state and jumps to an interrupt service routine (ISR). Afterwards the interrupted program continues as if nothing had happened. This is efficient, but it brings a new class of errors. The ISR and the main program run "at the same time" from the program's point of view, so every variable they share must be protected.

How an interrupt is handled on a Cortex-M

- the peripheral sets an interrupt flag (e.g. "falling edge on GP14")
- if enabled, the request goes to the **NVIC** (nested vectored interrupt controller)
  - RP2040: 32 interrupt lines per core, 4 priority levels
  - a higher priority interrupt can interrupt a running ISR (nesting)
- the core pushes registers onto the stack and loads the ISR address from the **vector table**
  - latency on the Cortex-M0+: 15 clock cycles (120 ns at 125 MHz)
- the ISR must **clear the flag**, otherwise it is entered again immediately
- on return the saved registers are restored automatically

Rules for interrupt service routines

- keep ISRs **short**: set a flag, store a value, wake up a task
- no blocking calls (delays, waiting for a UART, `printf`) inside an ISR
- variables shared with the main program
  - C: declare them `volatile`, so the compiler reads them from memory every time
  - `volatile` does **not** make an access atomic: a 64 bit value or a read-modify-write (`count++`) can still be interrupted in the middle
  - protect such accesses with a critical section (interrupts disabled for a short time)
- Rust enforces this: a `static` shared with an ISR must be wrapped, e.g. in a `Mutex` that can only be opened inside a critical section

GPIO interrupt (C, Arduino API)

- try it: in Wokwi one press gave more than 30 interrupts – the bouncing is visible again

```
const int LED = 15, BUTTON = 14;
volatile uint32_t irqCount = 0;   // shared with the ISR -> volatile

void onButton() {                 // interrupt service routine: keep it short
  digitalWrite(LED, !digitalRead(LED));
  irqCount++;
}

void setup() {
  Serial1.begin(115200);
  pinMode(LED, OUTPUT);
  pinMode(BUTTON, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(BUTTON), onButton, FALLING);
}
void loop() {
  static uint32_t last = 0;
  uint32_t now = irqCount;        // 32 bit read is atomic on the Cortex-M0+
  if (now != last) {
    Serial1.printf("interrupts so far: %lu\n", (unsigned long)now);
    last = now;
  }
}
```

GPIO interrupt (Rust)

```
// Hand-over main() -> ISR; the Mutex opens only in a critical section
static SHARED: Mutex<RefCell<Option<(LedPin, ButtonPin)>>> = Mutex::new(RefCell::new(
    None));

    let led = pins.gpio15.into_push_pull_output(); // in main()
    let button = pins.gpio14.into_pull_up_input();
    button.set_interrupt_enabled(EdgeLow, true);
    critical_section::with(|cs| SHARED.borrow(cs).replace(Some((led, button))));
    unsafe { pac::NVIC::unmask(pac::Interrupt::IO_IRQ_BANK0) };
    loop { cortex_m::asm::wfi(); } // sleep until an interrupt arrives

#[interrupt]
fn IO_IRQ_BANK0() {
    static mut PINS: Option<(LedPin, ButtonPin)> = None; // taken from SHARED once
    if PINS.is_none() {
```

```
15          critical_section::with(|cs| *PINS = SHARED.borrow(cs).take());
16      }
17      if let Some((led, button)) = PINS {
18          if button.interrupt_status(EdgeLow) {
19              led.toggle().unwrap();
20              button.clear_interrupt(EdgeLow); // otherwise the IRQ fires again at once
21          }
22      }
23  }
```

Ownership across the interrupt boundary

- the pins are **moved** into the ISR: after the hand-over, `main()` cannot touch them any more
- a data race between main program and ISR is therefore impossible – the compiler rejects such code
- the C version compiles even if both sides write `irqCount` without protection; the error shows up only now and then at run time
- in larger programs a framework handles this hand-over, e.g. RTIC or Embassy (see chapter Operating Systems)

6.4 Timers, PWM and Watchdog

Almost every embedded application needs time: a periodic control loop, a timeout, the frequency of a signal, the brightness of an LED. A timer is basically a counter that is incremented by a known clock. Comparing the counter with a value produces events at defined times (output compare, PWM). Copying the counter at an external edge measures the time of that edge (input capture). A special timer, the watchdog, resets the system if the software stops working.

Building blocks of a timer

- **prescaler / divider**: reduces the input clock, e.g. 125 MHz / 125 = 1 MHz
- **counter**: counts up (or down) with every divided clock
- **top / reload value**: the counter wraps to 0 when it reaches it ⇒ period
- **compare** register: an event (interrupt or pin change) when counter = compare
- **capture** register: the counter value is copied when an external edge arrives
  - the difference of two captures gives period or pulse width with clock accuracy
- RP2040 timer: a 64 bit counter incremented every microsecond, 4 alarms (compare on the lower 32 bits) that raise interrupts
  - `sleep_ms()`, `delay()` and timeouts are built on it
  - 64 bit at 1 MHz overflows after more than 500 000 years

Pulse width modulation (PWM)

- a digital output switched periodically with a variable **duty cycle**
- after averaging (LED + eye, motor + inertia, RC low-pass) the result behaves like an analog value
- applications: LED brightness, DC motor speed, servo position, simple D/A conversion

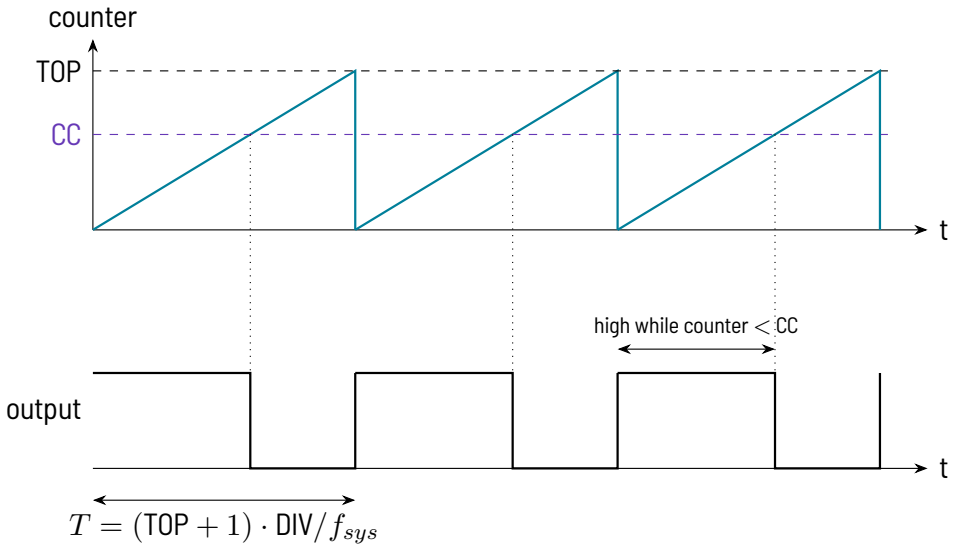


Figure 6.4.1: PWM: the output is high while the counter is below the compare value

- duty cycle $= \text{CC}/(\text{TOP}+1)$, $f_{PWM} = f_{sys}/(\text{DIV} \cdot (\text{TOP}+1))$

PWM on the RP2040

- 8 slices, each with a 16 bit counter, a fractional divider (8.4 bit) and two outputs A and B
- fixed mapping: GPIO $n \rightarrow$ slice $(n/2)$ mod 8, channel A for even, B for odd n
  - GP15 $\rightarrow$ slice 7, channel B
- both channels of a slice share frequency, but have their own compare value (duty cycle)
- phase-correct mode: count up and down – symmetric pulses, half the frequency
- channel B can also be an input: the counter then counts edges or measures the high time of an external signal

PWM example: fading LED (C)

```
1  // f_PWM = f_sys / (DIV * (TOP + 1)); with DIV = f_sys / 1 MHz -> 1 kHz
2  #include "hardware/clocks.h"
3  #include "hardware/pwm.h"
4
5  const uint LED = 15;
6  const uint16_t TOP = 999;
7  uint slice;
8
9  void setup() {
10   gpio_set_function(LED, GPIO_FUNC_PWM);  // FUNCSEL = PWM
11   slice = pwm_gpio_to_slice_num(LED);     // = 7
12   uint32_t div = clock_get_hz(clk_sys) / 1000000;   // e.g. 125 or 200
13   pwm_set_clkdiv_int_frac(slice, div, 0); // counter clock = 1 MHz
14   pwm_set_wrap(slice, TOP);               // counts 0..TOP
15   pwm_set_enabled(slice, true);
16 }
17
18 void loop() {
19   for (int duty = 0; duty <= TOP; duty++) {        // fade in
20     pwm_set_gpio_level(LED, duty);                  // duty = level / (TOP + 1)
```

```
    sleep_ms(1);
  }
  // ... fade out ...
}
```

PWM example: fading LED (Rust)

```rust
const TOP: u16 = 999;

let slices = hal::pwm::Slices::new(pac.PWM, &mut pac.RESETS);
let mut pwm = slices.pwm7;
pwm.set_div_int(125); // counter clock 125 MHz / 125 = 1 MHz
pwm.set_top(TOP); // counts 0..=999 -> period 1 ms
pwm.enable();

let channel = &mut pwm.channel_b;
channel.output_to(pins.gpio15); // FUNCSEL = PWM

loop {
    // Duty cycle = compare value / (TOP + 1)
    for duty in (0..=TOP).chain((0..=TOP).rev()) {
        channel.set_duty_cycle(duty).unwrap();
        delay.delay_us(1000);
    }
}
```

- `set_duty_cycle()` comes from the `SetDutyCycle` trait of `embedded-hal`: drivers written against this trait work on every microcontroller with a HAL
- look at LED_PWM in the logic analyzer: 1 kHz, changing pulse width

Watchdog

- a counter that resets the whole chip when it expires
- the software must restart ("feed", "kick") it regularly
- if the program hangs in an endless loop or a deadlock, the feeding stops and the system restarts in a defined state
- rules
  - feed it at **one** place in the main loop – never from a timer interrupt (that keeps running when the main loop hangs)
  - check the reset reason at start-up and log or report watchdog resets
  - pause it while the debugger halts the CPU
- RP2040: maximum timeout about 8.3 s; register `WATCHDOG_REASON` tells whether the last reset came from the watchdog

Watchdog example (Rust)

```rust
// Read the reset reason before the HAL takes the WATCHDOG peripheral.
let by_watchdog = pac.WATCHDOG.reason().read().timer().bit_is_set();
let mut watchdog = hal::Watchdog::new(pac.WATCHDOG);
// ... clocks, pins, UART ...
```

```
if by_watchdog {
    writeln!(uart, "Restarted by the WATCHDOG\r").unwrap();
} else {
    writeln!(uart, "Power-on reset\r").unwrap();
}

watchdog.pause_on_debug(true); // do not reset while halted in the debugger
watchdog.start(1_000.millis()); // timeout 1 s

loop {
    while button.is_low().unwrap() {
        // simulated "hang": the watchdog is no longer fed
    }
    watchdog.feed();
    led.toggle().unwrap();
    delay.delay_ms(100);
}
```

- hold the button for more than one second and watch the serial monitor – on a real Pico (the Wokwi simulation did not perform the watchdog reset in our tests)
- C version: `watchdog_enable()`, `watchdog_update()`, `watchdog_caused_reboot()`

6.5 Analog Input (ADC)

Sensors often deliver a voltage: a potentiometer, a thermistor in a voltage divider, a microphone amplifier. An analog-to-digital converter (ADC) turns this voltage into a number. Two parameters describe it: the resolution (how many steps) and the sampling rate (how many conversions per second). The datasheet value for the resolution is an upper bound; noise and non-linearity reduce the effective number of bits.

Principle and key figures

- **successive approximation** (SAR): the converter compares the input with the output of an internal D/A converter and determines one bit per step, starting with the MSB – n steps for n bits
- **resolution**: n bit $\Rightarrow 2^n$ steps; one step (LSB) $= V_{ref}/2^n$
  - RP2040: 12 bit, $V_{ref} = 3.3\,\text{V} \Rightarrow 1\,\text{LSB} \approx 0.8\,\text{mV}$
  - effective resolution (ENOB) is lower, about 9 bit
- **sampling rate**: RP2040 up to 500 000 samples/s (96 cycles of the 48 MHz ADC clock)
  - sampling theorem: the signal must not contain frequencies above half the sampling rate, otherwise they appear as wrong low frequencies (aliasing) $\Rightarrow$ analog low-pass filter before the ADC
- one converter, an input **multiplexer** selects the channel
  - channels 0–3 = GP26–GP29, channel 4 = internal temperature sensor

ADC example: potentiometer and chip temperature (C)

```
#include "hardware/adc.h"

void setup() {
  Serial1.begin(115200);           // UART0, 8N1
```

```
  adc_init();
  adc_gpio_init(26);                // disable digital input buffer on GP26
  adc_set_temp_sensor_enabled(true);
}

void loop() {
  adc_select_input(0);              // multiplexer -> channel 0 (GP26)
  uint16_t raw = adc_read();        // 12 bit: 0 .. 4095
  uint32_t millivolt = raw * 3300u / 4095u;

  adc_select_input(4);              // channel 4 = temperature sensor
  float vSense = adc_read() * 3.3f / 4095.0f;
  float celsius = 27.0f - (vSense - 0.706f) / 0.001721f;   // datasheet formula

  Serial1.printf("ADC0 = %4u (%4lu mV)   chip temperature = %.1f C\n",
                 raw, (unsigned long)millivolt, celsius);
  sleep_ms(500);
}
```

- turn the potentiometer in Wokwi and watch the values; Rust version: `adc_uart.rs`
- Wokwi does not simulate the temperature sensor: it reads 0 and the formula gives 437 °C – always check measured values for plausibility

6.6 Serial Interfaces

Sending data bit by bit over a few wires saves pins, cable and board space, so nearly all communication between chips and devices is serial. Three interfaces dominate on the board level. The UART is asynchronous and point-to-point and is used for consoles, GPS receivers and radio modules. I²C is a two-wire bus with addresses, used for slow sensors and configuration. SPI is a fast synchronous interface with a separate select line per device, used for displays, memories and converters. They differ in how the receiver knows when to sample a bit: an agreed baud rate (UART) or a separate clock line (I²C, SPI).

Overview

	UART	I²C	SPI
clock	none (agreed baud rate)	SCL from controller	SCK from controller
wires	TX, RX (+ GND)	SDA, SCL (+ GND)	SCK, COPI, CIPO, CS per device
topology	point-to-point	bus, 7 bit addresses	bus, chip select lines
direction	full duplex	half duplex	full duplex
typical speed	9.6 k – 1 Mbit/s	100 k, 400 k, 1 Mbit/s	1 – 50 Mbit/s
typical use	console, GPS, modems	sensors, EEPROM, RTC	displays, flash, SD card

- "controller" and "peripheral" (or "target") replace the older terms master and slave; for SPI the data lines are called COPI (controller out, peripheral in) and CIPO – older documents use MOSI and MISO

UART: asynchronous serial transmission

- idle line is high; a frame starts with a **start bit** (low)

- then 5–9 data bits, **LSB first**, optional parity bit, 1 or 2 **stop bits** (high)
- both sides must use the same settings, e.g. “115200 8N1” = 115200 bit/s, 8 data bits, no parity, 1 stop bit
- the receiver synchronises on the falling edge of the start bit and samples each bit in its middle (oversampling with 16 times the baud rate)
  - clocks may differ by a few percent before the last bit is sampled wrongly

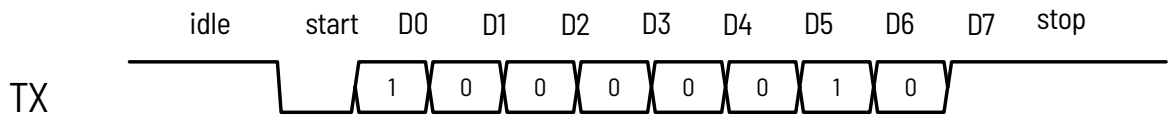


Figure 6.6.1: UART frame 8N1 for the character ‘A’ (0x41 = 0100 0001, sent LSB first)

UART: errors and signal levels

- error detection in the receiver
  - **parity error**: number of 1 bits does not match (detects single bit errors only)
  - **framing error**: stop bit is not high – wrong baud rate or disturbed line
  - **overrun**: new data arrived before the software read the old data
- the UART defines the frame, not the voltages
  - **TTL / CMOS level**: 0 V / 3.3 V directly at the chip pins, a few centimetres to metres
  - **RS-232**: ± 3 to ± 15 V, inverted, point-to-point, up to about 15 m
  - **RS-485**: differential pair, up to 1200 m, several devices on one bus (Modbus, DMX)
  - **USB-to-UART** bridges (CP2102, FT232, CH340): the usual way to a PC today
- RP2040: 2 UARTs (ARM PL011) with 32 byte FIFOs for transmit and receive

UART example: echo in upper case

```c
#include <ctype.h>
#include "hardware/uart.h"

void setup() {
  uart_init(uart0, 115200);                 // 8N1 is the default format
  gpio_set_function(0, GPIO_FUNC_UART);     // GP0 = TX
  gpio_set_function(1, GPIO_FUNC_UART);     // GP1 = RX
  uart_puts(uart0, "UART echo ready\r\n");
}

void loop() {
  if (uart_is_readable(uart0)) {            // RX FIFO not empty?
    char c = uart_getc(uart0);
    uart_putc_raw(uart0, toupper(c));
  }
}
```

```rust
let mut uart = UartPeripheral::new(pac.UART0, uart_pins, &mut pac.RESETS)
    .enable(
        UartConfig::new(115_200.Hz(), DataBits::Eight, None, StopBits::One),
        clocks.peripheral_clock.freq(),
```

```
    )
    .unwrap();
loop {
    // block!() polls the non-blocking read until a byte has arrived
    let byte = block!(uart.read()).unwrap();
    block!(uart.write(byte.to_ascii_uppercase())).unwrap();
}
```

I²C: a two-wire bus

- two open-drain lines with pull-up resistors: SDA (data) and SCL (clock)
- the controller generates the clock and starts every transfer
- each peripheral has a 7 bit address (some 10 bit), e.g. MPU6050 = 0x68
- open drain makes the bus safe: nobody drives high, so two devices can never short the line
- speeds: standard mode 100 kbit/s, fast mode 400 kbit/s, fast mode plus 1 Mbit/s
- typical pull-ups: 4.7 kΩ at 100 kHz, 2.2 kΩ at 400 kHz (the bus capacitance limits the rise time)

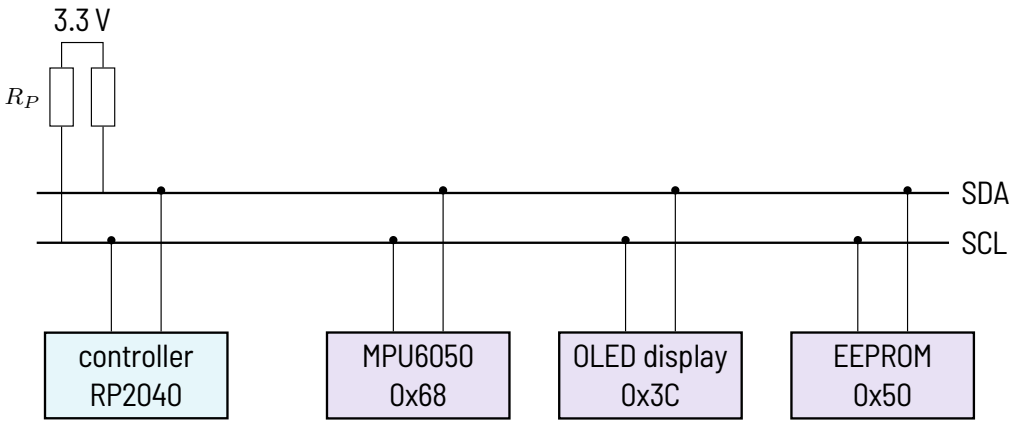


Figure 6.6.2: I²C bus: all devices share SDA and SCL, pull-up resistors R_P define the high level

I²C: protocol

- SDA may only change while SCL is low – with two exceptions
  - **START**: SDA falls while SCL is high
  - **STOP**: SDA rises while SCL is high
- after START: 7 bit address + R/$\overline{W}$ bit (0 = write, 1 = read)
- every byte is followed by an **acknowledge** bit: the receiver pulls SDA low (ACK) or leaves it high (NACK)
- a peripheral may hold SCL low to make the controller wait (**clock stretching**)

Reading a sensor register over I²C

- sensors are organised as a set of registers with an 8 bit register address
- reading a register is a combined transfer
  - START, address + W, **register address**

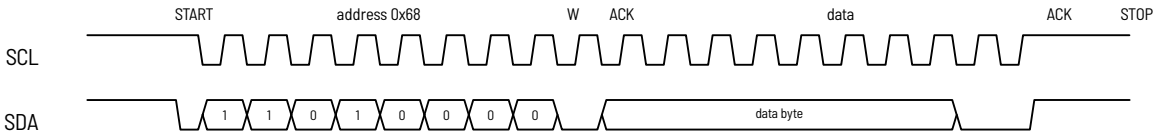


Figure 6.6.3: I²C write: START, address 0x68 with R/$\overline{W}$ = 0, ACK, data byte, ACK, STOP

  - repeated START (no STOP in between, so no other controller can interfere), address + R
  - read one or more bytes; the controller ACKs every byte except the last one (NACK), then STOP
- most sensors increment the register address automatically: one transfer reads a whole block ("burst read")

```
// Register read: write the register address without STOP,
// then read with a repeated START.
void readRegs(uint8_t reg, uint8_t *buf, size_t len) {
  i2c_write_blocking(i2c0, MPU6050, &reg, 1, true);   // true = no STOP
  i2c_read_blocking(i2c0, MPU6050, buf, len, false);
}
```

I²C example: MPU6050 acceleration sensor (Rust)

```
// I2C lines are open drain: the pins need pull-up resistors.
let sda = pins.gpio4.reconfigure::<gpio::FunctionI2C, gpio::PullUp>();
let scl = pins.gpio5.reconfigure::<gpio::FunctionI2C, gpio::PullUp>();
let mut i2c = hal::I2C::i2c0(
    pac.I2C0, sda, scl, 400.kHz(), // fast mode
    &mut pac.RESETS, clocks.system_clock.freq(),
);

// Register read = write the register address, then (repeated start) read.
let mut id = [0u8; 1];
i2c.write_read(MPU6050, &[REG_WHO_AM_I], &mut id).unwrap();
writeln!(uart, "WHO_AM_I = 0x{:02X}\r", id[0]).unwrap();

// Leave sleep mode: write 0 to PWR_MGMT_1.
i2c.write(MPU6050, &[REG_PWR_MGMT_1, 0x00]).unwrap();

loop {
    // Burst read: 6 bytes from ACCEL_XOUT_H (X, Y, Z as big endian i16).
    let mut buf = [0u8; 6];
    i2c.write_read(MPU6050, &[REG_ACCEL_XOUT_H], &mut buf).unwrap();
    // ... convert: +-2 g range -> 16384 LSB per g ...
}
```

- `write_read()` is the repeated-start transfer from the previous slide (trait `embedded_hal::i2c::I2c`)
- change the acceleration in Wokwi (click on the sensor) and compare SDA/SCL in the logic analyzer

SPI: Serial Peripheral Interface

- synchronous, full duplex: with every clock one bit goes out on COPI and one comes in on CIPO

- internally two shift registers that exchange their contents
- each peripheral has its own **chip select** (CS, active low); only the selected one drives CIPO
- push-pull lines and no addressing: much faster than I^2C, but one extra wire per device
- no acknowledge, no standard for frame length or commands – each device defines its own protocol in the datasheet

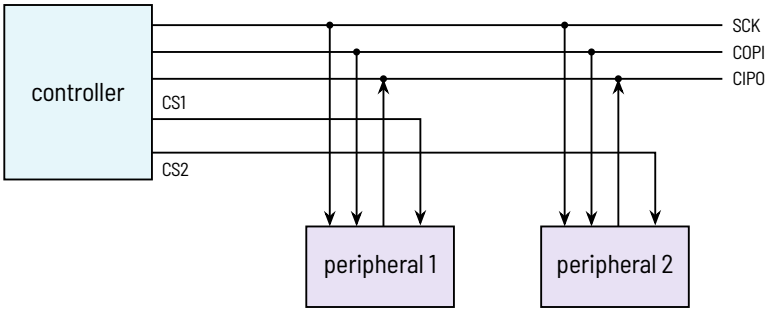


Figure 6.6.4: SPI bus with two peripherals: shared clock and data lines, one chip select each

SPI modes

- two settings define the timing; controller and peripheral must agree
  - **CPOL**: idle level of the clock (0 = low, 1 = high)
  - **CPHA**: sample on the first (0) or second (1) clock edge

mode	CPOL	CPHA	data sampled on
0	0	0	rising edge
1	0	1	falling edge
2	1	0	falling edge
3	1	1	rising edge

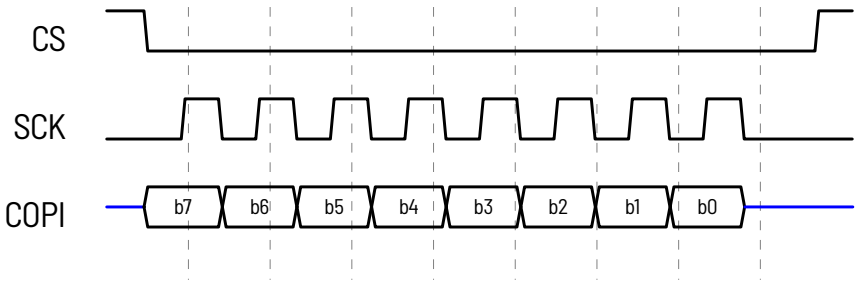


Figure 6.6.5: SPI mode 0: data changes on the falling edge and is sampled on the rising edge (dashed), MSB first

SPI example: 74HC595 shift register

- the 74HC595 is a simple SPI-like device: 8 bit shift register (DS = data, SHCP = clock) plus an output latch (STCP)
- a rising edge on STCP copies the shifted byte to the outputs Q0–Q7 – classic way to get more output pins

```c
#include "hardware/spi.h"

const uint PIN_SCK = 18, PIN_COPI = 19, PIN_LATCH = 17;

void setup() {
  spi_init(spi0, 1000 * 1000);              // 1 MHz
  spi_set_format(spi0, 8, SPI_CPOL_0, SPI_CPHA_0, SPI_MSB_FIRST);   // mode 0
  gpio_set_function(PIN_SCK, GPIO_FUNC_SPI);
  gpio_set_function(PIN_COPI, GPIO_FUNC_SPI);
  gpio_init(PIN_LATCH);
  gpio_set_dir(PIN_LATCH, GPIO_OUT);
}

void show(uint8_t pattern) {
  spi_write_blocking(spi0, &pattern, 1);    // 8 clocks shift the byte in
  gpio_put(PIN_LATCH, 1);                   // rising edge on STCP copies the
  gpio_put(PIN_LATCH, 0);                   // shift register to the outputs
}
```

- Rust version: `spi_shift_register.rs` – `spi.write()` followed by `spi.flush()`, which waits until the last bit has left the FIFO before the latch pulse

Interfaces beyond the board

- **USB**: differential pair, host-controlled, up to 12 Mbit/s on the RP2040 (full speed)
  - device classes avoid special drivers: CDC (virtual serial port), HID (keyboard, mouse), MSC (mass storage)
  - the RP2040 boot ROM appears as a USB drive – copy a `.uf2` file to flash it
- **CAN**: differential two-wire bus for cars and industrial machines
  - multi-controller: all nodes may send; the message identifier decides by bitwise arbitration (a "dominant" 0 wins over a "recessive" 1), so the most important message always gets through without collisions
  - classic CAN 1 Mbit/s, CAN FD up to 8 Mbit/s in the data phase
  - the RP2040 has no CAN controller: use an external one (MCP2515 via SPI) or emulate it with PIO
- **PIO** (RP2040 specific): small state machines that execute their own I/O programs – for protocols that have no hardware block (WS2812 LEDs, VGA, extra UARTs, CAN)

Looking at the signals: the logic analyzer

- the Wokwi logic analyzer records up to 8 digital signals during the simulation
- stop the simulation → the recording is downloaded as a VCD file (value change dump)
- open it in PulseView (sigrok) or GTKWave
  - PulseView has **protocol decoders**: UART, I^2C and SPI are shown as bytes, addresses, ACK/NACK
- this is the same work flow as with a real logic analyzer on real hardware – and the most efficient way to find errors in serial communication

Exercises

- GPIO: change `gpio_blink_registers` so that the LED is switched with GPIO_OUT_SET and GPIO_OUT_CLR. Then try to call `set_high()` on an input pin in Rust and read the compiler message.
- Interrupts: count the interrupts of one button press with bouncing enabled and disabled (attribute `"bounce": "0"` in `diagram.json`). Debounce in the ISR using the timer: ignore edges within 20 ms of the last accepted one.
- PWM: use the potentiometer (ADC) to set the LED brightness. Which PWM frequency is needed so that the LED does not flicker?
- UART: record “A” in the logic analyzer, measure the bit time and compare with 1/115200 s.
- I^2C: decode one register read of the MPU6050 in PulseView and mark START, address, ACK, repeated START and NACK.
- SPI: change the 74HC595 example to SPI mode 3. What happens and why?

7 Networking (TCP/IP)

7.1 Networks in Embedded Systems

More and more embedded systems are connected. A heating controller reports its state to a smartphone app, a machine sends its sensor data to a server for predictive maintenance, a car downloads software updates over the air. In almost all of these cases the protocols of the internet are used: IP, TCP and UDP, and on top of them HTTP, MQTT or similar application protocols.

This chapter explains these protocols layer by layer. We use an ESP32-C3 microcontroller with integrated Wi-Fi, simulated in Wokwi. The simulator provides a Wi-Fi access point and forwards the traffic to the real internet, so our simulated device can talk to real servers. As in the previous chapter every example exists in C and in Rust. The socket examples run on a PC or on Replit.

Why connect an embedded system?

- send measured data to a server or cloud service (monitoring, analysis)
- remote control and configuration (smartphone app, web interface)
- software updates over the air (OTA) – often the most important reason
- time synchronisation, communication between devices (machine to machine)

- but: a microcontroller has little memory and often a battery
  - a TCP/IP stack needs code and RAM: lwIP (used by ESP-IDF and Arduino) about 40 KB code; buffers for every connection come on top
  - Wi-Fi transmission needs well above 100 mA; Bluetooth Low Energy or LoRa are much more economical
  - every connected device can be attacked – security is part of the design

The reference platform: ESP32-C3

- 32 bit RISC-V core, 160 MHz, 400 KB SRAM, 4 MB flash on typical modules
- Wi-Fi 802.11 b/g/n (2.4 GHz) and Bluetooth LE 5 on the chip
- the radio is controlled by a driver with closed-source parts supplied by Espressif
- software stacks
  - C: Arduino core or ESP-IDF, both with lwIP as TCP/IP stack and FreeRTOS
  - Rust: `esp-hal` + `esp-radio` (Wi-Fi driver) + `embassy-net` (TCP/IP stack based on smoltcp)
- in Wokwi the device connects to the open access point **Wokwi-GUEST**; the simulator routes its packets to the internet
  - outgoing connections work (DNS, HTTP, MQTT, NTP); ping (ICMP) does not
  - incoming connections to a server on the device need the paid private gateway or port forwarding in VS Code

Layer models

- communication is split into layers; each layer uses the services of the layer below and offers a service to the layer above
- a layer can be replaced without changing the others (Wi-Fi instead of Ethernet, IPv6 instead of IPv4)
- the **OSI model** (7 layers) is the reference for terminology; the internet uses the simpler **TCP/IP model** (4 layers)

Encapsulation

- on the way down, every layer puts its own **header** in front of the data it gets from above
- the receiver removes the headers layer by layer on the way up
- names of the data units: message (application), segment (TCP) or datagram (UDP), packet (IP), frame (link layer)

- overhead: one byte of payload costs at least $14+20+20+4 = 58$ bytes of headers on Ethernet (plus preamble and gap) – small values should be collected and sent together

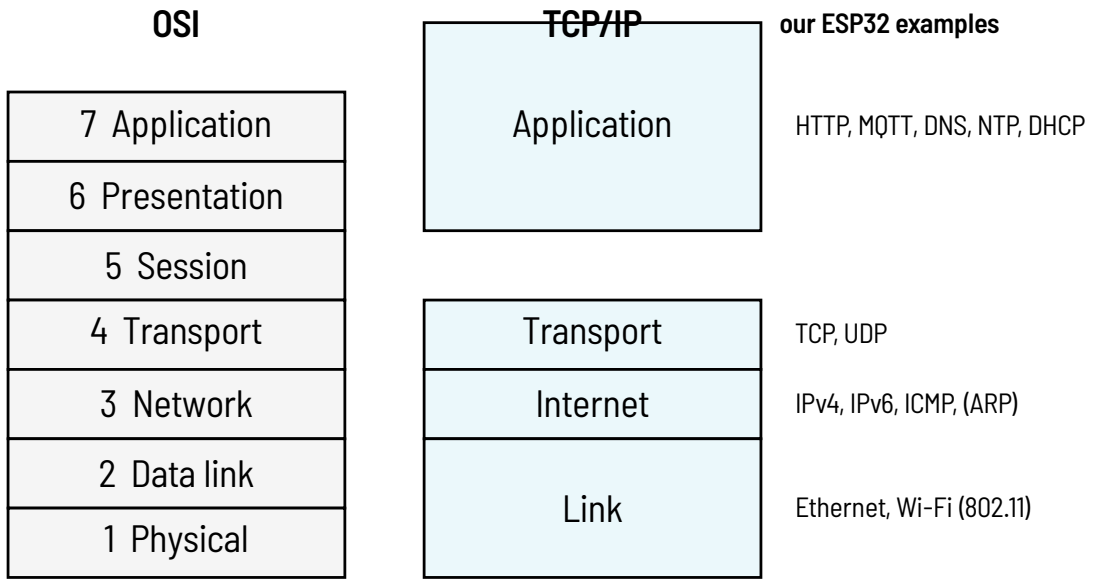


Figure 7.1.1: OSI reference model and the TCP/IP model with the protocols used in this chapter

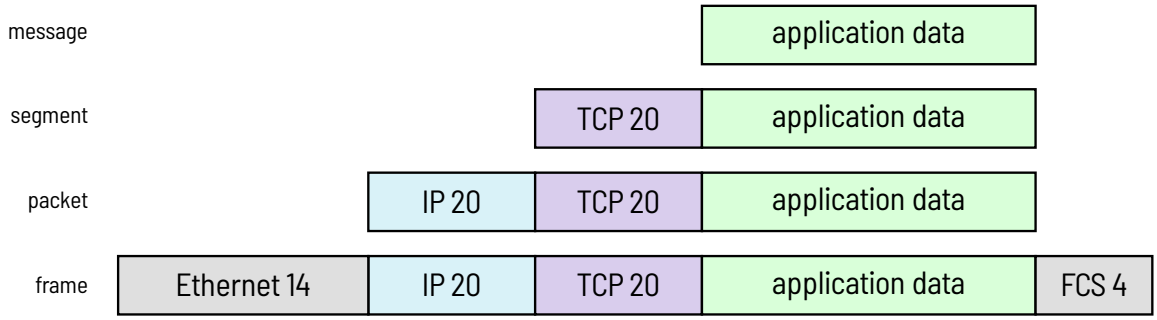


Figure 7.1.2: Encapsulation of application data in TCP, IPv4 and Ethernet (header sizes in bytes, minimum)

7.2 Link Layer: Ethernet and Wi-Fi

The link layer transports frames between devices that are directly connected: over a cable to the next switch, or over radio to the access point. It uses its own addresses, the MAC addresses, which are only meaningful within one local network. Ethernet and Wi-Fi use the same 48 bit address format and look the same to the layers above. This is why an IP stack does not care whether it runs over a cable or over radio.

MAC addresses

- 48 bit, written as six bytes in hexadecimal: `24:0A:C4:00:01:10`
- the first three bytes (OUI) identify the manufacturer – 24:0A:C4 belongs to Espressif
- stored in the chip (eFuse) and unique worldwide; can be overridden in software
- `FF:FF:FF:FF:FF:FF` is the **broadcast** address: every device in the local network receives the frame
- only valid in the local network – a router replaces the link layer header on every hop

Ethernet

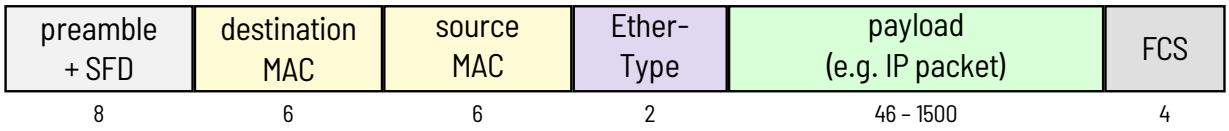


Figure 7.2.1: Ethernet frame (field sizes in bytes)

- **EtherType** tells the receiver what is inside: 0x0800 IPv4, 0x86DD IPv6, 0x0806 ARP
- **FCS**: CRC-32 over the frame; frames with errors are dropped silently – repairing is the job of higher layers (TCP)
- today: twisted pair cables, full duplex, **switches** that learn which MAC address is reachable on which port – no collisions any more
- for embedded systems
  - Power over Ethernet: data and power on one cable (cameras, access points)
  - single pair Ethernet: 10BASE-T1L (up to 1 km, process industry), 10BASE-T1S (several devices on one line, cars)
  - microcontrollers with Ethernet need an external PHY chip (e.g. STM32 + LAN8742)

Wi-Fi (IEEE 802.11)

- **stations** (our ESP32) connect to an **access point** (AP), identified by its name, the SSID
- frequency bands 2.4 GHz (channels 1–13 in Europe, only 3 do not overlap), 5 GHz and 6 GHz with more channels
- connection set-up: scan, authentication, association, then the key exchange of WPA2 or WPA3 – after that, frames are encrypted on the radio link
- radio cannot detect collisions while sending ⇒ **CSMA/CA**: listen before sending, random back-off, every frame is acknowledged by the receiver
- the frames carry three or four MAC addresses (station, AP, final destination); the AP converts them to Ethernet frames for the wired network

- power saving: the station sleeps and wakes up only for selected beacons of the AP; the AP buffers frames meanwhile

Wireless technologies for embedded systems

	data rate	range	power	typical use
Wi-Fi 4/6	10 – 1000 Mbit/s	50 m	high	cameras, gateways, devices on mains power
Bluetooth LE	1 – 2 Mbit/s	10 – 50 m	very low	wearables, smartphone accessories
IEEE 802.15.4 (Thread, Zigbee)	250 kbit/s	30 m, mesh	very low	home automation (Matter)
LoRaWAN	0.3 – 50 kbit/s	2 – 15 km	very low	sensors in agriculture, cities
NB-IoT / LTE-M	0.1 – 1 Mbit/s	mobile network	low	meters, tracking

- Thread is based on IPv6; LoRaWAN and Bluetooth LE use their own protocols and need a gateway to reach the internet

7.3 Network Layer: IP

The Internet Protocol connects networks. Each device has an IP address that consists of a network part and a host part. Within the own network a packet is delivered directly. Everything else is handed to a router, the default gateway, which forwards it towards the destination network. IP itself gives no guarantees: packets can be lost, duplicated or arrive in a different order. Before a device can send its first packet it needs an address, the address of its gateway and of a name server. In most networks it gets all three automatically via DHCP.

IPv4 addresses

- 32 bit, written as four decimal bytes: `10.13.37.2`
- **prefix length** (CIDR notation): `10.13.37.2/24` – the first 24 bits are the network, the remaining 8 bits the host
  - the same as subnet mask 255.255.255.0
  - network address 10.13.37.0, broadcast address 10.13.37.255, 254 usable host addresses
- the old address classes A, B, C (fixed prefix lengths 8, 16, 24) are no longer used
- **private** address ranges, not routed in the internet: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
  - the router translates private addresses to its public address (NAT); devices behind it cannot be reached from outside without extra configuration
- special addresses: 127.0.0.1 (own host, loopback), 0.0.0.0 (“any” / not yet known)

Routing: direct or via the gateway

- before sending, the device compares the destination with its own network
  - same network ⇒ deliver directly to the destination’s MAC address
  - other network ⇒ send the frame to the MAC address of the **default gateway**; the IP header still contains the final destination
- every router repeats this decision with its own routing table

ARP: from IP address to MAC address

- to send a frame in the local network the device needs the MAC address that belongs to an IP address (destination or gateway)

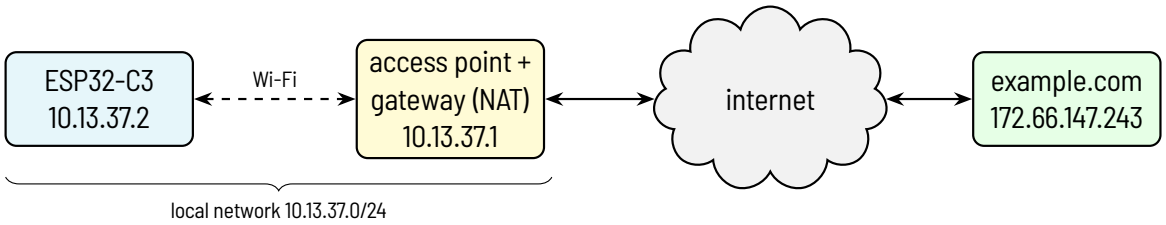


Figure 7.3.1: Our simulated device: everything outside 10.13.37.0/24 goes to the gateway

- **ARP request** as broadcast: "Who has 10.13.37.1? Tell 10.13.37.2"
- the owner answers with an **ARP reply** directly to the asker
- the result is kept for some minutes in the **ARP cache**
- IPv6 uses neighbour discovery (ICMPv6) for the same purpose

DHCP: automatic configuration

- a new device has no address yet ⇒ it sends broadcasts (UDP, ports 68 → 67)
- four messages ("DORA")

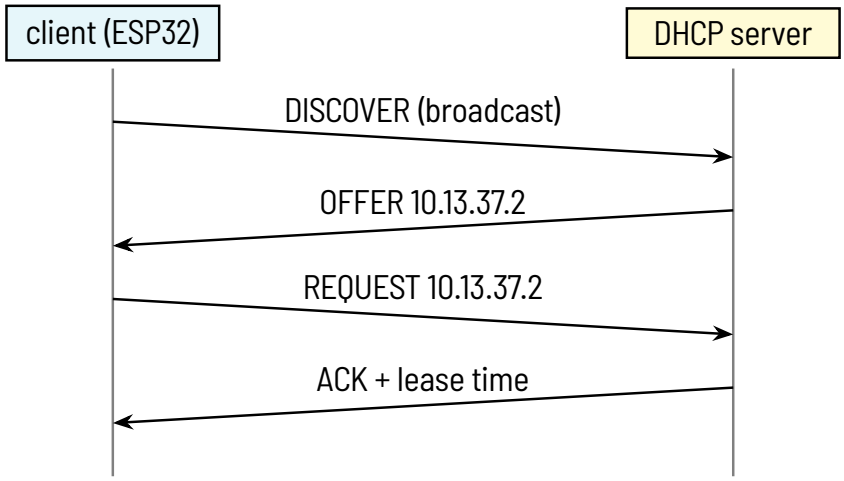


Figure 7.3.2: DHCP: Discover, Offer, Request, Acknowledge

- the ACK contains: IP address and prefix, default gateway, DNS server(s), lease time – the device must renew the lease before it expires

Example: Wi-Fi and DHCP (C, Arduino)

```
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  WiFi.begin("Wokwi-GUEST", "", 6);          // open network, channel 6
  while (WiFi.status() != WL_CONNECTED) delay(250);

  // Layer 2: our MAC address. Layer 3: what the DHCP server assigned.
  Serial.printf("MAC address : %s\n", WiFi.macAddress().c_str());
  Serial.printf("IP address  : %s\n", WiFi.localIP().toString().c_str());
```

```
  Serial.printf("Subnet mask : %s\n", WiFi.subnetMask().toString().c_str());
  Serial.printf("Gateway     : %s\n", WiFi.gatewayIP().toString().c_str());
  Serial.printf("DNS server  : %s\n", WiFi.dnsIP().toString().c_str());
}
```

```
MAC address : 24:0A:C4:00:01:10
IP address  : 10.13.37.2
Subnet mask : 255.255.255.0
Gateway     : 10.13.37.1
DNS server  : 10.13.37.1
```

Example: Wi-Fi and DHCP (Rust, embassy-net)

```
// examples/networking/esp32-rust/src/lib.rs (shortened)
let station = Config::Station(
    StationConfig::default()
        .with_ssid(SSID.try_into().unwrap())
        .with_authentication(AuthenticationMethodConfig::Open)
        .with_channel(6),
);
let controller =
    WifiController::new(wifi, ControllerConfig::default().with_initial_config(station)
    )
        .unwrap();
// The IP stack (embassy-net, based on smoltcp) runs on top of the Wi-Fi interface.
let (stack, runner) = embassy_net::new(
    Interface::station(),
    embassy_net::Config::dhcpv4(Default::default()),
    RESOURCES.init(StackResources::new()),
    seed,
);
spawner.spawn(wifi_task(controller).unwrap()); // keeps the Wi-Fi connected
spawner.spawn(net_task(runner).unwrap());      // processes the packets
stack.wait_config_up().await;                  // until DHCP is done
```

- `async`/`await`: while a task waits (for DHCP, for data) the executor runs other tasks – no threads, no blocking (more in chapter Operating Systems)

The IPv4 header and ICMP

- important fields of the IPv4 header (20 bytes without options)
  - source and destination address
  - **protocol**: what is inside – 6 = TCP, 17 = UDP, 1 = ICMP
  - **TTL** (time to live): decremented by every router, the packet is dropped at 0 – prevents endless loops
  - header checksum, total length, fragmentation fields
- **MTU**: largest packet a link can carry, 1500 bytes on Ethernet and Wi-Fi; larger packets must be fragmented – TCP avoids this by choosing small enough segments
- **ICMP**: error and diagnostic messages of the network layer

- `ping`: echo request / echo reply
- `traceroute`: packets with TTL 1, 2, 3, … – each router on the way answers "time exceeded"
- not available in the Wokwi simulation

IPv6

- 128 bit addresses, written in hexadecimal groups: `2001:db8:0:1::2` (`::` = a run of zero groups)
- enough addresses for every device – no NAT necessary, devices can be reachable directly (a firewall decides what is allowed)
- every interface has a **link-local** address (`fe80::/10`) at once, without any server
- addresses can be configured automatically from the router's announcements (SLAAC), DHCPv6 is optional
- simpler header with fixed length (40 bytes), no fragmentation by routers
- mandatory in new IoT standards: Thread and Matter run over IPv6 only
- IPv4 and IPv6 are not compatible: a device that supports both runs two stacks side by side (dual stack)

DNS: names instead of addresses

- people and programs use names (`example.com`); IP needs addresses
- the device asks its DNS server (address from DHCP) with a small UDP request to port 53
- the answer contains one or more addresses and a lifetime (TTL) for caching
- record types: A (IPv4 address), AAAA (IPv6 address), CNAME (alias), MX (mail server)
- never hard-code server IP addresses in a device: they change, names stay

```
// DNS: name -> IPv4 address (UDP request to the DNS server from DHCP)
let addr = stack.dns_query(HOST, DnsQueryType::A).await.unwrap()[0];
println!("{HOST} = {addr}");           // example.com = 172.66.147.243
```

7.4 Transport Layer: UDP and TCP

IP delivers packets to a device. The transport layer delivers data to a particular program on that device, identified by a port number. The internet offers two transport protocols. UDP adds little more than the port numbers: each datagram is sent on its own and may get lost. TCP builds a reliable, ordered byte stream on top of the unreliable IP. It numbers every byte, waits for acknowledgements and repeats what was lost. This costs time, memory and a connection set-up, but the application does not have to care about lost packets.

Ports

- 16 bit number that selects the program (service) on a host
- well-known ports of servers: 53 DNS, 67/68 DHCP, 80 HTTP, 123 NTP, 443 HTTPS, 1883 MQTT, 8883 MQTT over TLS
- the client usually gets a random ("ephemeral") port from its stack
- a TCP connection is identified by four values: source address, source port, destination address, destination port
  - this is how a web server can talk to thousands of clients on the same port 80

UDP: User Datagram Protocol

- header of only 8 bytes: source port, destination port, length, checksum
- no connection, no acknowledgement, no repetition, no order
- each `send` produces exactly one datagram, each `receive` returns exactly one – message boundaries are kept
- the application must handle loss itself: timeout and repeat, or simply accept it
- used where speed matters more than completeness or where a request fits into one datagram: DNS, NTP, DHCP, audio/video streaming, CoAP (a lightweight protocol for constrained devices), QUIC (the basis of HTTP/3)

UDP example: time from an NTP server (Rust)

```rust
let mut socket = UdpSocket::new(stack, &mut rx_meta, &mut rx_buf, &mut tx_meta, &mut tx_buf);
socket.bind(12345).unwrap(); // local port
loop {
    // 48 byte NTP packet: first byte = LI 0, version 4, mode 3 (client)
    let mut request = [0u8; 48];
    request[0] = 0b00_100_011;
    socket.send_to(&request, (server, 123)).await.unwrap();

    // UDP gives no guarantee: the answer may never arrive -> timeout
    let mut answer = [0u8; 48];
    match with_timeout(Duration::from_secs(3), socket.recv_from(&mut answer)).await {
        Ok(Ok((48, from))) => {
            // Transmit timestamp: seconds since 1900 in bytes 40..44 (big endian)
            let secs = u32::from_be_bytes(answer[40..44].try_into().unwrap()) as u64;
            println!("unix time {}", secs - NTP_TO_UNIX);
        }
        _ => println!("no answer within 3 s (UDP datagram lost?)"),
    }
    Timer::after(Duration::from_secs(10)).await;
}
```

TCP: Transmission Control Protocol

- **connection-oriented**: set-up before data, orderly shutdown afterwards
- **reliable byte stream**
  - every byte has a **sequence number**; the receiver acknowledges (ACK) the next byte it expects
  - missing acknowledgement ⇒ the sender repeats the data after a timeout
  - the receiver puts segments back into the right order and drops duplicates
- **flow control**: the receiver announces how much buffer space it has left (window); the sender never sends more
- **congestion control**: the sender starts slowly and reduces its rate when packets get lost – protects the network
- header of at least 20 bytes: ports, sequence and acknowledgement number, flags (SYN, ACK, FIN, RST, PSH), window, checksum

Connection set-up and shutdown

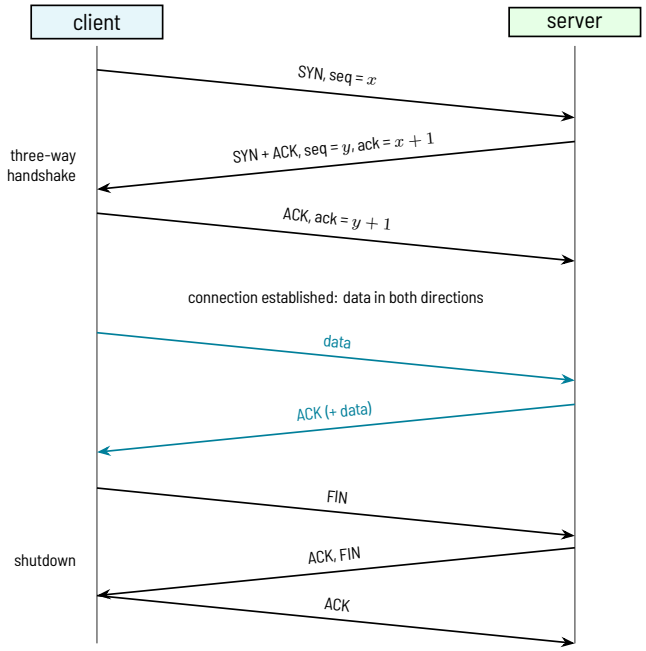


Figure 7.4.1: TCP: three-way handshake, data transfer and shutdown

- both sides choose random initial sequence numbers x and y
- `RST` instead of FIN: abort, e.g. when nobody listens on the port

A byte stream has no message boundaries

- TCP may split one `send()` into several segments, or merge several `send()`s into one segment
- one `recv()` can therefore return half a message or two messages at once
- the application protocol must mark where a message ends
  - a length field in front of the message (MQTT, our own binary protocols)
  - a delimiter such as a line break (HTTP headers, many text protocols)
  - end of the connection (HTTP with `Connection: close`)
- a lesson from the examples: the public MQTT test broker sometimes dropped the connection when a packet arrived split into two TCP segments – correct per standard, but real devices have to live with such servers (our Rust code now sends each packet with one write)

UDP or TCP?

	UDP	**TCP**
connection	none	set-up and shutdown (3 + 4 segments)
reliability	none – loss, duplicates, wrong order possible	complete, in order
data unit	datagram, boundaries kept	byte stream
header	8 bytes	20 – 60 bytes
memory on the device	small	buffers per connection (several KB)
broadcast / multicast	yes	no
typical use	DNS, NTP, DHCP, streaming, CoAP	HTTP, MQTT, file transfer, remote shell

7.5 Socket Programming

Programs use the transport layer through sockets. The socket API comes from BSD Unix (1983) and is available almost unchanged on Linux, macOS, Windows and in lwIP on microcontrollers. Higher-level libraries such as the Rust standard library, Arduino's `WiFiClient` or embassy-net wrap the same concepts. The examples in this section run on a PC or on Replit. They show the server side, which our simulated ESP32 cannot offer to the outside world.

The socket API

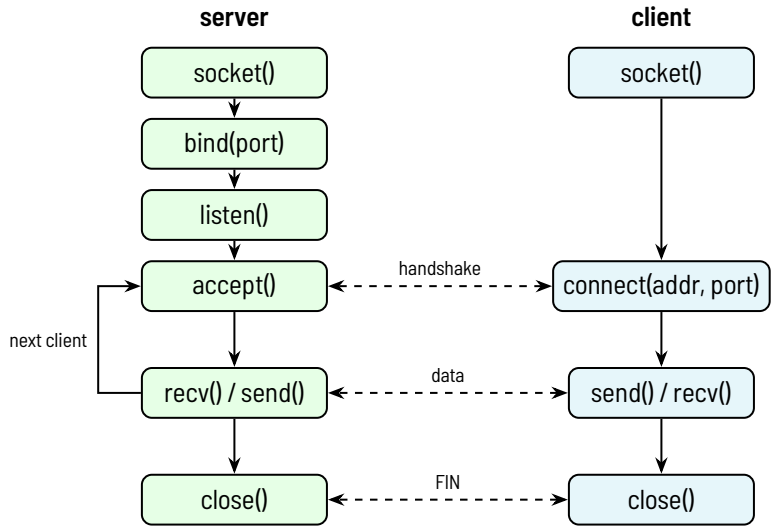


Figure 7.5.1: Order of the socket calls for a TCP server and a TCP client

- `accept()` returns a **new** socket for each connection; the listening socket stays open for further clients

TCP echo server (C, BSD sockets)

- `htons()`/`htonl()`: the network uses big endian, x86 and ARM (usually) little endian

```c
// 1. socket(): an endpoint for IPv4 (AF_INET) and TCP (SOCK_STREAM)
int server = socket(AF_INET, SOCK_STREAM, 0);

// 2. bind(): local address and port (INADDR_ANY = all interfaces)
struct sockaddr_in addr = {0};
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = htonl(INADDR_ANY);
addr.sin_port = htons(port);              // network byte order = big endian
bind(server, (struct sockaddr *)&addr, sizeof addr);
// 3. listen(): accept connection requests, queue up to 5
listen(server, 5);
for (;;) {
  // 4. accept(): wait for a client; returns a NEW socket for this connection
  int client = accept(server, (struct sockaddr *)&client_addr, &len);
  // 5. recv()/send(): TCP is a byte stream
  while ((n = recv(client, buf, sizeof buf, 0)) > 0) {
    send(client, buf, n, 0);
  }
  close(client);                          // n == 0: the client has closed its side
```

```
}
```

TCP echo server (Rust standard library)

```rust
fn handle(mut stream: TcpStream) -> std::io::Result<()> {
    let mut buf = [0u8; 256];
    loop {
        let n = stream.read(&mut buf)?; // TCP is a byte stream, not messages
        if n == 0 { break; }            // the client has closed its side (FIN)
        stream.write_all(&buf[..n])?;
    }
    Ok(())
}

fn main() -> std::io::Result<()> {
    // bind() + listen() in one step
    let listener = TcpListener::bind(format!("0.0.0.0:{port}"))?;
    for stream in listener.incoming() {
        // accept(): one new stream per connection
        let stream = stream?;
        thread::spawn(move || handle(stream));   // one thread per client
    }
    Ok(())
}
```

- errors are values (`Result`) – `?` passes them on; the C version has to check every return value by hand

The same on the microcontroller

- C (Arduino): `WiFiClient` wraps a lwIP socket – `connect()`, `print()`, `read()`, `stop()`
- Rust (embassy-net): `TcpSocket` with buffers provided by the caller – no heap needed
  - `connect().await`, `write_all().await`, `read().await`
  - while a socket waits, the executor runs other tasks
- the buffer sizes decide how much RAM a connection costs and how fast it can be (window size)

```rust
// TCP: the socket needs buffers for received and not yet acknowledged data
let mut rx_buf = [0u8; 2048];
let mut tx_buf = [0u8; 512];
let mut socket = TcpSocket::new(stack, &mut rx_buf, &mut tx_buf);
socket.set_timeout(Some(Duration::from_secs(10)));
socket.connect((addr, 80)).await.unwrap(); // three-way handshake
```

7.6 Application Protocols: HTTP and MQTT

On top of TCP the application protocols define what the bytes mean. HTTP is the protocol of the web. A client sends a request, the server answers, and every request stands on its own. Embedded devices use it to call

web services (REST APIs) or to offer a small web interface. MQTT was designed for devices that send small messages over unreliable links. Instead of talking to each other directly, all devices connect to a broker that forwards messages by topic. A sensor does not need to know who reads its values.

HTTP: request and response

- text protocol: request line, header lines, empty line, optional body
- methods: GET (read), POST (create / send data), PUT (replace), DELETE
- status codes: 200 OK, 301 moved, 404 not found, 500 server error
- **REST** APIs: resources addressed by URLs, data usually as JSON
- HTTPS = HTTP over TLS (port 443) – the normal case today

```
GET / HTTP/1.1                      <- request line: method, path, version
Host: example.com                   <- header lines
Connection: close
                                    <- empty line: end of the header
HTTP/1.1 200 OK                     <- status line of the response
Content-Type: text/html; charset=utf-8
Transfer-Encoding: chunked          <- body comes in pieces with length prefix
...
2c9                                 <- next chunk: 0x2c9 = 713 bytes
<!doctype html><html lang=en> ...
0                                   <- chunk of length 0: end of the body
```

HTTP by hand (Rust, raw TCP)

```rust
// DNS: name -> IPv4 address (UDP request to the DNS server from DHCP)
let addr = stack.dns_query(HOST, DnsQueryType::A).await.unwrap()[0];

let mut socket = TcpSocket::new(stack, &mut rx_buf, &mut tx_buf);
socket.connect((addr, 80)).await.unwrap(); // three-way handshake

// HTTP/1.1 request: request line, header lines, empty line
let request = "GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n";
socket.write_all(request.as_bytes()).await.unwrap();

// Read until the server closes the connection (read returns 0).
let mut buf = [0u8; 512];
loop {
    match socket.read(&mut buf).await {
        Ok(0) | Err(_) => break,
        Ok(n) => print!("{}", core::str::from_utf8(&buf[..n]).unwrap_or("<binary>")),
    }
}
```

- every layer of this chapter appears in these lines: Wi-Fi, DHCP, DNS, TCP, HTTP
- in practice an HTTP client library (C: `HTTPClient`, Rust: `reqwless`) handles chunking, redirects and TLS

MQTT: publish and subscribe

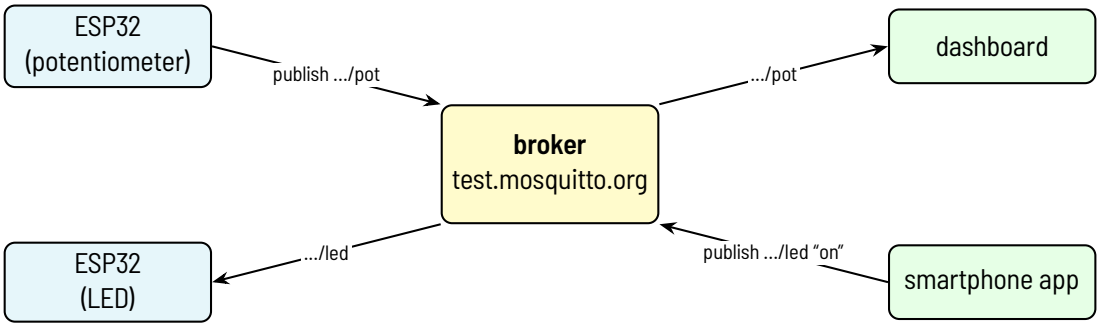


Figure 7.6.1: MQTT: publishers and subscribers only know the broker and the topics

- every client opens **one** TCP connection to the broker (port 1883, with TLS 8883) – also devices behind NAT can receive messages
- **topics** are hierarchical strings: `rwu-ec/device42/pot`
- wildcards for subscriptions: + one level (`rwu-ec/+/pot`), # all levels below (`rwu-ec/#`)

MQTT features

- **quality of service** per message
  - QoS 0 "at most once": send and forget
  - QoS 1 "at least once": repeated until acknowledged – duplicates possible
  - QoS 2 "exactly once": four-way handshake, most overhead
- **retained** message: the broker keeps the last message of a topic and sends it to every new subscriber at once (e.g. the current set point)
- **last will**: a message the broker publishes when a client disappears without saying goodbye ("device42/status = offline")
- **keep alive**: the client sends at least one packet per interval (PINGREQ if nothing else); otherwise the broker assumes the connection is dead
- MQTT 3.1.1 is the most widely supported version; MQTT 5 adds error codes, message expiry, user properties

The MQTT CONNECT packet byte by byte

- recorded from our Rust example (`src/mqtt.rs` builds it by hand)

bytes (hex)	field	meaning
10	fixed header	packet type 1 = CONNECT, flags 0
17	remaining length	23 bytes follow (variable length: 7 bits per byte)
00 04 4d 51 54 54	protocol name	length 4, "MQTT"
04	protocol level	4 = MQTT 3.1.1
02	connect flags	clean session
00 3c	keep alive	60 seconds
00 0b 72 77 75 2d …	client identifier	length 11, "rwu-ec-demo"

- the broker answers with CONNACK `20 02 00 00`: type 2, length 2, no session present, return code 0 = accepted

- every MQTT packet has this structure: 1 byte type and flags, length, then the content – the length solves the "no message boundaries" problem of TCP

MQTT example: sensor value and LED (C, PubSubClient)

```
WiFiClient tcp;
PubSubClient mqtt(tcp);
// Called by mqtt.loop() for every received PUBLISH packet.
void onMessage(char *topic, byte *payload, unsigned int len) {
  String msg((const char *)payload, len);
  if (msg == "on") digitalWrite(LED, HIGH);
  if (msg == "off") digitalWrite(LED, LOW);
}
void loop() {
  if (!mqtt.connected()) {                    // (re)connect: CONNECT + SUBSCRIBE
    if (mqtt.connect(DEVICE)) mqtt.subscribe(TOPIC_LED);
    else { delay(2000); return; }
  }
  mqtt.loop();                                // receive packets, keep-alive

  static uint32_t last = 0;
  if (millis() - last >= 2000) {
    last = millis();
    char text[8];
    snprintf(text, sizeof text, "%d", analogRead(POT));    // 12 bit: 0..4095
    mqtt.publish(TOPIC_POT, text);
  }
}
```

MQTT example: sensor value and LED (Rust, async)

```
socket.connect((broker, 1883)).await.unwrap(); // MQTT without TLS
mqtt::connect(&mut socket, DEVICE).await.unwrap();
mqtt::subscribe(&mut socket, TOPIC_LED).await.unwrap();

let (mut next_publish, mut packet) = (Instant::now(), [0u8; 256]);
loop {
    match select(mqtt::receive(&mut socket, &mut packet), Timer::at(next_publish)).
    await {
        Either::First(Ok((header, len))) => {
            if mqtt::packet_type(header) == 3 {            // PUBLISH
                if let Some((_topic, payload)) = mqtt::parse_publish(&packet[..len]) {
                    if payload == b"on" { led.set_high() } else if payload == b"off" {
     led.set_low() }
                }
            }
        }
        Either::Second(()) => {
            let raw: u16 = nb::block!(adc.read_oneshot(&mut pot)).unwrap();
            mqtt::publish(&mut socket, TOPIC_POT, text.as_bytes()).await.unwrap();
            next_publish += Duration::from_secs(2);
        }
    }
}
```

HTTP or MQTT?

	HTTP (REST)	MQTT
pattern	request / response, client asks	publish / subscribe via broker
connection	usually one per request	one permanent connection
server → device	difficult (device must poll, or WebSocket)	simple: device subscribes
overhead per message	hundreds of bytes of headers	a few bytes
infrastructure	any web server	broker needed
typical use	web services, configuration pages, OTA downloads	telemetry, commands, many devices

7.7 Security and Analysis

Our examples send everything in clear text to a public server. That is fine for learning, but not for a product. Anybody on the path can read the messages, change them or pretend to be the server. In the public test broker, anybody can even publish to our LED topic. This section summarises what a connected product needs, and shows how to look at the traffic with Wireshark.

Securing connected devices

- **TLS** (transport layer security) between TCP and the application
  - encryption: nobody on the way can read the data
  - integrity: changes are detected
  - authentication: the server proves its identity with a certificate; the device checks it against a stored root certificate
  - costs on a microcontroller: several tens of KB of RAM, handshake of up to a few seconds
- authentication of the device: user name/password or a client certificate per device
- never put credentials into the source code or the repository; one secret for all devices means all are broken at once
- no open ports that are not needed; no default passwords
- signed software updates – and a way to install them over the network
- EU Cyber Resilience Act: products with digital elements must meet security requirements (vulnerability reporting from 09/2026, full obligations from 12/2027)

Looking at the traffic: Wireshark

- Wokwi can record the network traffic of the simulated device as a `.pcap` file – open it in Wireshark
- the same tool is used for real networks (capture on the PC, or with a mirror port on a switch)
- useful display filters
  - `dhcp`, `dns`, `arp` – the start-up of the device
  - `tcp.flags.syn == 1` – connection set-ups
  - `http`, `mqtt` – decoded application protocols
  - `ip.addr == 10.13.37.2` – everything of our device
- "Follow TCP stream" shows the byte stream of a connection as text – the MQTT client id and the LED commands are plainly visible, which is exactly what TLS prevents

Exercises

- Run `wifi_dhcp`. Which part of the IP address is the network part? How many devices fit into this network?
- Record `http_get` as pcap. Find DHCP (four messages), the DNS query, the three-way handshake, the request and the FIN segments. How many bytes of headers were sent for the request?
- Change `udp_ntp` so that it prints the date as well. Why must the code care about the year 2036?
- MQTT: choose your own device name, switch the LED from MQTT Explorer and publish the potentiometer value only when it changes by more than 50.
- Run the echo server and the client on your PC (or in two Replit shells), also mixing C and Rust. Then ask a fellow student to connect to your server from another network. Why does this fail without port forwarding on your router?
- Extend the Rust MQTT client by PINGREQ so that the connection survives when nothing is published for more than 60 seconds.

8 Operating Systems

8.1 Why an Operating System?

Small embedded programs often consist of one endless loop that polls inputs and calls functions in turn, plus some interrupt service routines. This works well as long as all jobs are short and have similar timing requirements. It becomes hard when one job takes long (a network transfer, a display update) while another must react within milliseconds. A real-time operating system (RTOS) solves this by splitting the program into tasks. Each task is written as if it had the processor to itself, and a scheduler decides which task runs. The scheduler always picks the most important ready task, so the reaction time of high priority tasks no longer depends on the others.

In this chapter we look at the concepts behind an RTOS and at two ways to apply them on the ESP32-C3: FreeRTOS in C, which the Arduino core and ESP-IDF are built on, and the async framework Embassy in Rust. At the end we compare other systems, up to embedded Linux on larger processors.

Program structures without an operating system

- **super loop**: `for(;;) { read(); control(); show(); }`
  - simple, deterministic, no extra memory
  - the reaction time is the duration of the whole loop – one slow function delays everything
  - waiting ("`delay(500)`") blocks all other jobs
- **foreground / background**: interrupts for urgent events (foreground), super loop for the rest (background)
  - fast reaction to events
  - the ISRs must stay short; shared variables need protection; long work still ends up in the loop
- both are fine for small systems – many products never need more

What an RTOS adds

- **tasks** (threads): independent functions, each with its own stack and priority
- **scheduler**: decides which ready task runs; preempts a task when a more important one becomes ready
- **time services**: delays, periodic execution, timeouts
- **communication and synchronisation**: queues, semaphores, mutexes, event flags
- **memory management**: task stacks, heap for dynamic objects
- costs: a few KB of code (FreeRTOS kernel about 6–12 KB), RAM for every task stack, some microseconds per task switch, and new kinds of errors (race conditions, deadlocks)

Real time

- real time means **predictable**, not "fast": the result must be there before its **deadline**
  - **hard** real time: a missed deadline is a failure (airbag, motor control)
  - **soft** real time: a missed deadline reduces the quality (audio dropout, slow display)
- important figures
  - **latency**: time from an event to the start of the reaction
  - **jitter**: variation of this time or of a period
  - **WCET** (worst case execution time): the longest time a piece of code can take – the average is not enough
- an RTOS makes timing analysable: the highest priority ready task always runs, kernel operations have bounded duration
- general purpose systems (Windows, standard Linux) optimise throughput and fairness instead

8.2 Tasks and Scheduling

A task is a function that usually never returns. The kernel keeps a task control block (TCB) for every task, with its state, its priority and the saved stack pointer. When the scheduler switches from one task to another, it saves the registers of the running task on its stack and restores those of the next task. Each task therefore has its own stack, which is the main memory cost of an RTOS. A task that waits for something, such as a time, a message or a lock, is blocked and uses no processor time at all.

Task states

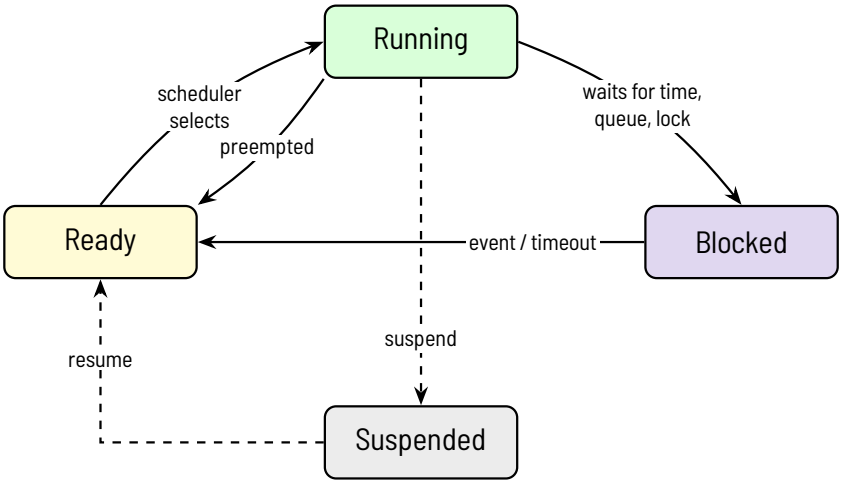


Figure 8.2.1: Task states in FreeRTOS

- on a single core exactly one task is Running; the idle task (lowest priority) runs when nothing else is ready

Preemptive priority scheduling

- every task has a fixed priority (FreeRTOS: higher number = more important)
- the scheduler always runs the ready task with the highest priority
- when a more important task becomes ready (timeout, message, interrupt), the running task is interrupted at once – **preemption**
- tasks with the same priority share the CPU in turns (time slicing at every tick)
- the **tick** interrupt (1 ms on the ESP32) drives delays and time slicing
- the context switch itself is done in a special low-priority interrupt (on Cortex-M: PendSV), so it never interrupts another ISR

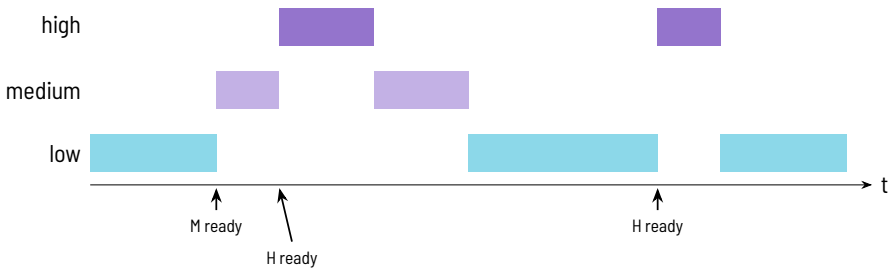


Figure 8.2.2: H preempts M, M preempts L; a task continues where it was interrupted

Choosing priorities

- priority follows urgency (deadline), not "importance" of the function
- **rate monotonic** assignment for periodic tasks: shorter period ⇒ higher priority
  - with n independent periodic tasks all deadlines are met if the CPU utilisation $U = \sum C_i/T_i \leq n(2^{1/n} - 1)$ (about 69 % for large n; C_i = execution time, T_i = period)
- long computations get a low priority; reactions to events a high one

- a task that never blocks starves all tasks with lower priority – every task must wait somewhere
- ISRs are always more urgent than any task: keep them short and hand the work to a task

FreeRTOS: creating tasks

```
void blinkTask(void *param) {
  pinMode(LED, OUTPUT);
  for (;;) {
    digitalWrite(LED, !digitalRead(LED));
    vTaskDelay(pdMS_TO_TICKS(500));           // blocked for 500 ms
  }
}
void heartbeatTask(void *param) {
  TickType_t lastWake = xTaskGetTickCount();
  for (uint32_t n = 0;; n++) {
    Serial.printf("heartbeat %lu at %lu ms\n", (unsigned long)n, (unsigned long)millis
    ());
    xTaskDelayUntil(&lastWake, pdMS_TO_TICKS(1000));    // fixed period, no drift
  }
}
void setup() {
  Serial.begin(115200);
  // function, name, stack size (ESP-IDF: in BYTES), parameter, priority, handle
  xTaskCreate(blinkTask, "blink", 2048, NULL, 2, NULL);
  xTaskCreate(heartbeatTask, "heartbeat", 4096, NULL, 1, NULL);
}
```

- `vTaskDelay`: relative, the period drifts; `xTaskDelayUntil`: absolute (Wokwi: 1178, 2178, 3178 ms)

Embassy: tasks as async functions (Rust)

```
/// A task is an async function. It runs until it awaits something and
/// then gives the CPU to the next ready task.
#[embassy_executor::task]
async fn blink(mut led: Output<'static>) {
    let mut ticker = Ticker::every(Duration::from_millis(500));
    loop {
        led.toggle();
        ticker.next().await; // fixed period, does not drift
    }
}

#[esp_rtos::main]
async fn main(spawner: Spawner) -> ! {
    // ... init ...
    let led = Output::new(peripherals.GPIO4, Level::Low, OutputConfig::default());
    spawner.spawn(blink(led).unwrap());
    spawner.spawn(heartbeat().unwrap());
    loop {
        embassy_time::Timer::after_secs(3600).await;
    }
}
```

- the LED is **moved** into the task: no other part of the program can use it any more

How Embassy differs from FreeRTOS

- the compiler turns every `async fn` into a state machine; `.await` = "save the state, return to the executor"
- **no stack per task**: the state machine contains only the variables that live across an `.await`, and its size is known at compile time ⇒ all memory is static, no stack overflows of tasks
- **cooperative** within one executor: a task runs until its next `.await` – a long computation without `.await` blocks all other tasks
- priorities: several executors, e.g. an `InterruptExecutor` that runs inside an interrupt with higher priority
- when no task is ready, the executor puts the CPU to sleep (`wfi`) – good for battery devices
- hardware events wake tasks directly: the interrupt handler calls the task's **waker**

8.3 Communication and Synchronisation

Tasks rarely work alone. They pass data to each other, wait for events from interrupts and share resources such as a bus, a display or a global variable. The kernel offers objects for this: queues to pass data, semaphores to signal events, and mutexes to protect shared resources. Used carelessly they lead to the classic errors of concurrent programs: race conditions, deadlocks and priority inversion. All three can be shown in a few lines of code, and our examples do exactly that.

Queues: passing data between tasks

- a FIFO buffer with a fixed number of elements; the data is **copied** into the queue
- sending to a full queue and receiving from an empty queue **block** the task (with an optional timeout)
- the receiver needs no polling: it sleeps until data arrives
- decouples producer and consumer: different speeds, different priorities

```
QueueHandle_t samples = xQueueCreate(8, sizeof(uint16_t));   // 8 elements

void producerTask(void *param) {                  // every 100 ms
  for (;;) {
    uint16_t raw = analogRead(POT);
    xQueueSend(samples, &raw, portMAX_DELAY);   // waits if the queue is full
    xTaskDelayUntil(&lastWake, pdMS_TO_TICKS(100));
  }
}
void consumerTask(void *param) {                  // average of 10 values
  for (;;) {
    uint32_t sum = 0;
    for (int i = 0; i < 10; i++) {
      uint16_t value;
      xQueueReceive(samples, &value, portMAX_DELAY);   // waits for data
      sum += value;
    }
    Serial.printf("average of 10 samples: %lu\n", (unsigned long)(sum / 10));
  }
}
```

Queues in Embassy: channels

```rust
/// Queue for 8 values. A full queue makes send() wait (back pressure),
/// an empty queue makes receive() wait.
static SAMPLES: Channel<CriticalSectionRawMutex, u16, 8> = Channel::new();

#[embassy_executor::task]
async fn consumer() {
    loop {
        let mut sum: u32 = 0;
        for _ in 0..10 {
            sum += SAMPLES.receive().await as u32; // waits for the next value
        }
        println!("average of 10 samples: {}", sum / 10);
    }
}

// in main (producer):
let mut ticker = Ticker::every(Duration::from_millis(100));
loop {
    let raw: u16 = nb::block!(adc.read_oneshot(&mut pot)).unwrap();
    SAMPLES.send(raw).await; // producer
    ticker.next().await;
}
```

From interrupt to task

- **deferred interrupt handling**: the ISR only acknowledges the hardware and signals a task; the task does the real work
  - the ISR stays short ⇒ small latency for all other interrupts
  - the work runs with a task priority and may block, print, use the network
- **semaphore**: a counter; `give` increments, `take` decrements or blocks at 0
  - binary semaphore (0/1) to signal an event; counting semaphore to count events or free resources
- in an ISR only the `...FromISR` functions are allowed: they never block, and they report whether a more important task was woken up
- FreeRTOS also offers direct **task notifications**: faster and smaller than a semaphore when exactly one task waits

Interrupt to task: C and Rust

```c
void IRAM_ATTR onButton() {           // ISR: as short as possible
  BaseType_t woken = pdFALSE;
  xSemaphoreGiveFromISR(buttonSem, &woken);   // never the normal API in an ISR
  portYIELD_FROM_ISR(woken);          // switch at once if a higher task woke up
}
void buttonTask(void *param) {
  for (;;) {
    xSemaphoreTake(buttonSem, portMAX_DELAY);   // sleep until the ISR gives
    // ... toggle LED, print, debounce ...
  }
}
```

```
loop {
    button.wait_for_falling_edge().await; // interrupt wakes the task
    presses += 1;
    led.toggle();
    Timer::after_millis(50).await; // debounce: ignore the bouncing
    button.wait_for_high().await; // wait until released
}
```

- in Rust the interrupt, the flag and the waiting are hidden behind one `.await`; tested in Wokwi with bouncing buttons: two presses → exactly two events

Race conditions

- two tasks change the same variable with read – modify – write
- a task switch between read and write ⇒ the other task's update is overwritten ("lost update")
- the error appears rarely and randomly – very hard to find by testing

```
for (int i = 0; i < 1000; i++) {      // in two tasks of equal priority
  if (useMutex) xSemaphoreTake(lock, portMAX_DELAY);
  uint32_t value = counter;           // read
  taskYIELD();                        // provoke a task switch right here
  counter = value + 1;                // modify-write
  if (useMutex) xSemaphoreGive(lock);
}
```

```
without mutex: counter = 1001 (expected 2000)
with    mutex: counter = 2000 (expected 2000)
```

- the `taskYIELD()` makes the error certain; in real code an interrupt or the tick causes it from time to time

Mutual exclusion

- the code between read and write is a **critical section**: only one task at a time may execute it
- ways to protect it
  - disable interrupts / the scheduler: very short sections only (a few instructions), affects the whole system
  - **mutex** (mutual exclusion): a lock that only the task that took it may release; other tasks block while it is taken
- in Rust, data shared between tasks must be wrapped in a type that guarantees exclusive access; writing to a plain global variable requires `unsafe`

```
static COUNTER: Mutex<CriticalSectionRawMutex, u32> = Mutex::new(0);

for _ in 0..1000 {
    let mut counter = COUNTER.lock().await; // wait until the mutex is free
    let value = *counter; // read
    Timer::after(Duration::from_micros(10)).await; // other task may run now
```

```
    *counter = value + 1; // modify-write
} // guard dropped here -> mutex released
```

- the data is only reachable through the lock, and the lock is released automatically – forgetting it is impossible

Deadlock

- two tasks wait for each other forever: A holds lock 1 and waits for lock 2, B holds lock 2 and waits for lock 1
- possible only if all four conditions hold: mutual exclusion, hold and wait, no preemption of locks, circular wait
- prevention in practice
  - always take several locks in the same, fixed order
  - hold locks as short as possible; never block (queue, delay) while holding a lock
  - use timeouts instead of waiting forever, and report the timeout
  - better: avoid sharing – let one task own the resource and send it requests through a queue

Priority inversion

- a low priority task L holds a lock that the high priority task H needs ⇒ H must wait for L (unavoidable, but short)
- a medium priority task M, which does not need the lock, preempts L ⇒ H effectively waits for M: the priorities are inverted
- famous case: Mars Pathfinder (1997) – repeated resets by the watchdog until priority inheritance was switched on remotely
- solution: **priority inheritance** – while L holds a mutex that H waits for, L runs with H's priority, so M cannot preempt it
  - FreeRTOS mutexes do this; binary semaphores do not

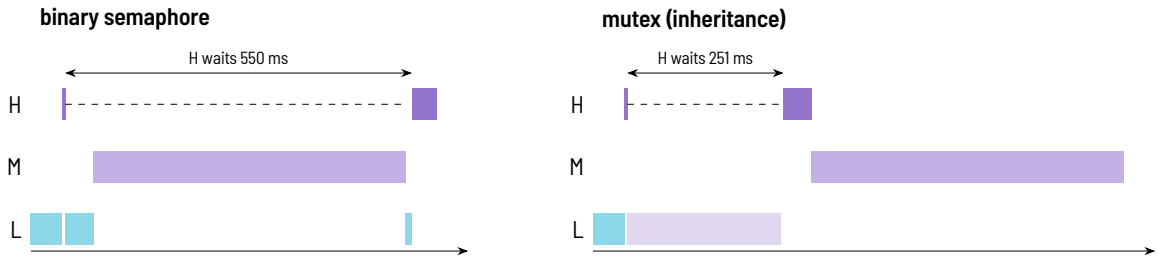


Figure 8.3.1: Measured in Wokwi (`priority_inversion.ino`): L holds the lock 300 ms, M runs 500 ms

Memory in an RTOS

- every task needs its own stack – too small is the most common RTOS error
  - the stack overflows silently into other data; use the overflow check of the kernel (`configCHECK_FOR_ST` and measure the high water mark (`uxTaskGetStackHighWaterMark`)
  - `printf` and floating point need much stack – on the ESP32 4 KB are a safe start for such tasks

- kernel objects (tasks, queues, semaphores) can be allocated
  - dynamically from the RTOS heap: simple, but the heap can fragment and allocation can fail at run time
  - statically (`xTaskCreateStatic`, …): all memory is known at link time – preferred in safety-related systems
- FreeRTOS offers several heap implementations: from "allocate only, never free" (heap_1) to a general allocator that merges free blocks (heap_4)
- Embassy: everything static; the size of every task's state is checked by the compiler

8.4 Case Study: a Controller and its Simulated Environment

An embedded program never runs on its own. It reads sensors, drives actuators, and the physical process it controls answers by changing what the sensors measure. Whether the program is correct can therefore only be decided together with its environment: a temperature controller is right if the temperature is right, and a fault handler is right if the device reaches a safe state when the fault really happens. This section puts the pieces of this and the previous chapters together in one Wokwi project. An ESP32-C3 with FreeRTOS controls a small climate chamber. The chamber and its fan do not exist as hardware: they are two *custom chips*, small C programs that the simulator runs next to the processor and that model the physics. A logic analyzer records the signals between program and environment, and a test scenario switches faults on and off. The example is in `examples/os/chamber`, in C with FreeRTOS and in Rust with Embassy.

The embedded paradigm in the simulator

- recall the definition: an embedded system is part of a larger technical system and interacts with it through **sensors** and **actuators**
- the program closes a **loop**: it changes the process, the process changes the measurements, the measurements change the program's next decision
- a test that only feeds fixed values into the program misses this loop – and misses most errors that matter
- in the simulator we can model the environment as well:
  - **custom chips** in Wokwi: own parts written in C, with pins, I2C/SPI/UART, timers and controls on the part
  - the same firmware binary that would be flashed runs against the model – like a hardware-in-the-loop test bench, where the hardware is virtual as well

The setup: program, interfaces, environment

Custom chips: modelling the environment

- a chip = `name.chip.json` (pins, controls, display) + `name.chip.c`, compiled to WebAssembly – in the browser automatically, locally with `wokwi-cli chip compile`
- the chip API: edges (`pin_watch`), time steps (`timer_start`), analog pins (`pin_adc_read`, `pin_dac_write`), bus devices (`i2c_init`), controls on the part (`attr_read`)

```
static void on_step(void *user_data) {             // every 10 ms of simulated time
  chip_t *c = user_data;
  float duty = c->high_ns / (STEP_US * 1000.0f);   // heater PWM, measured from the edges
  float air = pin_adc_read(c->air) / 3.3f;         // from the fan chip
  float ambient = attr_read_float(c->ambient_attr);    // slider on the part
```

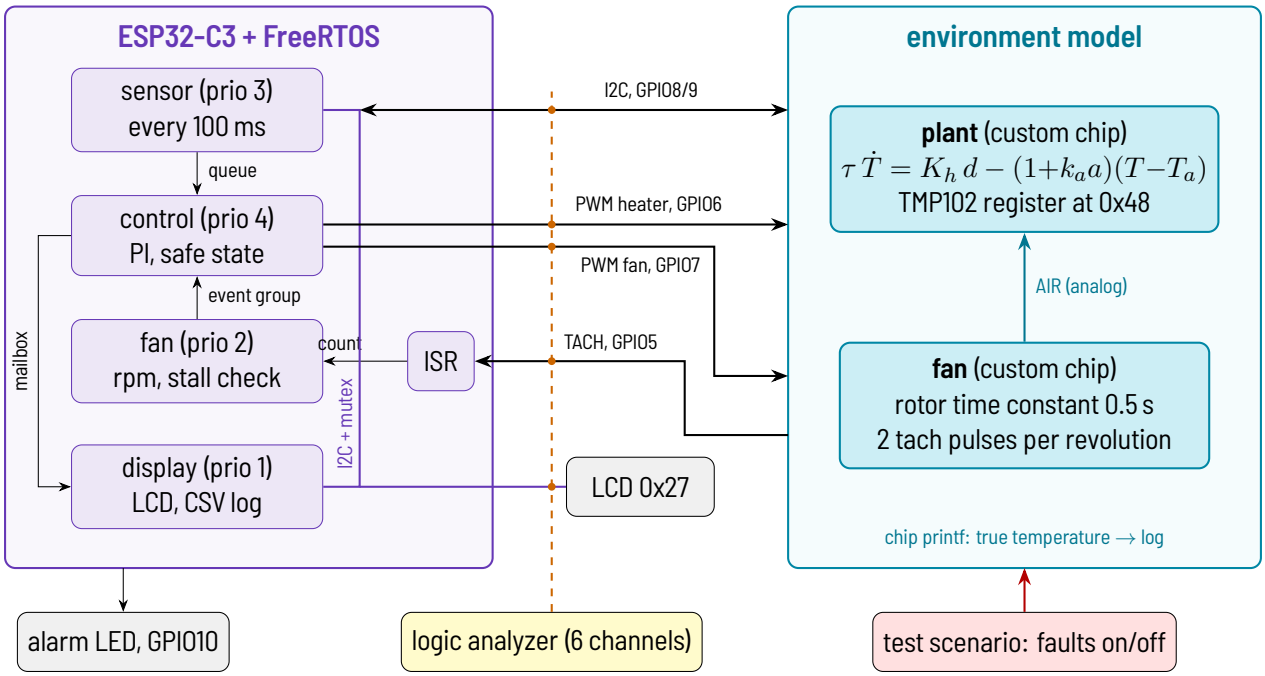


Figure 8.4.1: Climate chamber: the FreeRTOS program on the left knows the environment on the right only through its interfaces

```
  c->temp += dt / TAU_S * (K_HEAT * duty - (1 + K_AIR * air) * (c->temp - ambient));
}
static bool on_connect(void *user_data, uint32_t address, bool read) {   // I2C
  chip_t *c = user_data;
  if (attr_read(c->fault_attr)) return false;     // injected fault: NACK
  c->latched = c->pointer == 0 ? temp_register(c) : 0x60A0;   // as in the TMP102
  return true;
}
```

The program: four tasks

task	prio	job	communication
control	4	PI controller, safe state on faults	waits on queue (timeout 500 ms)
sensor	3	reads TMP102 every 100 ms	I2C under mutex, sends to queue
fan	2	rpm from tach pulses every 500 ms, stall check	ISR counter, event group
display	1	LCD and CSV log every 500 ms	mailbox (`xQueueOverwrite`), I2C under mutex

```
if (xQueueReceive(measurements, &m, pdMS_TO_TICKS(500)) == pdTRUE) {
  temp = m.temp;
  xEventGroupClearBits(faults, SENSOR_LOST);
} else {
  xEventGroupSetBits(faults, SENSOR_LOST);   // no value for 500 ms
}
EventBits_t f = xEventGroupGetBits(faults);
if (f) {                                     // safe state: heater off, full air
  heat = 0;  fan = 1.0;  integral = 0;
} else { /* PI controller */ }
```

The same program in Rust (Embassy)

```rust
static MEASUREMENTS: Channel<CriticalSectionRawMutex, f32, 4> = Channel::new();
type I2cBus = Mutex<NoopRawMutex, I2c<'static, Blocking>>;   // shared by sensor and LCD

loop {   // control loop in main: it owns the PWM channels and the alarm LED
    match with_timeout(Duration::from_millis(500), MEASUREMENTS.receive()).await {
        Ok(t) => { temp = t; set_fault(SENSOR_LOST, false); }
        Err(_) => set_fault(SENSOR_LOST, true),          // no value for 500 ms
    }
    let faults = FAULTS.lock(|f| f.get());   // critical section around a Cell
    let (heat, fan) = if faults != 0 { (0.0, 1.0) } else { /* PI controller */ };
    heater.set_duty_hw((heat * 1023.0) as u32);
    ...
}
```

- one executor, no priorities; the tach pulses are counted by a task that waits for the next edge (`wait_for_rising_edge`)
- own LCD driver (40 lines): an update takes 29 ms instead of 95 ms – and now it overlaps a sensor read: the trace shows the read **waiting 7.5 ms for the mutex**
- found in the simulator: the async I2C driver stopped the whole executor at the first transfer – the blocking driver works (one transfer $\leq$ 0.5 ms)

Looking at the interfaces: the logic analyzer

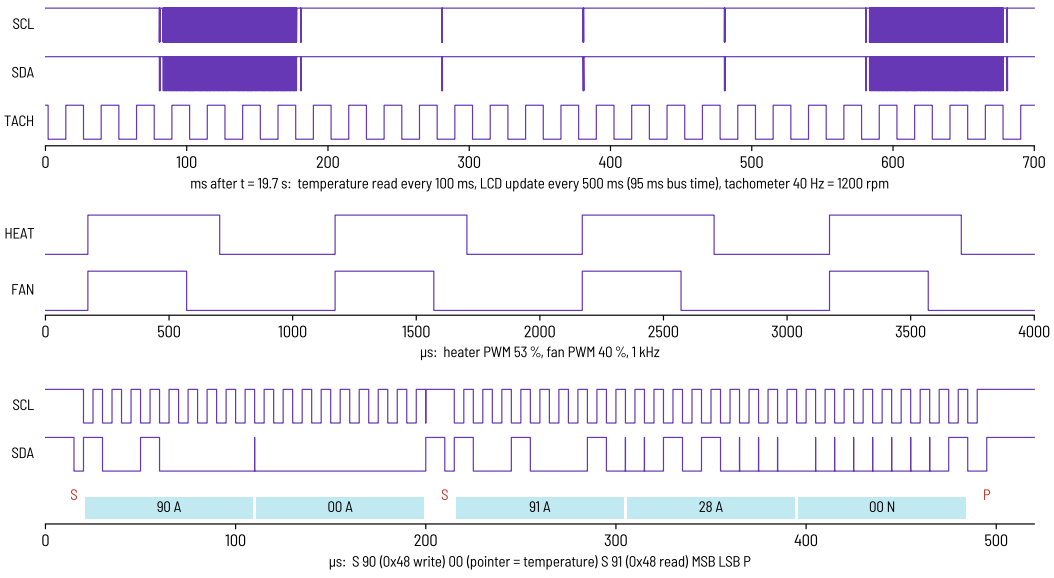


Figure 8.4.2: Recorded with `wokwi-cli --vcd-file`, decoded with `test/figures.py` (PulseView works as well)

What the trace tells us

- the sensor read comes every 100.0 ms – `xTaskDelayUntil` gives a fixed period, independent of the run time of the task

- one read takes 0.5 ms at 100 kHz: `S 90 00 S 91 28 00 P` = 0x280 · 0.0625 = 40.0 °C
- the LCD occupies the shared bus for 95 ms every 500 ms – without the mutex a sensor read inside this time would mix its bytes with the LCD's
- the LCD update starts 2.8 ms after a sensor read: if it took 3 ms longer, the sensor task would block on the mutex (priority inheritance keeps the delay short)
- heater PWM 53 %, fan 40 %, tach 40 Hz ⇒ 1200 rpm: what the program sets is what the environment receives

Fault injection with a test scenario

```
steps:
  - wait-serial: 't_ms,temp,heat,fan,rpm,faults'
  - delay: 20000ms
  - set-control: { part-id: fan,   control: stall, value: 1 }   # rotor blocked
  - delay: 10000ms
  - set-control: { part-id: fan,   control: stall, value: 0 }
  - delay: 8000ms
  - set-control: { part-id: plant, control: fault, value: 1 }   # sensor silent
  ...
```

- `wokwi-cli --scenario test/faults.yaml .` runs it without a browser – also in continuous integration
- the plant chip prints the **true** temperature, the program logs what it **measures** – in the simulator we can compare both

Result: the program reaches the safe state

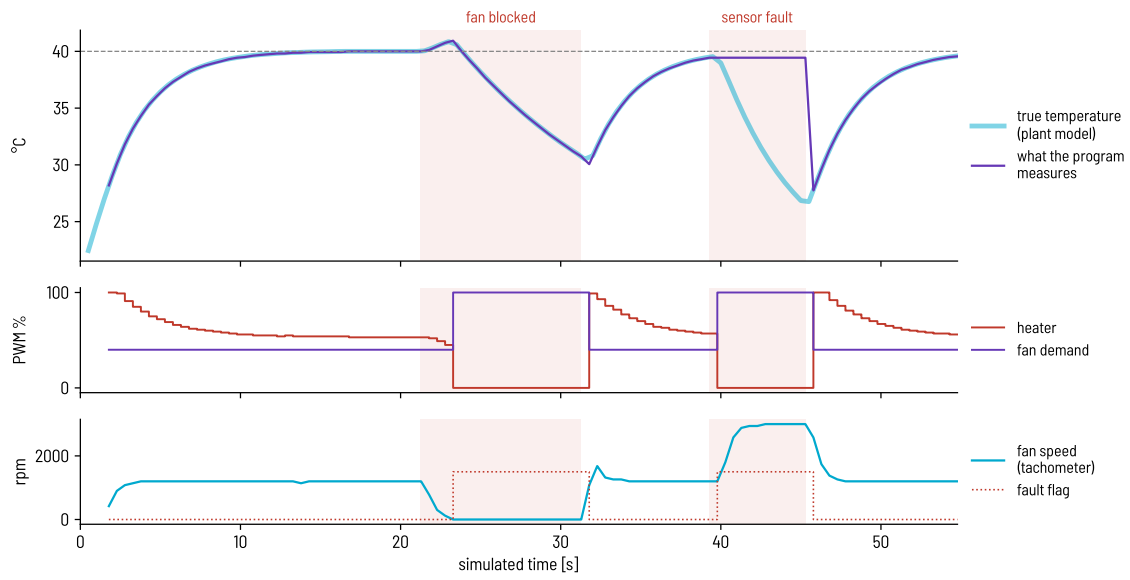


Figure 8.4.3: Blocked fan: alarm after 2.0 s (rotor slows down, then two slow 500 ms windows), heater off. Silent sensor: alarm after 0.5 s (queue timeout), heater off – the program shows the last value, the chamber cools down. Both faults clear by themselves.

The same fault without a return value check

```
Wire.endTransmission(false);                    // results ignored ...
Wire.requestFrom(TMP102, (uint8_t)2);
int16_t raw = (int16_t)(Wire.read() << 8 | Wire.read());    // no data: -1, -1
```

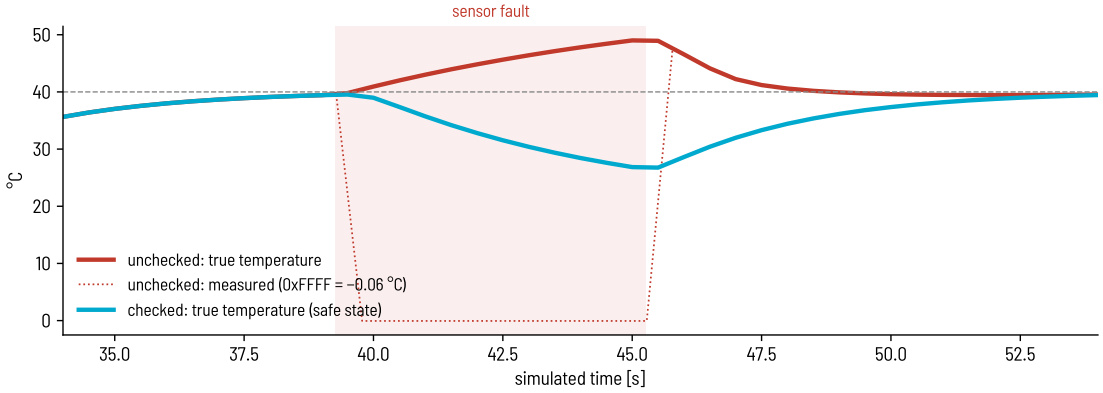


Figure 8.4.4: The program reads 0xFFFF = $-0.06\,^{\circ}\mathrm{C}$, drives the heater at 100 % and overheats the chamber – while its own log shows a cold chamber

Why simulate the environment?

- the program is only correct **together with** its environment: the second variant compiles, runs and logs plausible values – the error is visible only in the plant
- faults on demand: blocked fan, broken sensor wire, extreme ambient temperature – dangerous or expensive on real hardware, one slider in the simulator
- reproducible: the same scenario gives the same trace, so it can run in every build (regression test)
- ground truth: the model knows the real state, a real test bench needs extra instruments
- early: program and test exist before the hardware does
- but: a model is only as good as its assumptions – here no noise, no electrical faults, no sensor delay, ideal heater
  - validate the model against measurements of the real system, then use it
  - in industry the same idea in steps: model, software, processor and hardware in the loop (MiL, SiL, PiL, HiL)

Exercises: climate chamber

- set the ambient temperature slider to 0 °C. Why does the chamber stay below 40 °C? Compute the highest reachable temperature from the model equation. What should the program report in this case?
- add a third fault to the plant chip (the heater element breaks: no heating at any duty cycle) and extend the program so that it detects it. What is a sensible time limit?
- print the time the sensor task waits for the I2C mutex. Make the LCD output longer until the sensor read is delayed and find the delay in the logic analyzer.
- make the I2C sensor fail only for single reads (every tenth read NACKs). Does the program raise an alarm? Should it?

8.5 Operating Systems for Embedded Systems

FreeRTOS is only one of many options. The choice depends on the hardware (microcontroller or application processor with a memory management unit), on the required functions (network stack, file system, drivers), on certification needs, on the language and on the license. This section gives an overview. At the upper end, embedded Linux is used whenever the hardware is large enough and complex functions such as graphical user interfaces, many network protocols or containers are needed.

Real-time systems for microcontrollers

system	characteristics	license
FreeRTOS	small kernel, very widely used, basis of ESP-IDF	MIT
Zephyr	complete OS with drivers, networking, Bluetooth; device tree	Apache 2.0
Embassy	async Rust: executor, HALs, networking, USB, Bluetooth	MIT / Apache 2.0
RTIC	Rust: tasks = interrupt handlers, deadlock-free by design	MIT / Apache 2.0
Eclipse ThreadX	formerly Azure RTOS; certified for safety standards	MIT
NuttX	POSIX interface, "small Linux" for microcontrollers	Apache 2.0
commercial	VxWorks, QNX, embOS, SafeRTOS: certified, with support	commercial

- safety standards (IEC 61508, ISO 26262 for cars) require a certified kernel or a large verification effort

Embedded Linux

- needs an application processor with **MMU** and typically $\geq$ 64 MB RAM (Raspberry Pi, i.MX, STM32MP, AM62x)
- advantages: complete network stack, file systems, USB, graphics, thousands of existing programs, high level languages, containers
- disadvantages: boot time of seconds, much more memory and energy, larger attack surface, regular security updates necessary
- real time: the standard kernel gives no guarantees; **PREEMPT_RT** (part of the mainline kernel since 6.12, 2024) reduces latencies to some 10 μs
- common architecture: Linux for user interface and communication, plus a microcontroller (often on the same chip) running an RTOS for the hard real-time part

Booting embedded Linux

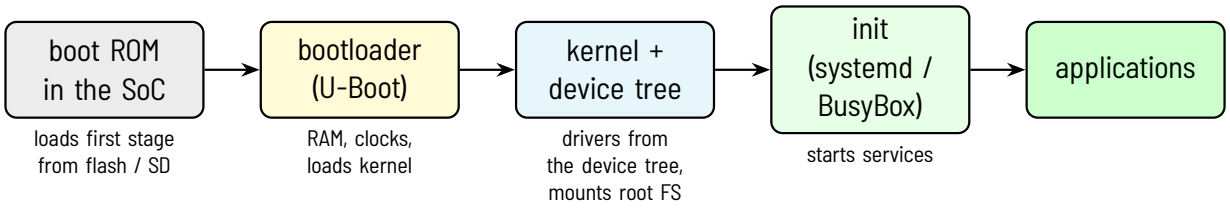


Figure 8.5.1: Boot chain of an embedded Linux system

- the **device tree** describes the hardware (which peripherals at which addresses, which pins) – the same kernel runs on different boards

- the **root file system** contains libraries, programs and configuration

Building embedded Linux: cross development

- the target is too small (or too slow) to compile its own software ⇒ **cross compiler** on the PC
  - named by the target triple, e.g. `aarch64-linux-gnu-gcc`
  - Rust: `rustup target add aarch64-unknown-linux-gnu` – the same way we built for the RP2040 and the ESP32-C3
- build systems create the complete image (toolchain, bootloader, kernel, root file system) from source
  - **Buildroot**: simple, configuration with `make menuconfig`, small images, quick to learn
  - **Yocto Project**: layers and recipes, reproducible, industry standard for products with long maintenance
- alternative for prototypes: a ready-made distribution (Raspberry Pi OS, Debian) and native compilation
- the output format is the same as on microcontrollers: **ELF** files with sections for code (`.text`), data (`.data`, `.bss`) and debug information

Embedded Linux without hardware

- Wokwi simulates microcontrollers, not Linux boards – for embedded Linux we use **QEMU**, an emulator for complete computers (ARM, RISC-V, x86)
- QEMU runs on any Linux PC or in a cloud environment such as GitHub Codespaces (free quota: 120 core-hours per month)
- the Yocto Project publishes ready-made images and SDKs for its emulated boards (`qemuarm64`, `qemuriscv64`, `qemux86-64`)

level	environment	effort
1: explore, compile, debug	JSLinux in the browser (Buildroot RISC-V, Alpine x86)	none
2: boot a Yocto image	QEMU + prebuilt image, PC or 2-core Codespace	30 MB download, 1–2 minutes
3: look inside Yocto	`bitbake-setup` + BitBake without building	0.5 GB, about 5 minutes
4: cross-compile and debug	Yocto SDK + QEMU + GDB	1 GB download, 5 GB disk
5: build an image	full Yocto build on a large machine	hours, 100 GB and more

- levels 1 to 4 were tested for this lecture; the files are in `examples/linux`

Exercise: an embedded Linux in the browser

- open https://bellard.org/jslinux/ and start "Buildroot Linux" (riscv64, console) – an emulated RISC-V computer in the browser, its software built with Buildroot
- explore the system

```
uname -a                 # kernel version and architecture
cat /proc/cpuinfo        # the emulated CPU (RISC-V, ISA extensions)
cat /proc/meminfo        # memory
ls -l /bin | head -n 15  # most commands are links to one program: BusyBox
ps                       # which processes run after the boot?
df -h                    # file systems and their size
dmesg | head -n 40       # the kernel messages of the boot
```

- questions: how many processes run? Which program is process 1? How large is the complete system compared with a desktop Linux?

Exercise: programming on the target, in the browser

- the Buildroot system in JSLinux contains GCC 7.3, `make`, `objdump` and `strace` – we compile directly on the (emulated) RISC-V target
- type `cat > procinfo.c`, paste the program from `examples/jslinux`, end with Ctrl-D

```
# gcc -g -O1 -o procinfo procinfo.c
# ./procinfo
uptime: 0 s                     <- wrong: the system has been running for a while
load:   0.13
sum:    126541
# strace ./procinfo 2>&1 | grep proc
openat(AT_FDCWD, "/proc/uptme", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/proc/loadavg", O_RDONLY) = 3
# objdump -d procinfo | less     # search for <sum16>: RISC-V code of chapter
    Processors
```

- `strace` shows every system call: here the typo in the file name, which the program hides
- in `objdump` find `lhu` (load halfword unsigned) and the conditional branch of the loop

Exercise: booting a Yocto image in QEMU

```
$ ./boot-yocto-qemu.sh       # examples/linux: download image, start QEMU
...
[    2.970384] Run /sbin/init as init process
qemuarm64 login: root
root@qemuarm64:~# uname -a
Linux qemuarm64 6.18.39-yocto-standard #1 SMP PREEMPT ... aarch64 GNU/Linux
root@qemuarm64:~# cat /proc/device-tree/model
linux,dummy-virt
root@qemuarm64:~# free
               total        used        free      shared  buff/cache   available
Mem:          231752       15992      201864          76       13896      209372
root@qemuarm64:~# df -h /
Filesystem                Size      Used Available Use% Mounted on
/dev/root                20.2M     13.5M      5.1M  73% /
root@qemuarm64:~# readlink /bin/sh /sbin/init
/bin/busybox.nosuid
/sbin/init.sysvinit
```

- `core-image-minimal`: 13.5 MB files, 16 MB RAM in use; quit QEMU with Ctrl-A, X
- the device tree in `/proc/device-tree` describes the emulated board

Exercise: looking inside Yocto without building

```
$ python3 -m venv venv && . venv/bin/activate && pip install bitbake-setup
$ bitbake-setup init --non-interactive poky-wrynose poky distro/poky machine/qemuarm64
$ . bitbake-builds/poky-wrynose/build/init-build-env
$ bitbake-layers show-layers                 # core, yocto, yoctobsp
$ bitbake -e core-image-minimal | grep -E '^(IMAGE_INSTALL|MACHINE|TUNE_FEATURES|
    DISTRO_VERSION)='
DISTRO_VERSION="6.0.4"
IMAGE_INSTALL="packagegroup-core-boot "
MACHINE="qemuarm64"
TUNE_FEATURES="aarch64 crc cortexa57"
$ bitbake-layers show-recipes busybox        # busybox 1.37.0 in layer meta
$ bitbake -g core-image-minimal               # dependency graph, no build
$ wc -l pn-buildlist
273 pn-buildlist                              # recipes needed for this small image
```

- only parses the recipes: about 5 minutes on 2 cores, 0.5 GB disk
- BitBake needs user namespaces: in a Codespace the provided dev container allows them; on Ubuntu 24.04 an AppArmor setting must be changed (see README)

Exercise: cross-compiling with the Yocto SDK

```
$ sh poky-glibc-...-cortexa57-qemuarm64-toolchain-6.0.3.sh -d ~/sdk
$ . ~/sdk/environment-setup-cortexa57-poky-linux     # sets CC, GDB, sysroot, ...
$ $CC -g -Og -o sensor_avg sensor_avg.c
$ file sensor_avg
sensor_avg: ELF 64-bit LSB pie executable, ARM aarch64, version 1 (SYSV)
$ qemu-aarch64 -L $SDKTARGETSYSROOT ./sensor_avg    # ARM program on the PC
average = 8 (expected 200)
```

- the SDK contains the cross compiler, GDB and a copy of the target's libraries – exactly matching the image
- **QEMU user mode** (`qemu-aarch64`) runs a single ARM program on the PC; **system mode** (`qemu-system-aarch64`) emulates the whole board
- on the emulated board, image `core-image-full-cmdline` with SSH:

```
$ scp -P 2222 sensor_avg root@localhost:/tmp/
$ ssh -p 2222 root@localhost /tmp/sensor_avg
average = 8 (expected 200)            # the bug: see chapter Programming
```

Building a complete image

- `bitbake core-image-minimal` builds the cross toolchain, the kernel and every package from source
- requirements from the Yocto documentation: at least 32 GB RAM and 140 GB disk; hours of CPU time
- the **shared state cache** reuses prebuilt results (configuration fragment `core/yocto/sstate-mirror-cdn`)

    - our test with the current branch: 273 of 2 287 tasks came from the mirror (11 %) – most still had to be built
- feasible in a large Codespace (16 cores, 64 GB RAM, 128 GB disk; uses the free quota in about 7 hours) or on a lab machine – a project, not a weekly exercise
- for comparison: Buildroot builds a small image with fewer features much faster – it has no shared state and no package management, but also far less to build

Choosing the right approach

approach	suitable for	limits
super loop + interrupts	small devices, simple timing	long jobs block everything
RTOS (FreeRTOS, Zephyr)	several jobs, different deadlines	stack per task, concurrency errors
async (Embassy)	event-driven, battery powered	long computations block
RTIC	hard real time, provable	interrupt-driven structure
embedded Linux	GUI, many protocols, complex software	boot time, memory, real time only with PREEMPT_RT

Exercises

- `tasks`: replace `xTaskDelayUntil` by `vTaskDelay` and add a 50 ms busy loop to the heartbeat. What happens to the period?
- `queue`: make the consumer slower than the producer. What happens when the queue is full? Change `portMAX_DELAY` to 0 and count lost samples.
- `race_mutex`: remove the `taskYIELD()`. Is the program correct now? Argue why not.
- `priority_inversion`: predict both waiting times before running the example, then compare with the measurement. Why is it 550 ms and not 750 ms? (Hint: how does `busy()` measure time?)
- Embassy: add a long computation without `.await` to one task of `tasks.rs` and observe the LED. Then split the computation with `embassy_futures::yield_now().await`.
- Deadlock: write a FreeRTOS example with two tasks and two mutexes that deadlocks, then fix it with a fixed locking order.
- Embedded Linux: boot `core-image-minimal` and `core-image-full-cmdline` in QEMU and compare the number of processes, the RAM and disk usage and the boot time. Which packages make the difference? (Hint: the image manifests on downloads.yoctoproject.org)
- Yocto: find out with `bitbake -e` which kernel version and which init system `core-image-minimal` uses, and how you would add the package `gdbserver` to the image.

Bibliography

[1] Peter Marwedel. *Embedded System Design: Embedded Systems Foundations of Cyber-Physical Systems, and the Internet of Things*. Springer, 4th edition, 2021.

[2] Edward A. Lee and Sanjit A. Seshia. *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*. MIT Press, 2nd edition, 2017.

[3] Edward A. Lee. Cyber physical systems: Design challenges. In *11th IEEE International Symposium on Object Oriented Real-Time Distributed Computing (ISORC)*, pages 363–369, 2008.

[4] David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, 1987.

[5] Daniel D. Gajski and Robert H. Kuhn. Guest editors' introduction: New VLSI tools. *Computer*, 16(12):11–14, 1983.

[6] C. L. Liu and James W. Layland. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, 20(1):46–61, 1973.

[7] Peter Fritzson. *Principles of Object-Oriented Modeling and Simulation with Modelica 3.3: A Cyber-Physical Approach*. Wiley-IEEE Press, 2nd edition, 2015.

[8] Joseph Yiu. *The Definitive Guide to ARM Cortex-M0 and Cortex-M0+ Processors*. Newnes, 2nd edition, 2015.

[9] David Patterson and Andrew Waterman. *The RISC-V Reader: An Open Architecture Atlas*. Strawberry Canyon, 2017.

[10] James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach*. Pearson, 8th edition, 2021.

[11] Raspberry Pi Ltd. RP2040 datasheet. https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf.

[12] Espressif Systems. ESP32-C3 technical reference manual. https://www.espressif.com/en/support/documents/technical-documents.

[13] Richard Barry. Mastering the FreeRTOS real time kernel – a hands-on tutorial guide. https://www.freertos.org.

[14] Rust Embedded Working Group. The embedded rust book. https://docs.rust-embedded.org/book/.

[15] Embassy project. Embassy book. https://embassy.dev/book/.

[16] Wokwi. Wokwi documentation. https://docs.wokwi.com.

[17] Stack Overflow. 2025 developer survey. https://survey.stackoverflow.co/2025/, 2025. 1.4

[18] DORA / Google Cloud. State of AI-assisted software development. https://dora.dev/research/publications/, 2025. 1.4

[19] METR. Measuring the impact of early-2025 AI on experienced open-source developer productivity. https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/, 2025. 1.4

[20] METR. We are changing our developer productivity experiment design. https://metr.org/blog/2026-02-24-uplift-update/, 2026. 1.4

[21] Veracode. 2025 GenAI code security report. https://www.veracode.com/blog/ai-generated-code-security-risks/, 2025. 1.4

[22] Anthropic. How AI assistance impacts the formation of coding skills. https://www.anthropic.com/news/AI-assistance-coding-skills, 2026. 1.4

[23] M. Babiuch and P. Smutný. Benchmarking large language models for embedded systems programming in microcontroller-driven IoT applications. *Future Internet*, 18(2):94, 2026. 1.4

[24] Ruiyang Xu, Jialun Cao, Mingyuan Wu, Wenliang Zhong, Yaojie Lu, Ben He, Xianpei Han, Shing-Chi Cheung, and Le Sun. EmbedAgent: Benchmarking large language models in embedded system development. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*, 2026. arXiv:2506.11003. 1.4

[25] World Economic Forum. The future of jobs report 2025. https://www.weforum.org/publications/the-future-of-jobs-report-2025/, 2025. 1.4

[26] PwC. Two futures for jobs in an AI era: 2026 global AI jobs barometer. https://www.pwc.com/gx/en/issues/artificial-intelligence/ai-jobs-barometer.html, 2026. 1.4

[27] E. Brynjolfsson, B. Chandar, and R. Chen. Canaries in the coal mine? six facts about the recent employment effects of artificial intelligence. https://digitaleconomy.stanford.edu/publication/canaries-in-the-coal-mine-six-facts-about-the-recent-employment-effects-of-artifici 2026. Stanford Digital Economy Lab, revised August 2026. 1.4

[28] Bitkom. Der arbeitsmarkt für IT-fachkräfte 2026. https://www.bitkom.org/Presse/Presseinformation/IT-Fachkraeftemangel-in-den-letzten-drei-Jahren-halbiert, 2026. 1.4

[29] European Union. Regulation (EU) 2024/1689 (artificial intelligence act), article 4: AI literacy. https://eur-lex.europa.eu/eli/reg/2024/1689/oj, 2024. 1.4

[30] ISO. ISO/PAS 8800:2024 road vehicles – safety and artificial intelligence. https://www.iso.org/standard/83303.html, 2024. 1.4

[31] Orrick. EU AI Act update: Digital omnibus finalizes 8 compliance changes. https://www.orrick.com/en/Insights/2026/07/EU-AI-Act-Update-Digital-Omnibus-Finalizes-8-Compliance-Changes, 2026. 1.4